



**Politechnika
Śląska**

PRACA MAGISTERSKA

System przeznaczony dla osób niewidomych do rozpoznawania dotykanych
obiektów 3D

Piotr MARCOL

Nr albumu: 300463

Kierunek: Informatyka

Specjalność: Oprogramowanie Systemowe

PROWADZĄCY PRACĘ

Dr hab. inż. Michał Maćkowski, prof. PŚ

KATEDRA Systemów Rozproszonych i Urządzeń Informatyki

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2026

Tytuł pracy

System przeznaczony dla osób niewidomych do rozpoznawania dotykanych obiektów 3D

Streszczenie

Celem niniejszej pracy było opracowanie systemu wspomagającego osoby niewidome i słabowidzące w nauce geometrii przestrzennej. Zaprojektowano i zaimplementowano prototyp systemu działający w architekturze klient-serwer, wykorzystujący aplikację mobilną oraz część serwerową. System umożliwia rozpoznanie widocznej powierzchni bryły oraz przekazanie informacji zwrotnej na jej temat. Przygotowano zbiór danych obejmujący sześć klas brył i przeprowadzono eksperymenty dla siedmiu konfiguracji modeli YOLO26, badając wpływ skali modelu, rozdzielczości i reprezentacji obrazu na skuteczność detekcji. Najlepszy kompromis uzyskano dla modelu YOLO26n wykorzystującego obrazy RGB o rozdzielczości 1024. Wyniki potwierdzają możliwość wykorzystania uczenia głębokiego w systemie wspomagającym naukę stereometrii.

Słowa kluczowe

Technologie asystujące, Detekcja obiektów, YOLO, Geometria przestrzenna, Osoby niewidome

Thesis title

A system designed for the blind to recognize touched 3D objects

Abstract

The aim of this thesis was to develop a system supporting blind and visually impaired people in learning spatial geometry. A prototype system operating in a client-server architecture was designed and implemented, using a mobile application and a server component. The system allows for the recognition of the visible surface of a solid and provides feedback about it. A dataset covering six classes of solids was prepared, and experiments were conducted for seven configurations of YOLO26 models, examining the impact of model scale, image resolution, and representation on detection accuracy. The best compromise was achieved for the YOLO26n model using RGB images with a resolution of 1024. The results confirm the possibility of using deep learning in a system supporting the learning of stereometry.

Key words

Assistive technologies, Object detection, YOLO, Spatial geometry, Visually impaired

Spis treści

1	Wstęp	1
1.1	Motywacja podjęcia tematu	1
1.2	Cel i zakres pracy	2
1.3	Wkład autora	2
1.4	Struktura pracy	2
2	Analiza problemu i przegląd literatury	5
2.1	Perspektywa osób niewidomych w rozpoznawaniu obiektów przestrzennych	5
2.2	Technologie wspomagające osoby z niepełnosprawnością wzroku	8
2.2.1	Rozwój technologii asystujących	8
2.2.2	Standardy i regulacje prawne dotyczące dostępności	9
2.2.3	Współczesne systemy wspomagające osoby niewidome	9
2.3	Problem rozpoznawania obiektów 3D w interakcji dotykowej	12
2.4	Metody detekcji obiektów w obrazach	13
2.4.1	Sformułowanie problemu detekcji obiektów	14
2.4.2	Klasyczne metody wizji komputerowej	15
2.4.3	Detektory dwuetapowe	15
2.4.4	Detektory jednoetapowe	16
2.4.5	Detektory oparte na architekturze transformera	17
2.4.6	Porównanie metod detekcji	17
2.5	Metryki oceny detekcji obiektów	19
2.6	Rodzina modeli YOLO	22
2.6.1	Architektura modeli YOLO	22
2.6.2	Warianty ze względu na realizowane zadanie	25
2.6.3	Warianty ze względu na skalę modelu	26
2.6.4	Dobór wariantów modelu	27
2.7	Wykorzystanie danych wizualnych i przestrzennych w rozpoznawaniu obiektów	28
2.7.1	Obraz RGB	28
2.7.2	Pozyskiwanie informacji o głębi	29

2.7.3	Dane LiDAR i chmury punktów	30
2.7.4	Fuzja danych wizualnych i przestrzennych	31
2.8	Podsumowanie analizy tematu	33
3	Koncepcja systemu rozpoznawania dotykanych obiektów 3D	35
3.1	Założenia funkcjonalne i нефункционалне	35
3.2	Architektura systemu	37
3.3	Dobór klas rozpoznawanych obiektów	39
3.4	Uzasadnienie wyboru metody detekcji	40
3.5	Przepływ danych w systemie	41
3.6	Ograniczenia przyjętej koncepcji	43
4	Implementacja systemu	45
4.1	Wykorzystane technologie i organizacja projektu	45
4.2	Implementacja aplikacji mobilnej	46
4.2.1	Organizacja kodu aplikacji	48
4.2.2	Rejestrowanie i przygotowanie obrazu	48
4.2.3	Zarządzanie transmisją	49
4.2.4	Obsługa interakcji głosowej	50
4.3	Implementacja części serwerowej	50
4.3.1	Organizacja i uruchamianie serwera	50
4.3.2	Odbieranie i przygotowanie obrazu	51
4.3.3	Moduł detekcji obiektów	52
4.3.4	Analiza wycinka i przygotowanie informacji zwrotnej	53
4.3.5	Organizacja przetwarzania współbieżnego	54
4.3.6	Interfejs graficzny	55
4.3.7	Obsługa błędów i cykl życia	56
4.4	Komunikacja klient-serwer	56
4.4.1	Nawiązywanie połączenia i wykrywanie serwera	57
4.4.2	Protokół i format przesyłanych danych	57
4.4.3	Przesyłanie klatek obrazu	58
4.4.4	Obsługa komunikatów sterujących	58
4.4.5	Cykl życia połączenia i obsługa błędów	59
4.5	Narzędzia pomocnicze	59
4.5.1	Ekstrakcja klatek z nagrań	60
4.5.2	Narzędzie do podziału zbioru danych	60
4.5.3	Narzędzia do treningu i testowania modeli	61
4.5.4	Filtry wejściowe i zapis wyników	62

5	Zbiór danych i metodyka badań	63
5.1	Zbiór danych	63
5.1.1	Charakterystyka zbioru danych	63
5.1.2	Proces pozyskiwania i oznaczania danych	64
5.1.3	Podział danych na zbiory treningowy, walidacyjny i testowy	66
5.1.4	Warianty danych wejściowych	67
5.2	Metodyka eksperymentalna	67
5.2.1	Konfiguracja treningu modeli	68
5.2.2	Metryki oceny	69
5.2.3	Procedura eksperymentalna	70
6	Wyniki badań i ich analiza	71
6.1	Zestawienie wyników treningów	71
6.2	Wpływ skali modelu na jakość detekcji	74
6.3	Wpływ rozdzielczości obrazu wejściowego na jakość detekcji	75
6.4	Wpływ reprezentacji obrazu na jakość detekcji	76
6.5	Porównanie najlepszych konfiguracji	78
6.6	Analiza błędów detekcji	80
6.7	Dyskusja wyników i ograniczeń badań	87
7	Podsumowanie i wnioski końcowe	91
7.1	Podsumowanie wykonanych prac	91
7.2	Ocena realizacji celu pracy	92
7.3	Najważniejsze wnioski	93
7.4	Możliwe kierunki rozwoju	94
	Bibliografia	104
	Spis skrótów i symboli	107
	Lista dodatkowych plików, uzupełniających tekst pracy (jeżeli dotyczy)	111
	Spis rysunków	113
	Spis tabel	115

Rozdział 1

Wstęp

1.1 Motywacja podjęcia tematu

Utrudnienia codziennego funkcjonowania osób niewidomych i słabowidzących zauważyć można każdego dnia. Wiele czynności, które dla osób widzących są proste i intuicyjne, dla osób z dysfunkcją wzroku wymagają zastosowania specjalnych metod i narzędzi. Korzystanie z transportu publicznego, płatności gotówkowe czy też zwykłe rozpoznawanie przedmiotów codziennego użytku stanowią dla nich poważne wyzwanie. Realia szkolne i zawodowe również nie są wolne od trudności. Nauka abstrakcyjnych pojęć matematycznych, szczególnie związanych z geometrią przestrzenną, wymaga od uczniów stosowania specjalnych metod nauczania, które ułatwiają im zrozumienie i przyswojenie wiedzy.

Przy nauce stereometrii, szczególnie użyteczne okazują się fizyczne modele brył, które dają możliwość poznania ich kształtu i właściwości poprzez dotyk. Jednakże same modele, mimo że umożliwiają dotykowe poznanie bryły, nie przekazują wszystkich informacji potrzebnych w procesie nauki. Często, aby uzyskać pełniejszy obraz, konieczne jest korzystanie z dodatkowych treści edukacyjnych, jak np. wydruki brajlowskie. Ich użycie może prowadzić jednak do przerywania procesu eksploracji i wydłużenia czasu pozyskiwania informacji, ponieważ wymaga od ucznia oderwania rąk od modelu i sięgnięcia po dodatkowe materiały.

Istniejące rozwiązania wspomagające osoby z dysfunkcją wzroku koncentrują się głównie na wspomaganiu użytkownika w codziennych czynnościach i przemieszczaniu się w przestrzeni. Istnieje jednak niewiele rozwiązań wspomagających naukę geometrii przestrzennej. Ten fakt stanowił główną motywację podjęcia tematu pracy – stworzenia rozwiązania, które wesprze osoby niewidome i słabowidzące w nauce geometrii przestrzennej i ułatwi im dostęp do informacji o poznawanych obiektach.

1.2 Cel i zakres pracy

Celem pracy jest opracowanie systemu wspomagającego osoby niewidome i słabowidzące w nauce geometrii przestrzennej za pomocą fizycznych modeli 3D, wykorzystującego metody detekcji obiektów na obrazie. Część badawcza pracy obejmuje przygotowanie i eksperymentalną ocenę użycia modeli uczenia głębokiego do wykrywania obiektów, skupiając się na ich skuteczności detekcji i czasie inferencji. W ramach analizy należy również ocenić wpływ wybranych parametrów modeli i reprezentacji danych wejściowych na wyniki detekcji.

Zakres pracy obejmuje przeprowadzenie analizy literatury, przygotowanie działającego prototypu systemu, własnego zbioru danych do trenowania modeli detekcji, wykonanie eksperymentów w celu oceny skuteczności wybranych modeli oraz opracowanie wniosków i rekomendacji dotyczących dalszego rozwoju systemu. Praca nie skupia się na opracowaniu nowych metod detekcji obiektów, a jedynie na dostosowaniu istniejących rozwiązań do wymagań projektowanego systemu. Zakres eksperymentalny obejmuje ocenę wpływu rozmiaru modelu, rozdzielczości obrazu wejściowego oraz reprezentacji obrazu na skuteczność detekcji.

Z powodu ograniczeń czasowych w zakresie pracy nie przewidziano badań z udziałem osób niewidomych i słabowidzących, objęcia całego zakresu nauczania stereometrii ani oceny wszystkich dostępnych metod detekcji obiektów.

1.3 Wkład autora

Wkład własny obejmuje część projektowo-implementacyjną oraz badawczą pracy. W ramach pierwszej z nich opracowano koncepcję i architekturę systemu oraz zaimplementowano aplikację mobilną i część serwerową. Przygotowano dla nich mechanizmy komunikacji i przesyłania danych, detekcji obiektów, ich analizy oraz generowania komunikatów głosowych. Opracowano również zestaw narzędzi pomocniczych do przygotowania danych, trenowania modeli i prowadzenia eksperymentów.

Część badawcza objęła przygotowanie własnego zbioru danych, opracowanie metodyki eksperymentów oraz trening i ocenę siedmiu konfiguracji modeli detekcji obiektów. Zbadano wpływ skali modelu, rozdzielczości obrazu wejściowego oraz jego reprezentacji na skuteczność detekcji. Następnie wyniki poddano analizie i wskazano konfigurację modelu, która najlepiej odpowiada przyjętym wymaganiom.

1.4 Struktura pracy

Pełny tekst pracy został podzielony na siedem rozdziałów. Każdy z nich obejmuje odrębny obszar tematyczny, a ich kolejność odpowiada przyjętej logice realizacji projektu.

W rozdziale 2 przedstawiono analizę literatury związanej z tematem pracy. Omówiono w nim sposoby poznawania obiektów przez osoby niewidome i słabowidzące, rozwój technologii asystujących oraz problem rozpoznawania obiektów przy interakcji dotykowej. Następnie przedstawiono przegląd metod detekcji obiektów, stosowane metryki oceny, rodzinę modeli YOLO (ang. *You Only Look Once*). Rozdział zakończony jest podsumowaniem zawierającym wnioski z analizy literatury wykorzystywane w dalszych etapach pracy.

Rozdział 3 przedstawia koncepcję działania i architekturę systemu. Omówiono w nim wymagania, sposób przepływu danych, funkcjonalności oraz ograniczenia systemu. Szczegóły implementacyjne zostały przedstawione w rozdziale 4, który opisuje proces implementacji aplikacji mobilnej i części serwerowej, sposób ich komunikacji oraz przygotowane narzędzia pomocnicze.

Część badawczą pracy rozpoczyna rozdział 5, w którym opisano sposób przygotowania własnego zbioru danych oraz jego charakterystykę. Zawarto w nim również metodykę eksperymentów. W rozdziale 6 zostały przedstawione wyniki treningów i testów modeli. Porównano w nim wpływ czynników na skuteczność detekcji oraz przeanalizowano błędy popełniane przez modele.

Podsumowanie pracy i wnioski z przeprowadzonych badań zawarto w rozdziale 7. Wskazano w nim najważniejsze wnioski, możliwe kierunki rozwoju oraz ocenę stopnia realizacji celu pracy.

Rozdział 2

Analiza problemu i przegląd literatury

Zagadnienie rozpoznawania obiektów przez osoby niewidome wymaga uwzględnienia nie tylko aspektów technicznych, ale też społecznych i psychologicznych – tego, jak użytkownik poznaje otoczenie i jakie informacje z niego wyciąga. Osoby widzące zazwyczaj nie mają problemów z rozpoznawaniem kształtów, kolorów, orientacji czy położenia obiektów w przestrzeni, natomiast osoby niewidome i słabowidzące w większym stopniu wykorzystują inne zmysły i technologie wspomagające, aby móc zbudować mentalną reprezentację otoczenia.

Ze względu na to, projektowanie i wykonanie systemu rozpoznawania obiektów nie może skupiać się wyłącznie na aspektach technicznych, ale należy również uwzględnić perspektywę użytkownika docelowego. Należy zrozumieć, w jaki sposób informacja przekazywana przez komputer może wesprzeć proces poznawania otoczenia. W niniejszym rozdziale przedstawiono perspektywę osób niewidomych w kontekście rozpoznawania obiektów przestrzennych, a następnie omówiono wybrane technologie wspomagające osoby z niepełnosprawnością wzroku, metody komputerowej detekcji obiektów, metryki oceny modeli i możliwość wykorzystania zróżnicowanych danych wejściowych, takich jak obrazy kolorowe RGB (ang. *Red-Green-Blue*), mapy głębi czy dane LiDAR (ang. *Light Detection and Ranging*). Szczególną uwagę poświęcono rodzinie modeli YOLO, wykorzystywanej w niniejszej pracy. Na koniec podsumowano przeprowadzoną analizę.

2.1 Perspektywa osób niewidomych w rozpoznawaniu obiektów przestrzennych

Przy projektowaniu systemu przeznaczonego dla osób niewidomych należy wziąć pod uwagę różnice w percepcji przestrzennej pomiędzy osobami widzącymi a niewidomymi.

W literaturze zagadnienie to jest analizowane zarówno z perspektywy technicznej, psychologicznej, jak i pedagogicznej. Badania nad percepcją osób niewidomych pozwalają lepiej zrozumieć ich sposób poznawania otoczenia. Costa et al. w swojej pracy, na podstawie studium przypadku, pokazują, że rozpoznawanie obiektu jest całym procesem stopniowego pozyskiwania i integrowania informacji. Zaznaczają, że w przypadku osób widzących spojrzenie zazwyczaj pozwala zrozumieć obiekt całościowo, natomiast w przypadku osób niewidomych poznanie odbywa się sekwencyjnie, etapami. Duże znaczenie mają więc cechy obiektu, które są wyraźnie dostępne w eksploracji dotykowej: prostota, regularność, krawędzie oraz faktura. Wspierają one budowę mentalnej reprezentacji przestrzennej obiektu. Osoba niewidoma może opierać ją nie tylko na aktualnej informacji dotykowej, ale również wcześniejszych doświadczeniach, kontekście i wiedzy o świecie. Autorzy ukazują, że brak wzroku nie oznacza prostego zastąpienia go przez inne zmysły – dotyk, słuch, pamięć i doświadczenia dostarczają informacji innego rodzaju oraz są wykorzystywane w odmienny sposób [5].

Rozwinięcie tej myśli znajdujemy w pracy autorstwa Figueiras i Arcavi, którzy pokazują jak osoba niewidoma uczy się matematyki i w jaki sposób komunikuje się z nauczycielem. Autorzy opisują przebieg lekcji prowadzonych przez niewidomego nauczyciela dla trzech uczniów, z czego dwóch było w pełni niewidomych, a jeden miał częściową utratę wzroku. Szczególnie istotne okazują się zapisy rozmów pomiędzy nauczycielem a uczniami, które ukazują wcześniej wspomniane etapowe poznawanie obiektu. Podczas omawiania stożka nauczyciel bazuje na wcześniejszych doświadczeniach i wiedzy uczniów, a także na ich wyobrażeniach przestrzennych.

W kolejnej części artykułu autorzy ukazują w jaki sposób nauczyciel prowadzi uczniów przez analizę wykresu funkcji kwadratowej. Nauczyciel celowo odracza rozwiązanie algebraiczne i prowadzi uczniów przez proces rozumowania. Początkowo uczestnicy rozważają położenie jednego z pierwiastków, następnie wierzchołka i osi symetrii. Dopiero po tym, gdy uczniowie przeprowadzili rozumowanie, nauczyciel pozwala na rozwiązanie algebraiczne. Pokazuje to, że w przypadku, gdy nie mamy dostępu do globalnej reprezentacji obrazu, uwypukla się znaczenie jawnych opisów relacji pomiędzy charakterystycznymi cechami obiektu, odwołania do wcześniejszych reprezentacji i uporządkowanie procesu rozumowania [14]. W kontekście projektowanego systemu sugeruje to, że sama informacja o klasie rozpoznanego obiektu nie jest wystarczająca, a zasadne może być również przekazywanie informacji o jego cechach charakterystycznych i zachodzących między nimi relacjach.

Odchodząc od ściśle teoretycznych i obserwacyjnych badań, w literaturze znajdujemy również przykłady artykułów naukowych, które bezpośrednio wskazują problemy osób niewidomych. Güler i Ürey przeprowadzili badanie jakościowe obejmujące 13 tureckich uczniów szkoły dla osób z niepełnosprawnością wzroku oraz ich 10 nauczycieli. Dotyczyło ono wyzwań i potrzeb uczniów przy nauce przedmiotów przyrodniczych. Istotnym

wnioskiem z dokumentu jest wskazanie, że nauczyciele częściej korzystali z podręczników pisanych alfabetem Braille’a oraz tabliczek i ryłców do zapisu brajlowskiego, aniżeli z materiałów przestrzennych, modeli 3D czy eksperymentów. Jeden z uczniów wskazywał, że ma problem z wyobrażaniem sobie reprezentacji przestrzennych słów, twierdząc, że gdy słyszy np. słowo ”piramida” nie potrafi zbudować w umyśle odpowiadającego mu obrazu. Nauczyciele wskazują również, że uczniowie dotknięci niepełnosprawnością wzroku nie mają przewidzianego osobnego toku nauczania. Są więc zmuszeni do konkurowania z widzącymi rówieśnikami tak, jakby byli w stanie wyuczyć się wszystkiego w ten sam sposób. Zaznaczają też, że zagadnienia związane z naukami przyrodniczymi są często abstrakcyjne, co utrudnia ich zrozumienie, a sam proces nauki powinien być dostosowany do potrzeb uczniów z niepełnosprawnością wzroku [16]. Podobne problemy związane z brakiem materiałów edukacyjnych, koniecznością dostosowania istniejących, wydłużonym czasem przygotowania zajęć i dostosowania komunikacji odnotowano również w badaniach dotyczących nauki języka angielskiego [2].

Kolejnym zagadnieniem dotyczącym perspektywy osób niewidomych, na które zwrócili uwagę Li et al., jest sposób, w jaki różne źródła informacji – dotyk, opis dźwiękowy, wcześniejsze doświadczenia – wspierają rozumienie obiektów. Zaznaczają, że osoby niewidome nie bazują jedynie na pojedynczym źródle informacji, ale łączą informacje pochodzące z różnych modalności. Badana grupa osób wskazała, że poprzez dotyk mogą zrozumieć kształt obiektu, jego położenie, formę i szczegóły przestrzenne, ale by w pełni zrozumieć kontekst, historię lub znaczenie obiektu, potrzebują dodatkowego opisu audio. Dodatkowo, autorzy zwracają uwagę na to, że osoby niewidzące różnią się między sobą pod względem preferencji i sposobów poznawania obiektów. Zaznaczają, że różnią się one w zależności od stopnia niepełnosprawności wzroku oraz tego, czy osoba jest niewidoma od urodzenia, czy utraciła wzrok w późniejszym okresie życia [21]. Na tej podstawie można przyjąć, że projektowany system nie powinien ograniczać się do jednego sposobu komunikacji, ale pozwolić na personalizację i dostosowanie do preferencji konkretnego użytkownika.

Przytoczone badania wskazują, że rozpoznawanie obiektów przez osoby niewidome nie jest pojedynczym aktem identyfikacji, ale etapowym procesem poznawczym. Eksploracja dotykowa pozwala na poznanie lokalnych części obiektu: krawędzi, powierzchni, faktury i orientacji, ale to wcześniejsze doświadczenia i przekaz słowny pozwalają na zbudowanie jego pełnej reprezentacji mentalnej. Jednocześnie, niewielka liczba dedykowanych materiałów edukacyjnych mocno utrudnia procesy nauczania – zwłaszcza przy pojęciach abstrakcyjnych.

Wynika z tego, że system wspomagający nie powinien ograniczać się jedynie do przekazywania nazw obiektów, ale wspierać użytkownika w całym procesie jego rozpoznawania. Informacja zwrotna powinna być jednoznaczna, możliwa do dostosowania do potrzeb użytkownika i wspierać go w naturalnym procesie eksploracji dotykowej. Te wnioski są

podstawą do analizy istniejących rozwiązań technologicznych przedstawionej w kolejnym podrozdziale.

2.2 Technologie wspomagające osoby z niepełnosprawnością wzroku

Analiza perspektywy osób niewidomych pozwala zauważyć, że odmienne sposoby poznawania przestrzeni wokół nich wiążą się z potrzebą wsparcia w codziennym funkcjonowaniu. Może ono przyjmować formę tradycyjnych rozwiązań, takich jak biała laska lub pies przewodnik, pomocy drugiej osoby, a także coraz bardziej zaawansowanej technologii asystującej, która ciągle rozwijana daje coraz lepsze możliwości.

2.2.1 Rozwój technologii asystujących

Naukowcy od dawna widzieli w rozwoju techniki okazję do wsparcia osób z jakąkolwiek niepełnosprawnością. Jednym z pierwszych urządzeń wspomagających niesłyszących był aparat słuchowy opracowywany przez Millera Reese'a Hutchisona od około 1895 roku [30]. Rozwiązaniem, które zapowiadało późniejszy rozwój elektronicznych systemów wspomagających był system POSSUM (ang. *Patient Operated Selector Mechanism*) opracowany początkiem lat 60. XX wieku. Umożliwiał on osobom z poważną niepełnosprawnością ruchu sterowanie maszyną do pisania [29]. Nie można jednak mówić o cyfrowych systemach wspomagających przed pojawieniem się elektronicznych komputerów. W latach 60. i 70. XX wieku zaczęły pojawiać się bardziej zaawansowane systemy wspomagające oparte o cyfrowe jednostki obliczeniowe. Przykładem jednego z nich może zostać Tufts Interactive Communicator, który umożliwiał niemym z ograniczeniami ruchowymi komunikację z otoczeniem poprzez wybieranie znaków za pomocą mechanizmu skanowania, a następnie prezentację pełnego komunikatu na wyświetlaczu lub w formie wydruku [55]. Późniejszy przykład podobnego systemu był wykorzystywany przez wybitnego fizyka Stephena Hawkinga, który pomimo swojej postępującej niepełnosprawności ruchowej, mógł sprawnie komunikować się z otoczeniem poprzez niewielkie ruchy policzka [18].

Równocześnie rozwijały się technologie wspomagające osoby z niepełnosprawnością wzroku, w których dane przekształcano na formę możliwą do odbioru przez inne zmysły. Kamieniem milowym okazało się urządzenie Optacon (ang. *Optical to Tactile Converter*) opatentowane w 1966 roku przez Johna G. Linvilla. Umożliwiała ono przekształcenie obrazu na vibracje matrycy, które mogły być odczytane przez użytkownika za pomocą dotyku [24]. W kolejnych dekadach rozwijano m.in. systemy rozpoznawania znaków, syntezy mowy, czytniki ekranów i urządzenia mobilne, stopniowo zwiększając dostęp osób niewidomych do informacji cyfrowych.

2.2.2 Standardy i regulacje prawne dotyczące dostępności

Rozwój technologiczny nie był jedynym czynnikiem, który przyczynił się do upowszechnienia systemów wspomagających. Istotną rolę odegrały również regulacje prawne i standardy, które nakładały obowiązek zapewnienia dostępności systemów informatycznych dla osób z niepełnosprawnością. Bardzo istotnym krokiem było opublikowanie w 1999 roku przez World Wide Web Consortium (W3C) pierwszej wersji wytycznych WCAG (ang. *Web Content Accessibility Guidelines*) określających, w jaki sposób powinny być projektowane treści internetowe, by były one dostępne dla osób z różnymi rodzajami niepełnosprawności [59]. Kolejne wersje WCAG uporządkowały wytyczne wokół czterech głównych zasad: postrzegalności, funkcjonalności, zrozumiałości i solidności (kompatybilności). Najnowsza wersja standardu WCAG 2.2 została opublikowana 5 października 2023 [60]. Obecnie trwają prace nad kolejną wersją WCAG 3.0.

Znaczenie dostępności zostało następnie zwiększone dzięki wdrożeniu kroków prawnych. Artykuł 9 Konwencji ONZ¹ o prawach osób z niepełnosprawnościami nakłada na państwa członkowskie obowiązek przystosowania dostępu do informacji, komunikacji, infrastruktury i technologii do potrzeb osób z niepełnosprawnością [54]. Unia Europejska wprowadziła w 2016 roku dyrektywę 2016/2102, która określa wymagania dotyczące dostępności stron internetowych i aplikacji mobilnych podmiotów sektora publicznego [11]. W Polsce wymagania wynikające z dyrektywy wdrożono ustawą z dnia 4 kwietnia 2019 roku o dostępności cyfrowej stron internetowych i aplikacji mobilnych podmiotów publicznych [44]. Od 28 czerwca 2025 roku w Polsce obowiązuje również Europejski Akt o Dostępności, który nakłada wymagania na wybrane usługi i produkty, takie jak urządzenia mobilne, komputery, czytniki książek, handel elektroniczny, bankowość i transport [12, 43]. Szczegółowe wymagania techniczne dla produktów i usług ICT określa norma EN 301 549, która jest stosowana w całej Unii Europejskiej [10].

Dla osób niewidomych te regulacje mają szczególne znaczenie w kontekście współpracy czytników ekranowych z interfejsami, możliwości obsługi bez użycia wzroku i jednoznacznego przekazu informacji. Rozwój prawa sprawił więc, że dostępność cyfrowa nie pozostała w fazie teoretycznej, ale stała się wymogiem, a przez to istotnym czynnikiem technologii wspomagających.

2.2.3 Współczesne systemy wspomagające osoby niewidome

Wraz z rozwojem technologii informatycznej i postępującym zwiększaniem dostępności cyfrowej zaczęły powstawać kompletne systemy wspomagające osoby niewidome, których rozmiary i możliwości zazwyczaj nie krępowały codziennego funkcjonowania. Dziś wiele z nich wykorzystuje uczenie maszynowe i kamery do analizy przestrzeni wokół użytkownika. Przykład ich użycia zaproponowali Potdar, Pai i Akolkar, którzy oparli swoją pracę

¹Organizacja Narodów Zjednoczonych

na konwolucyjnych sieciach neuronowych (CNN) i kamerze. Zaznaczyli, że jednym z ich celów było, aby osoba niewidoma nie musiała wchodzić w bezpośredni kontakt z obiektem przed sobą, aby zminimalizować ryzyko uszczerbku na zdrowiu lub utraty życia. Sieć zastosowana przez autorów zawierała warstwy odpowiadające za wyodrębnianie cech, lokalizację i klasyfikację obiektów. System umożliwia rozpoznawanie 200 klas obiektów obejmujących ludzi, pojazdy, zwierzęta i elementy wyposażenia pomieszczeń. Sens działania systemu jest prosty: użytkownik naciska przycisk, kamera rejestruje obraz, a następnie model CNN rozpoznaje obiekty i przekazuje użytkownikowi informacje o nich. Niestety, założenie o naciśnięciu przycisku wyklucza możliwość ciągłego rozpoznawania obiektów. Autorzy zaznaczają, że ich wyniki potwierdzają użyteczność systemu, ale jednocześnie mówią o utracie jakości przy małych obiektach. Sugerują, że zwiększenie liczby warstw konwolucyjnych może poprawić skuteczność, ale jednocześnie spowolni działanie systemu. Te wnioski pokazują, że należy zachować kompromis pomiędzy skutecznością a prędkością działania systemu [32].

Rozszerzeniem idei wykorzystania sieci neuronowych do wsparcia osób niewidomych jest system MagicEye opracowany przez zespół pod przewodnictwem Sethuramana. Tym razem twórcy oparli się na idei stworzenia urządzenia ubieralnego w formie inteligentnych okularów z kamerą. Podobnie jak w przypadku poprzedniego systemu, po wejściu w interakcję z urządzeniem obraz z kamery jest analizowany przez model sieci neuronowej, a następnie wyniki detekcji są przekazywane użytkownikowi w formie dźwiękowej. Model oparto na architekturze YOLOv5, wybranej ze względu na szybkość działania, skuteczność i niewielki rozmiar sieci. Trening przeprowadzono na wybranym podzbiórze Google Open Images liczącym ponad 13 200 obrazów, obejmującym 35 klas obiektów. Dodatkowo, w systemie zaimplementowano mechanizmy rozpoznawania twarzy, nominałów banknotów, odbiornik GPS² i czujnik zbliżeniowy [46].

Kolejnym krokiem w rozwoju technologii asystujących jest już nie tylko rozpoznanie obiektów, ale przekazanie użytkownikowi informacji dotyczących kontekstu całej sceny. Takie rozwiązanie znajdujemy w pracy Baiga et al., którzy zaproponowali system oparty na Raspberry Pi 4 i model sztucznej inteligencji. Dzięki temu, nie byli ograniczeni do skończonego zbioru klas obiektów, ale dzięki użyciu dużego modelu vision-language mogli generować opisy całej sceny i przekazywać je użytkownikowi w formie audio. Dodatkowo, w systemie zaimplementowano rozpoznawanie twarzy i detekcję odległości od przeszkód. Zgodnie z założeniami, system działał w czasie rzeczywistym, ale radził sobie znacząco gorzej w złych warunkach oświetleniowych [1]. Ta praca rozwiązuje problem poruszony w podrozdziale 2.1, mówiący o tym, że sama informacja o obiekcie może być niewystarczająca. Tutaj twórcy skupili się na przekazaniu jak najpełniejszej informacji o przestrzeni,

²Global Positioning System – globalny system pozycjonowania satelitarnego

w której znajduje się użytkownik. Warto jednak zauważyć, że skorzystanie z dużego modelu wymaga ciągłego połączenia z Internetem, a lokalne działanie systemu jest ograniczone do rozpoznawania twarzy i detekcji pobliskich przeszkód.

Poprzednio omawiane systemy skupiały się na rozpoznawaniu obiektów w określonej scenie, natomiast głównym celem systemu zaproponowanego przez Wang et al. jest pomoc użytkownikowi w omijaniu przeszkód w przestrzeni. Ponieważ rozwiązanie ma działać jako ciągła pomoc nawigacyjna, nie powinno być aktywowane przyciskami, a zamiast tego stale analizować obraz i – w razie potrzeby – informować użytkownika o potencjalnym zagrożeniu. Autorzy przedstawili metodę detekcji YOLO-OD³, do której wykorzystali model o architekturze inspirowanej siecią YOLOv8 rozszerzoną o dodatkowe bloki odpowiedzialne za zwiększenie skuteczności detekcji w różnych warunkach. Sam model na publicznie dostępnym zbiorze danych osiągnął wartość średniej precyzji na poziomie 30,02%, co zdaniem autorów wskazuje, że zaproponowane rozwiązanie może być wartościowe w kontekście wspomagania osób niewidomych [58].

Odrębnym zagadnieniem, na jakie należy zwrócić uwagę przy tworzeniu systemów wspomagających, jest sposób przekazywania informacji. Wcześniej omawiane rozwiązania bazowały na prostym przekazie dźwiękowym, opartym o buzzer lub syntezytor mowy. Dzierzgowska et al. zaproponowali metodę audio-graficzną, łączącą interaktywną prezentację wykresów funkcji matematycznych z opisami dźwiękowymi. Użytkownik, aby uzyskać kolejne informacje o wybranej części wykresu, musiał wykonać pojedyncze, podwójne lub potrójne stuknięcie na elemencie. Badanie przeprowadzone na grupie 54 osób wskazało, że zaproponowana metoda umożliwiła zmniejszenie obciążenia, frustracji, a w przypadku części zadań pozwoliła na skrócenie czasu ich wykonania. Wyniki wskazują, że skuteczność systemu wspomagającego nie zależy jedynie od stopnia możliwości rozpoznawania obiektów, ale również od poprawnego i odpowiedniego zaprojektowania przekazu informacji zwrotnej [9].

Współczesne technologie wspomagające uzupełniają tradycyjne metody wsparcia osób niewidomych, oferując coraz bardziej złożone systemy i algorytmy. Wspólnym mianownikiem większości z nich jest wykorzystanie małych kamer do pozyskiwania obrazu, który następnie jest analizowany przez modele uczenia maszynowego, a wyniki są prezentowane użytkownikowi. Większość opisywanych systemów koncentruje się jednak na wsparciu w codziennym funkcjonowaniu – poruszaniu się w przestrzeni, wykrywaniu przeszkód czy rozpoznawaniu twarzy i banknotów. Niewiele systemów skupia się jedynie na obiektach aktualnie eksplorowanych dotykowo przez użytkownika. W tym kontekście stworzenie systemu dedykowanego nauce przestrzennej wydaje się być istotnym uzupełnieniem istniejących rozwiązań.

³You Only Look Once–Obstacle Detection – model detekcji przeszkód przeznaczony do wspomagania nawigacji osób z niepełnosprawnością wzroku

2.3 Problem rozpoznawania obiektów 3D w interakcji dotykowej

Problemem, jaki należy koniecznie poruszyć, jest rozpoznanie obiektów 3D w interakcji dotykowej, ponieważ zachodzi w tym miejscu istotna różnica od systemów opisywanych we wcześniejszym podrozdziale (2.2.3). W odróżnieniu od nich, głównym celem projektowanego systemu nie jest rozpoznanie obiektów przed użytkownikiem, a tych, z którymi wchodzi w interakcję. Mamy więc do czynienia z sytuacją, w której kamera nie jest zwrócona na to, co znajduje się przed użytkownikiem, ale na niego samego, a w szczególności na jego dłoń. W tym kontekście detekcja staje się bardziej wymagająca ze względu na to, że spora część obiektu może być przez nie zasłonięta, a sam kształt może być trudny do rozpoznania nawet dla osoby widzącej. W tym miejscu warto przytoczyć kilka prac, które poruszają podobne zagadnienia.

Pierwszą z nich jest badanie przeprowadzone przez Shan et al., którzy sprawdzali zdolność modelu do rozpoznawania dłoni w kontakcie z obiektami. W tym celu przygotowali zbiór danych ponad 100 tys. obrazów, na których oznaczono ponad 180 tys. dłoni i ponad 110 tys. obiektów. Celem zaproponowanego modelu nie było jednak określenie klasy dotykane go przedmiotu, a jak najlepsze wskazanie pozycji dłoni i jej orientacji. Dzięki temu, możliwe było wyznaczenie nie tyle położenia przedmiotu, a formy relacji, w jaką wchodził z dłonią. Detekcja została oparta o model Faster R-CNN, który dla pełnej predykcji wymagającej poprawnego wykrycia dłoni i obiektu uzyskał 38,5 AP50 [47]. Pokazuje to, że w przypadku wykrywania trzymany obiektów należy zwrócić uwagę na ich otoczenie, gdyż może dojść do sytuacji, gdy model poprawnie rozpozna obiekt, ale nie będzie on trzymany przez użytkownika, co może prowadzić do błędnych wyników.

Kolejnym aspektem na jaki należy zwrócić uwagę jest możliwość zasłonięcia części obiektu przez dłoń użytkownika. W odróżnieniu od poprzedniej pracy, Zhang et al. skupili się na rekonstrukcji trójwymiarowej przedmiotów trzymany w dłoniach. Podczas uczenia wykorzystywali dane z wielu widoków, aby ograniczyć wpływ zasłaniania obiektu przez dłoń. Autorzy wskazali dwa źródła niepełności informacji: zasłonięcie części przedmiotu przez dłoń lub niewidoczność części obiektu wynikająca z obserwowania go tylko pod jednym kątem. Wskazuje to, że w pojedynczej klatce nawet dobrze widoczny przedmiot może nie ujawniać wszystkich swoich cech [63]. Przez to istotne jest, aby zbiór danych treningowych zawierał bardzo zróżnicowane kombinacje ułożenia dłoni i obiektu. Warto zwrócić uwagę na to, że możliwa jest również analiza obiektu z kilku ujęć, co może pozwolić na odtworzenie jego widocznej części w przestrzeni trójwymiarowej. W literaturze znajdujemy również propozycje analizy sekwencji obrazów, która pozwoli na ograniczenie problemu zasłaniania obiektu, a możliwe, że również wykluczy niektóre przypadki błędnej predykcji. Jedną z prac poruszających ten problem jest badanie zespołu pod przewodnictwem Tekina. Zaproponowali oni system analizujący sekwencje obrazów RGB rejestrowanych

przez użytkownika z perspektywy pierwszej osoby. Model jednocześnie określa położenie dłoni i obiektu, rozpoznaje jego klasę i wykonywaną czynność. Kolejne klatki są następnie analizowane przez moduł rekurencyjny, badający zmiany położenia wykrywanych dłoni i obiektów. Autorzy zaznaczają, że takie podejście pozwoliło im na poprawę skuteczności rozpoznawania czynności z 85,56% do 96,99% w porównaniu z analizą pojedynczych klatek. System działał również w czasie rzeczywistym z prędkością około 25 klatek na sekundę [50]. Wskazuje to, że wykorzystanie informacji z kolejnych klatek może stanowić sposób ograniczenia wpływu chwilowych zasłonień obiektów.

Na problem rozpoznawania obiektów można również spojrzeć z perspektywy robotyki. Możliwa jest eksploracja obiektu bez użycia kamer, a jedynie poprzez dotyk. Mimo że w tym przypadku nie mamy bezpośredniego odniesienia do osób niewidomych, to warto zwrócić uwagę na inne sposoby akwizycji danych. Ramię robota stworzonego przez zespół Xu et al. wyposażono w czujnik dotyku, a następnie umożliwiono mu eksplorację 10 przedmiotów ze zbioru YCB⁴. Każde dotknięcie tworzyło wpis w lokalnej chmurze punktów, która następnie była wykorzystywana do predykcji lub określenia kolejnego ruchu pozwalającego na lepsze rozróżnienie przedmiotów. Autorzy wskazali, że już kilka charakterystycznych punktów pozwalało na odróżnienie obiektów [61]. Sugeruje to, że kamery nie są jedynym sposobem na pozyskanie informacji, a ciekawym kierunkiem badań może być analiza chmury punktów powstałej w wyniku dotyku lub skanowania przestrzeni. Odpowiednikiem wspomnianego robota może być rękawica sensoryczna, która będzie zbierała dane nie tylko o samym obiekcie, ale również o rękach użytkownika. Zhang et al. opracowali właśnie takie rozwiązanie, które na podstawie zbieranych danych mogło rozpoznać 16 klas obiektów z dokładnością ponad 95% [64]. Ukazuje to, że do rozpoznania obiektów nie musimy wykorzystywać wyłącznie obrazów, a możliwe jest połączenie różnych źródeł danych, takich jak obraz, głębia czy informacje dotykowe. Jednakże rozwiązania wykorzystujące czujniki dotykowe wymagają dodatkowego wyposażenia, które przez utworzenie dodatkowej warstwy na dłoniach użytkownika może utrudniać mu eksplorację obiektu. W dalszej analizie skoncentrowano się na sposobach pozyskiwania danych możliwych za pomocą urządzeń, które nie wchodzi w bezpośredni kontakt z użytkownikiem.

2.4 Metody detekcji obiektów w obrazach

Mimo że dzisiejsza technika detekcji obiektów w obrazach opiera się głównie na sieciach neuronowych, istnieją różne podejścia do tego problemu. Należy wspomnieć o klasycznych metodach wizji komputerowej, wykorzystujących cechy samego obrazu, takie jak krawędzie, kolory czy histogramy [67]. W niniejszej sekcji przedstawiono przegląd najważniejszych metod detekcji obiektów.

⁴Yale–Carnegie Mellon University–Berkeley – zbiór obiektów i modeli wykorzystywany w badaniach nad manipulacją robotyczną

2.4.1 Sformułowanie problemu detekcji obiektów

Detekcja jest zadaniem trudniejszym od klasyfikacji obrazu, ponieważ oprócz określenia, jakie obiekty znajdują się na obrazie, musi również wskazać ich położenie. Sama klasyfikacja opiera się na przypisaniu jednej lub kilku etykiet do całego obrazu, co w wielu przypadkach może być niewystarczające. Lokalizacja rozszerza klasyfikację o wyznaczenie ramki ograniczającej (ang. *bounding box*) określającej położenie obiektu, najczęściej zakładając pojedynczą instancję. Zadaniem detekcji jest więc odnalezienie wszystkich istotnych obiektów na obrazie, a następnie przypisanie odpowiadających im klas i określenie ich położenia [39]. Wynik działania detektora może być przedstawiony jako zbiór predykcji:

$$D = \{(c_i, b_i, p_i)\}_{i=1}^N, \quad (2.1)$$

gdzie N jest liczbą wykrytych obiektów, c_i to przewidywana klasa i -tego obiektu, b_i opisuje jego ramkę, a p_i jest wartością pewności predykcji.

Ramka ograniczająca najczęściej jest prostokątem opisanym współrzędnymi dwóch przeciwległych narożników lub częściej współrzędnymi środka, szerokości i wysokości:

$$b_i = (x_i, y_i, w_i, h_i), \quad (2.2)$$

gdzie x_i i y_i określają położenie środka ramki, a w_i i h_i jej szerokość oraz wysokość. Zależnie od implementacji każda z tych wartości może być wyrażona w pikselach lub znormalizowana względem rozmiaru obrazu. Drugie podejście uniezależnia reprezentację ramki od rozdzielczości obrazu wejściowego i jest powszechnie stosowane przy adnotacjach i predykcji.

Wartość pewności predykcji, w zależności od implementacji, określa stopień przekonania modelu, że dany obszar zawiera obiekt przewidywanej klasy. Jeżeli ta wartość nie przekracza przyjętego progu, wynik wykonanej predykcji może zostać odrzucony. Jest to więc istotny parametr, jednak nie przesądza on poprawności predykcji. Przy ocenie należy również spojrzeć na zgodność klasy przewidywanej ze stanem faktycznym oraz na stopień pokrywania się ich ramek ograniczających (ang. *bounding box*). Zagadnienia te są szczegółowo opisane w podrozdziale 2.5, dotyczącym metryk oceny detekcji.

Zadaniem powiązaniem z detekcją obiektów jest segmentacja instancji, której celem oprócz wyznaczenia ramki i klasy każdego obiektu jest określenie ich masek pikselowych. Umożliwia to dokładne określenie kształtu obiektów, ale wymaga przygotowania dokładniejszych danych uczących [17]. W przypadku projektowanego systemu zadaniem modelu jest wykrycie trzymanej bryły geometrycznej, określenie jej klasy, położenia oraz wartości pewności predykcji. Istotna przy tym jest nie tylko dokładność predykcji, ale szczególnie szybkość działania, gdyż wynik ma być przekazywany użytkownikowi w momencie jego

bezpośredniej interakcji z obiektem. W tym kontekście segmentacja instancji może nałożyć za duże wymagania obliczeniowe, a jej zastosowanie może nie wiązać się z istotną poprawą jakości.

2.4.2 Klasyczne metody wizji komputerowej

Zanim w detekcji obiektów upowszechniły się głębokie sieci neuronowe, wykorzystywano przede wszystkim ręcznie projektowane metody przetwarzania obrazów. Między innymi wykorzystywano progowanie, segmentację na podstawie koloru, detekcję krawędzi i analizę konturów. Pozwalało to na wyodrębnienie obszarów różniących się od tła i opisanie ich cech, takich jak rozmiar, proporcje czy kształt. W kontrolowanych warunkach te metody mogły być skuteczne, jednak obiekty musiały wyraźnie odróżniać się od tła.

Późniejsze, bardziej rozbudowane podejścia wykorzystywały deskryptory cech, jak histogramy zorientowanych gradientów (HOG) czy skalo-niezmienne przekształcenie cech (SIFT) [7, 27]. Deskryptory obliczano dla całego obrazu lub kolejnych jego fragmentów za pomocą przesuwanego okna, a następnie wyodrębnione cechy przekazywano do klasyfikatora, którym mogła być maszyna wektorów nośnych (SVM). Warto również zwrócić uwagę na kaskady klasyfikatorów wykorzystujące cechy podobne do cech Haara, pozwalające na szybkie wykrywanie obiektów, głównie wykorzystywane w metodzie Viola–Jonesa [56].

Największe zalety metod klasycznych ujawniają się w ich prostocie, stosunkowo niskim koszcie obliczeniowym oraz możliwości interpretacji kolejnych etapów działania. Niestety, wymagają one ręcznego wyboru cech i ciągłego dostosowywania parametrów, aby uzyskać jak najlepsze wyniki. Są również bardzo wrażliwe na zmieniające się warunki otoczenia – niewielkie zmiany w oświetleniu, tle czy częściowe zasłonięcie obiektu mogą skutkować krytycznym spadkiem skuteczności. Te ograniczenia uzasadniają wykorzystanie głębokich sieci neuronowych, które w wielu przypadkach radzą sobie lepiej i mogą same, automatycznie uczyć się cech z danych treningowych.

2.4.3 Detektory dwuetapowe

Z czasem rozwój metod uczenia głębokiego doprowadził do coraz częstszego zastępowania ręcznie projektowanych deskryptorów cech przez ich reprezentacje automatycznie określone przez CNN. Jedną z rodzin takich rozwiązań są detektory dwuetapowe. Ich działanie podzielone jest na dwa główne etapy: wyznaczenie obszarów zainteresowań, a następnie ich klasyfikacja i dopasowanie położenia ramek ograniczających.

Jednym z pierwszych detektorów dwuetapowych był model R-CNN wykorzystujący algorytm do wyznaczenia propozycji regionów, które następnie kolejno były przetwarzane przez sieć konwolucyjną. Niestety, prowadziło to do wielokrotnego sprawdzania tych samych fragmentów obrazu przez sieć, co skutkowało dużym kosztem obliczeniowym. Kolejna wersja, Fast R-CNN, wprowadziła jednokrotne przetwarzanie całego obrazu przez

sieć, a cechy były pobierane ze wspólnej mapy cech. Ograniczyło to liczbę wykonywanych obliczeń, ale wciąż generowano propozycje regionów za pomocą algorytmu. Dalszym rozwinięciem było wprowadzenie wersji Faster R-CNN, w którym zastąpiono algorytm modułem sieci propozycji regionów RPN (ang. *Region Proposal Network*), która analizuje mapę cech i wskazuje obszary zawierające potencjalne obiekty. Oba moduły korzystały ze wspólnych cech obrazu, co pozwoliło znacznie ograniczyć koszt obliczeniowy [41].

Detektory dwuetapowe mogą osiągać wysoką skuteczność i dokładność lokalizacji dzięki rozdzieleniu od siebie etapu generowania propozycji regionów od ich klasyfikacji i dopasowania ramek ograniczających. Niestety, ten podział wpływa na złożoność obliczeniową całego modelu, a w konsekwencji na jego czas działania. W wielu przypadkach może stanowić to istotną przeszkodę, szczególnie jeżeli wynik ma być otrzymywany w czasie rzeczywistym.

2.4.4 Detektory jednoetapowe

Ostatnia dekada przyniosła znaczny rozwój detektorów jednoetapowych, których rdzeń działania opiera się na przewidywaniu klas obiektów i ich ramek ograniczających w pojedynczym kroku. Za pierwszy nowoczesny przykład takiego podejścia uznaje się OverFeat z 2013 roku [45], jednakże dopiero modele wprowadzone po roku 2015 zyskały wysoką popularność i rozpoznawalność. Mimo że historycznie detektory jednoetapowe oferowały mniejszą dokładność od detektorów dwuetapowych, ich główną zaletą była szybkość działania, co umożliwiło ich wykorzystanie w systemach czasu rzeczywistego (ang. *Real-Time*, RT) [22, 67].

Jednym z pierwszych znaczących zaproponowanych modeli jest SSD (ang. *Single Shot MultiBox Detector*) zaprezentowany na konferencji ECCV⁵ w 2016 roku przez Wei Liu. Jego działanie opiera się na przewidywaniu klas obiektów i położenia ramek ograniczających na kilku mapach o różnej rozdzielczości. Większe mapy umożliwiają wykrywanie mniejszych obiektów, a głębsze o mniejszej rozdzielczości służą rozpoznawaniu obiektów większych. Model wykorzystuje domyślne ramki o różnych proporcjach i skalach, których współrzędne następnie koryguje. Taki mechanizm umożliwia wykrycie obiektów o różnych rozmiarach bez osobnego etapu generowania propozycji regionów [26]. Głównym ograniczeniem modelu SSD stała się zależność skuteczności detekcji od odpowiedniego doboru ramek, a także spadek dokładności przy wykrywaniu małych obiektów.

W podobnym czasie został zaprezentowany również model YOLO, w którym zadanie detekcji przedstawiono jako pojedynczy problem regresji. Takie podejście doprowadziło do modelu, który przewiduje ramki ograniczające oraz prawdopodobieństwa klas w oparciu bezpośrednio o obraz wejściowy [39]. Uproszczenie i ujednolicenie procesu umożliwiło

⁵European Conference on Computer Vision

uzyskanie krótkiego czasu działania, co z kolei sprawiło, że modele YOLO znalazły zastosowanie w systemach czasu rzeczywistego. Kolejne lata przyniosły rozwój rodziny modeli YOLO, w której znalazły się warianty o różnej liczbie parametrów i złożoności obliczeniowej, umożliwiając wybranie modelu dopasowanego do wymagań i ograniczeń systemu. Szczegółowy opis rodziny modeli YOLO znajduje się w podrozdziale 2.6.

2.4.5 Detektory oparte na architekturze transformera

Podjęciem alternatywnym do detektorów opartych na sieciach konwolucyjnych są modele oparte na architekturze transformera. Podstawowym elementem tego podejścia jest mechanizm samo-uwagi (ang. *self-attention*), który pozwala na analizę zależności pomiędzy różnymi fragmentami danych wejściowych. Podczas działania modelu Vision Transformer obraz wejściowy jest dzielony na mniejsze fragmenty, które następnie są reprezentowane jako sekwencja wektorów i przetwarzane przez warstwy transformera. Takie podejście pozwala na znajdowanie relacji pomiędzy odległymi od siebie elementami obrazu. Użycie transformerów wizyjnych wymaga również przygotowania dużych zbiorów danych treningowych tak, aby model mógł nauczyć się zależności globalnych [8].

Pierwszą propozycją zastosowania tego podejścia w detekcji obiektów jest model DETR (ang. *Detection Transformer*) zaprezentowany przez zespół będący częścią Facebook AI Research pod kierownictwem Cariona w 2020 roku. Jego podstawowa wersja opiera się na wyznaczeniu podstawowych cech przez CNN, a następnie przekazaniu ich do transformera zrealizowanego w architekturze enkoder-dekoder. W dekodzie wykorzystywany jest zbiór zapytań obiektowych (ang. *object queries*), na których podstawie model dokonuje równoległej predykcji klas obiektów i ich ramek ograniczających. Wynik detekcji jest traktowany przez model jako zbiór predykcji, które w trakcie uczenia są dopasowywane do obiektów referencyjnych poprzez dopasowanie dwudzielne (ang. *bi-partite matching*) przy pomocy algorytmu węgierskiego. Pozwala to na uniknięcie użycia ramek domyślnych i algorytmu NMS (ang. *Non-Maximum Suppression*). W stosunku do klasycznych detektorów, DETR upraszcza potok detekcji, co pozwala na jego uczenie jako pojedynczego rozwiązania end-to-end. Niestety zalety płynące z użycia tego podejścia w detekcji wiążą się z wysokim kosztem treningu i trudniejszą optymalizacją modelu, szczególnie w przypadku wykrywania małych obiektów [3].

2.4.6 Porównanie metod detekcji

Przytoczone metody detekcji obiektów różnią się od siebie przede wszystkim sposobem wyznaczania położenia obiektów na obrazie, złożonością obliczeniową oraz ich stosunkiem pomiędzy dokładnością a szybkością działania. Zestawienie ich najważniejszych właściwości, ważnych z punktu widzenia systemu, przedstawiono w tabeli 2.1.

Tabela 2.1: Porównanie wybranych metod detekcji obiektów

Metoda	Charakterystyka	Atuty	Ograniczenia
Metody klasyczne	Ręcznie projektowane cechy i osobne etapy przetwarzania	Niewielkie wymagania obliczeniowe, interpretowalność i możliwość działania w kontrolowanych warunkach	Konieczność ręcznego określania cech, wrażliwość na zmianę oświetlenia, tła, orientacji i częściowe zasłonięcie obiektu
Faster R-CNN	Dwuetapowe generowanie propozycji regionów, ich klasyfikacja i dopasowanie ramek	Wysoka skuteczność i dokładne określenie położenia obiektów	Większa złożoność obliczeniowa i opóźnienie wynikające z przetwarzania dwuetapowego
SSD	Jednoetapowe predykcje na mapach cech o różnych rozdzielczościach	Krótki czas odpowiedzi i możliwość wykrywania obiektów o różnych rozmiarach	Zależność od konfiguracji ramek domyślnych i gorsze wyniki dla małych obiektów
YOLO	Jednoetapowa predykcja klas i ramek ograniczających	Szybkie działanie, możliwość pracy w czasie rzeczywistym i dostępność wariantów modelu	Konieczny kompromis pomiędzy szybkością a skutecznością, szczególnie przy małych lub częściowo zasłoniętych obiektach
DETR	Bezpośrednia predykcja zbioru obiektów z wykorzystaniem transformera i mechanizmu uwagi	Modelowanie globalnych zależności w scenie i brak konieczności używania ramek domyślnych ani algorytmu NMS	Długi trening i trudniejsza optymalizacja

Zestawienie nie wskazuje jednego najlepszego rozwiązania, ale przedstawia zalety i wady poszczególnych metod. Wybór podejścia zależy od nałożonych wymagań oraz ograniczeń sprzętowych i środowiskowych. W projektowanym systemie szczególnie ważny jest krótki czas reakcji, odporność na zmieniające się warunki środowiskowe oraz zachowanie odpowiedniej skuteczności. Tych założeń – w większości – nie spełniają, mimo niewielkich wymagań, rozwiązania klasyczne będąc silnie zależnymi od ręcznie dobieranych cech i warunków. Detektory dwuetapowe mogą natomiast osiągać wysoką skuteczność, ale oddzielne generowanie i przetwarzanie skutkuje zwiększeniem ich złożoności obliczeniowej i czasu uzyskania odpowiedzi. Z tych względów uwagę należy

kierować na detektory jednoetapowe, które oferują kompromis między czasem i skutecznością detekcji. Predykcje o wartości pewności niższej od ustalonego progu mogą zostać odrzucone, ograniczając ryzyko przekazania użytkownikowi wyników, których model nie jest dostatecznie pewny [33].

Spośród detektorów jednoetapowych wymagania projektowanego systemu najlepiej spełnia rodzina modeli YOLO. W pojedynczym kroku detekcji potrafią one bezpośrednio przewidzieć klasę obiektu i jego położenie, co znacznie skraca czas potrzebny na uzyskanie wyniku. Rodzina zapewnia duży wybór wariantów modeli, dopasowanych do konkretnych zadań i zdolności obliczeniowej urządzeń. Pierwsza wersja YOLO osiągnęła prędkość predykcji około 45 FPS (ang. *frames per second*), a jej uproszczone warianty nawet 155 FPS, co potwierdziło, że może zostać wykorzystana w rozwiązaniach RT [39].

Wykorzystanie detektora opartego na architekturze transformera nie jest w tym wypadku uzasadnione. Analizowana scena ma zawierać niewielką liczbę obiektów, a zadaniem modelu jest przede wszystkim określenie ich lokalizacji i klasy. Jawne modelowanie globalnych zależności nie stanowi kluczowego wymagania, a korzyści płynące z tej metody nie równoważą dodatkowego kosztu, który należy ponieść przy uczeniu i optymalizacji modelu [3].

2.5 Metryki oceny detekcji obiektów

Ocena modeli detekcji obiektów wymaga zastosowania odpowiednich metryk, które pozwalają na porównanie uzyskanych wyników. Metryki te powinny uwzględniać zarówno poprawność klasyfikacji, jak i dokładność lokalizacji obiektu na obrazie. W przeciwieństwie do klasyfikacji obrazów, przy detekcji nie wystarczy, aby model poprawnie rozpoznał klasę obiektu. Równie istotne jest, aby poprawnie wyznaczył ramkę ograniczającą wokół wykrytego obiektu. Jedną z najczęściej stosowanych metryk jest IoU (ang. *Intersection over Union*), określająca stopień pokrycia między ramką obiektu przewidywaną przez model a ramką rzeczywistą [31]. Wartość IoU obliczana jest jako stosunek pola części wspólnej obu ramek do pola ich części łącznej:

$$IoU = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|}, \quad (2.3)$$

gdzie B_p jest ramką przewidywaną przez model, a B_{gt} jest ramką referencyjną (ang. *ground truth*). Im większa wartość IoU, tym lepsze dopasowanie ramki przewidywanej do ramki rzeczywistej. W praktyce najczęściej stosuje się progi, np. $IoU \geq 0,5$, aby zdecydować, czy dana detekcja może być uznana za poprawną. Próg ten został wykorzystany w wielu klasycznych procedurach oceny detekcji, takich jak PASCAL VOC [13], i jest nadal powszechnie stosowany w wielu benchmarkach. Na podstawie wartości IoU oraz

zgodności klasy obiektu możliwe jest przypisanie wyników detekcji do jednej z kategorii: prawdziwych pozytywów (TP, ang. *true positive*), fałszywych pozytywów (FP, ang. *false positive*) i fałszywych negatywów (FN, ang. *false negative*). Detekcja jest traktowana jako TP, jeżeli model poprawnie rozpoznał klasę obiektu, a przewidywana ramka spełnia zadany próg IoU względem ramki referencyjnej. FP to detekcja błędna, w której model przewidział klasę obiektu, ale nie spełnił progu IoU lub przewidziana klasa była błędna. FN natomiast to sytuacja, w której model nie wykrył obiektu, który rzeczywiście był na obrazie.

Jedną z podstawowych metryk oceny jest precyzja (ang. *precision*), określająca udział poprawnych detekcji modelu wśród wszystkich detekcji, jakie zwrócił:

$$Precision = \frac{TP}{TP + FP}. \quad (2.4)$$

Wysoka precyzja oznacza, że model generuje niewiele fałszywych pozytywów. Drugą, powiązaną z nią metryką, jest czułość (ang. *recall*), określająca, jaki jest udział poprawnych detekcji wśród wszystkich rzeczywistych obiektów:

$$Recall = \frac{TP}{TP + FN}. \quad (2.5)$$

Obie metryki pozostają ze sobą w relacji kompromisu, ponieważ zwiększenie progu pewności detekcji może ograniczyć liczbę fałszywych pozytywów, ale jednocześnie prowadzić do pominięcia części rzeczywistych obiektów. Z kolei obniżenie tego progu może zwiększyć liczbę detekcji, ale równocześnie powodować wzrost liczby fałszywych detekcji. Z tego względu, przy analizie detektorów często łączy się je w jedną metrykę – krzywą PR (precyzja-czułość, ang. *precision-recall curve*), przedstawiającą zależność pomiędzy nimi dla różnych wartości progu decyzyjnego. Na jej podstawie wyznacza się metrykę AP (ang. *Average Precision*), odpowiadającą polu pod krzywą PR.

W przypadku detekcji wieloklasowej używa się średniej wartości AP obliczonej dla wszystkich klas, określanej jako *mAP* (ang. *mean Average Precision*):

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i, \quad (2.6)$$

gdzie N jest liczbą klas, a AP_i jest wartością AP dla i -tej klasy. W praktyce najczęściej wykorzystuje się dwie wersje tej metryki: *mAP*50, gdzie detekcję uznaje się za poprawną przy progu $IoU \geq 0,5$, oraz *mAP*50-95, gdzie wynik jest uśredniany dla progów IoU z zakresu od 0,5 do 0,95 ze skokiem 0,05. Pierwsza z nich jest bardziej liberalna i pozwala określić ogólną zdolność modelu do wykrycia obiektu. Druga natomiast jest bardziej rygorystyczna, ponieważ wymaga dokładniejszego dopasowania ramek ograniczających, dzięki czemu lepiej pokazuje zarówno skuteczność klasyfikacji, jak i lokalizacji obiektu. Uśrednianie wyników dla wielu progów zostało szeroko zastosowane m.in. w metodyce oceny

detekcji zbioru Microsoft COCO [23] i jest obecnie standardem w wielu benchmarkach detekcji obiektów.

Dodatkowo, w pracy wykorzystano metrykę $F1$ -score, będącą średnią harmoniczną precyzji i czułości:

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}. \quad (2.7)$$

Metryka ta umożliwia przedstawienie kompromisu pomiędzy liczbą fałszywych detekcji a liczbą pominiętych obiektów. Może być szczególnie przydatna, gdy sama precyzja lub czułość nie dają pełnego obrazu jakości modelu. W niniejszej pracy metrykę $F1$ -score wykorzystano jako metrykę pomocniczą względem podstawowych metryk $mAP50$ i $mAP50-95$, aby lepiej zrozumieć kompromis pomiędzy precyzją a czułością w kontekście detekcji obiektów dotykanych przez użytkownika.

Oprócz wartości liczbowych wykorzystano również macierze pomyłek (ang. *confusion matrix*), pozwalające przeanalizować, które klasy obiektów są najczęściej mylone przez model. Są one przedstawiane w formie tabel, w których jedna oś odpowiada rzeczywistym klasom, a druga klasom przewidywanym przez model. Dla dobrze działającego modelu większość wartości powinna znajdować się na przekątnej macierzy, co wskazuje na poprawną klasyfikację obiektów. Macierze pomyłek są szczególnie istotne przy rozpoznawaniu brył geometrycznych o podobnych kształtach, np. sześcianu i prostopadłościanu, ponieważ sama wartość mAP nie wskazuje bezpośrednio, pomiędzy którymi klasami model popełnia najwięcej błędów. Analiza macierzy pomyłek może również pomóc w identyfikacji problemów z danymi treningowymi, np. niedostatecznej liczby przykładów dla wybranych klas lub błędów w oznaczeniach.

Skuteczność detekcji w systemach czasu rzeczywistego nie zależy jednak wyłącznie od dokładności predykcji, ale również od czasu potrzebnego na jej uzyskanie i kosztu obliczeniowego jaki należy ponieść. Podstawowymi metrykami w tym kontekście są liczba parametrów modelu, liczba operacji wykonywanych podczas inferencji oraz czas potrzebny na przetworzenie pojedynczego obrazu. Liczba parametrów modelu wpływa na rozmiar modelu i jego zapotrzebowanie na pamięć, a koszt obliczeniowy może zostać wyrażony w liczbie operacji zmiennoprzecinkowych (ang. *floating point operations*, FLOPs). W praktyce miarę wydajności modelu wskazuje czas inferencji, określający czas potrzebny na przetworzenie pojedynczego obrazu.

Powiązana z czasem inferencji jest liczba klatek na sekundę (ang. *frames per second*, FPS), określająca liczbę obrazów, które model jest w stanie przetworzyć w ciągu pojedynczej sekundy. Przy sekwencyjnym przetwarzaniu obrazów można ją przybliżyć jako odwrotność czasu inferencji:

$$FPS \approx \frac{1000}{t_{inf}}, \quad (2.8)$$

gdzie t_{inf} stanowi czas inferencji wyrażony w milisekundach. Obie wartości są silnie zależne od zastosowanego sprzętu, oprogramowania, rozmiaru obrazu wejściowego oraz sposobu wykonania pomiaru. Z tego powodu ich bezpośrednie porównanie może zostać wykonane jedynie wtedy, gdy ich wartości zostały uzyskane w jednakowych warunkach. W niniejszej pracy zostały wykorzystane jako metryki pomocnicze przy ocenie kompromisu pomiędzy skutecznością detekcji a kosztem działania modelu.

2.6 Rodzina modeli YOLO

Pierwsza wersja YOLO zadebiutowała wraz z publikacją artykułu *You Only Look Once: Unified, Real-Time Object Detection* przez Josepha Redmona, Santosha Divvalę, Rossa Girshicka i Ali Farhadi w 2016 roku i od tego czasu rodzina modeli YOLO należy do najbardziej rozpoznawalnych i popularnych detektorów jednoetapowych. Autorzy określili problem detekcji obrazu jako regresję, w której pojedyncza sieć neuronowa przewiduje zarówno ramki obiektów, jak i odpowiadające im prawdopodobieństwa klas [39]. Kolejne lata przyniosły rozwój kolejnych modeli rodziny YOLO przez różne zespoły badawcze, więc nie istnieje pojedyncza gałąź rozwoju rodziny. Jedną z głównych gałęzi jest rozwijana przez firmę Ultralytics, która w 2020 roku zaprezentowała YOLOv5, a w następnych latach jej kolejne generacje. W chwili opracowywania pracy najnowszą wersją jest YOLOv6 [20].

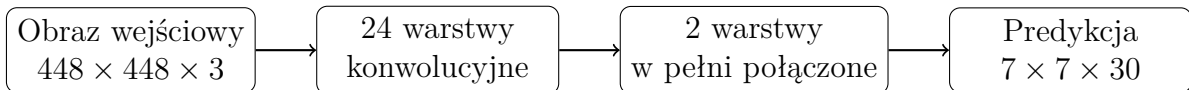
2.6.1 Architektura modeli YOLO

Mimo że architektura każdej wersji modelu YOLO różni się od siebie, to idea pierwszej wersji pozostaje niezmienną. Założenie, że obraz jest analizowany w pojedynczym przebiegu sieci neuronowej, a wynik wyznaczany jest bez wcześniejszego wygenerowania propozycji regionów jest obecne w każdej kolejnej wersji. Sama architektura sieci mocno zmieniała się na przestrzeni kolejnych wersji, a porównanie kilku kolejnych modeli pozwala określić kierunek rozwoju całej rodziny.

Pierwszy model YOLO składał się z CNN złożonej z 24 warstw konwolucyjnych i dwóch warstw w pełni połączonych. Zadaniem warstw konwolucyjnych była ekstrakcja cech z obrazu wejściowego, a warstwy końcowe generowały wynik detekcji. W modelu wykorzystano m.in. warstwy 1×1 do redukcji liczby kanałów oraz warstwy 3×3 [39].

Dla obrazów ze zbioru PASCAL VOC wejście sieci miało rozmiar 448×448 pikseli, które następnie były przekształcane w logiczną siatkę o wymiarach $S \times S$. W zastosowanej konfiguracji sieci przyjęto wartość $S = 7$ dzielącą obraz na 49 komórek. Wykrycie obiektu następowało w oparciu o komórkę, w której znajdował się środek jego ramki ograniczającej. Każda z komórek mogła przewidzieć $B = 2$ ramki, wartości ich pewności i prawdopodobieństwa $C = 20$ klas obiektów występujących w zbiorze. Rozmiar tensora

był definiowany jako $S \times S \times (B \cdot 5 + C)$, co w omawianym przypadku dawało tensor o wymiarach $7 \times 7 \times 30$. Każda ramka opisana była za pomocą pięciu wartości: współrzędnych środka ramki, jej szerokości i wysokości oraz wartości pewności predykcji. Jednoczesne przewidywanie ramek i klas obiektów w pojedynczym kroku wyeliminowało wymaganie stosowania oddzielnych etapów generowania propozycji regionów i ich klasyfikacji. Schemat przedstawiony na rysunku 2.1 pokazuje uproszczoną architekturę pierwszego modelu YOLO, w której, pomimo stosunkowo prostej budowy, udało się połączyć w jeden model zarówno klasyfikację, jak i lokalizację obiektów.



Rysunek 2.1: Uproszczony schemat architektury pierwszego modelu YOLO [39].

Kolejne wersje YOLO przyniosły przede wszystkim zmiany w architekturze przez stosowanie coraz wydajniejszych modułów sieci, korzystanie z kilku skal obrazu i modyfikację sposobu generowania ramek ograniczających. Z czasem architekturę opartą na pojedynczej siatce zastąpiono architekturami wykorzystującymi wieloskalowe mapy cech. Zmodernizowano również sposób w jaki łączone są informacje z różnych warstw sieci, a także budowę głowic. Dzięki temu, współczesne modele można przedstawić za pomocą trzech głównych elementów: *backbone*, *neck* i *head*. Najważniejsze wersje i zmiany w ich architekturze zestawiono w tabeli 2.2.

Współczesna architektura modelu YOLO26 zachowała konwolucyjny rdzeń rodziny, ale znacząco różni się od pierwszej wersji modelu. Sieć podzielona jest na trzy główne części, z których każda pełni inną funkcję. Pierwszą częścią jest *backbone*, który ma za zadanie ekstrakcję hierarchicznych reprezentacji cech z obrazu. Przechodzenie przez kolejne warstwy sieci wiąże się ze zmniejszeniem rozdzielczości przestrzennej map cech, a jednocześnie zwiększeniem zdolności do rozpoznawania coraz bardziej złożonych i abstrakcyjnych cech. Kolejna część to *neck* odpowiedzialny za połączenie informacji pochodzących z różnych poziomów sieci, co jest kluczowe przy wykrywaniu obiektów o różnych wymiarach: cechy pochodzące z głębszych warstw są bardziej abstrakcyjne, a płytsze warstwy dostarczają informacji przestrzennych. To połączenie umożliwia modelowi prowadzenie detekcji na kilku skalach jednocześnie. Ostatnią częścią jest głowica detekcyjna (ang. *head*), która przewiduje klasy obiektów i położenie ich ramek ograniczających na podstawie dostarczonych z poprzedniej części map cech. Model YOLO26 wyposażony jest w dwie głowice detekcyjne: głowicę *one-to-many* i głowicę *one-to-one*. Pierwsza odpowiada za generowanie dużej liczby kandydatów obiektów, dostarczając gęstszy sygnał nadzorującego podczas uczenia, a druga uczy się przypisywać pojedynczą predykcję do każdego obiektu referencyjnego i jest domyślnie używana podczas inferencji [20].

Tabela 2.2: Wybrane etapy rozwoju architektury modeli YOLO. Opracowanie własne na podstawie [20, 39, 40, 53].

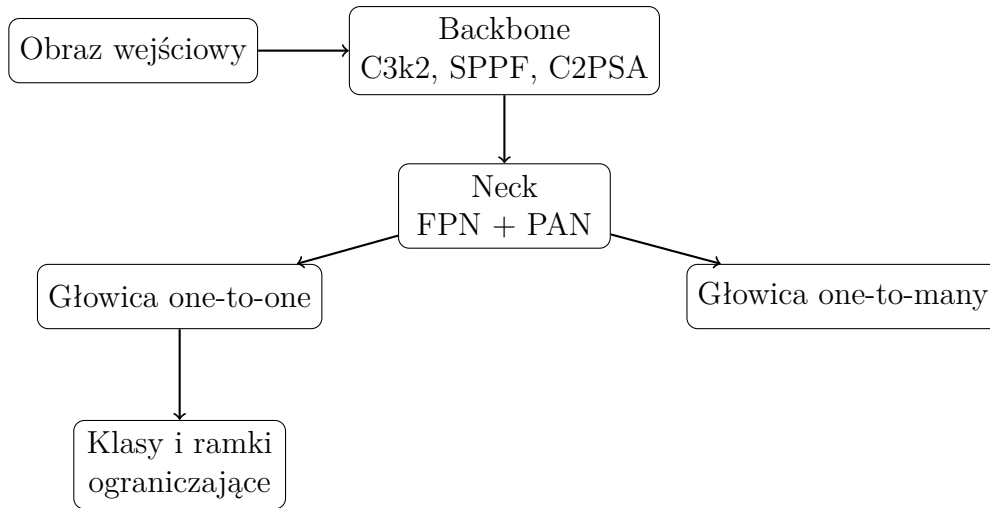
Generacja	Ekstrakcja cech	Sposób detekcji	Charakterystyczna zmiana
YOLO (v1)	Własna sieć CNN	Jedna siatka predykcji	Ujęcie detekcji jako pojedynczego problemu regresji
YOLOv3	Darknet-53	Detekcja na trzech skalach, ramki kotwiczące	Wprowadzenie wieloskalowej detekcji i architektury FPN
YOLOv5	CSP/C3	Wieloskalowa, ramki kotwiczące	Rozbudowanie fuzji cech o ścieżkę PAN i moduł SPPF
YOLOv8	C2f	Wieloskalowa, bez ramek kotwiczących	Przejsięcie na głowicę anchor-free i wykorzystanie DFL
YOLO11	C3k2, C2PSA	Wieloskalowa, bez ramek kotwiczących	Wprowadzenie kolejnych mechanizmów uwagi i zmian w blokach ekstrakcji cech
YOLO26	C3k2, C2PSA	Detekcja end-to-end	Domyślna detekcja bez NMS oraz rezygnacja z DFL

Co ważne, podczas domyślnego wnioskowania model YOLO26 wykorzystuje głowicę *one-to-one*. Dzięki temu, że predykcja jest jednoznacznie przypisana, model może dokonywać detekcji bez użycia algorytmu NMS, a co za tym idzie może realizować detekcję typu end-to-end. W sytuacjach, w których użycie algorytmu NMS jest dopuszczalne model może korzystać z głowicy *one-to-many*, aby uzyskać większą skuteczność kosztem dodatkowego czasu [20].

Kolejnym ważnym aspektem wersji 26 jest fakt rezygnacji z użycia DFL (ang. *Distribution Focal Loss*) w trakcie regresji ramek, co pozwala zmniejszyć złożoność głowicy detekcyjnej i usunąć ograniczenia wynikające z dyskretnej reprezentacji odległości. Do procesu uczenia wprowadzono mechanizm Progressive Loss, stopniowo zwiększający znaczenie głowicy, oraz STAL (ang. *Small-Target-Aware Label Assignment*), poprawiający przypisanie przykładów pozytywnych dla małych obiektów. W procesie optymalizacji wykorzystano optymalizator MuSGD⁶ [20].

Na rysunku 2.2 przedstawiono uproszczony schemat współczesnej architektury modelu YOLO26. Ukazuje on znaczny rozwój względem pierwszej wersji modelu i wyraźne odejście od przekształcania wejścia w pojedynczą siatkę predykcji. Zastąpiono ją hierarchicznym przetwarzaniem cech, ich wieloskalowym połączeniem i dostosowaną głowicą decyzyjną.

⁶Muon-inspired Stochastic Gradient Descent



Rysunek 2.2: Uproszczony schemat architektury detektora YOLO26 [20].

Pomimo tych zmian, zachowano główną ideę rodziny, czyli detekcję przez jeden model, bez osobnego etapu generowania propozycji regionów.

2.6.2 Warianty ze względu na realizowane zadanie

Dzisiejsza implementacja rodziny YOLO przez Ultralytics nie ogranicza się jedynie do klasycznej detekcji, a stara się dostarczać wiele modeli obsługujących różne zadania. Głównym kierunkiem jest wciąż model klasycznej detekcji, ale jego architektura może zostać rozszerzona o dodatkowe głowice przeznaczone do obsługi innych zadań. Obecnie dostępne modele mogą obsługiwać zadania detekcji, klasyfikacji, segmentacji instancji, segmentację semantyczną, estymację głębi, estymację pozy oraz detekcję z obróconymi ramkami [52]. Poszczególne zadania zostały zestawione w tabeli 2.3.

Ze względu na specyfikę systemu spośród dostępnych wariantów YOLO26 do dalszej analizy wybrano model przeznaczony do detekcji obiektów. Wynik zawierający klasę obiektu, wartość pewności oraz jego prostokątną ramkę ograniczającą dostarcza wystarczającej liczby informacji o obiekcie by móc zrealizować moduł detekcji. Segmentacja instancji umożliwiłaby dokładne wyodrębnianie obiektu z tła, ale wymagałaby przygotowania masek segmentacyjnych dla zbioru uczącego, a także zwiększyłaby złożoność obliczeniową modelu. Możliwym kierunkiem dalszego rozwoju może być wykorzystanie wariantu z ramkami obróconymi, które mogłyby ograniczyć udział tła w obszarze detekcji.

Tabela 2.3: Wybrane zadania obsługiwane przez modele YOLO26

Zadanie	Oznaczenie modelu	Wynik działania
Detekcja obiektów	yolo26*.pt	Klasy obiektów, wartości pewności i prostokątne ramki ograniczające
Segmentacja instancji	yolo26*-seg.pt	Klasy, ramki ograniczające i osobne maski pikselowe obiektów
Segmentacja semantyczna	yolo26*-sem.pt	Przypisanie klasy do każdego piksela obrazu
Estymacja głębi	yolo26*-depth.pt	Mapa przewidywanej głębi obrazu
Klasyfikacja obrazu	yolo26*-cls.pt	Klasa przypisana do całego obrazu
Estymacja pozy	yolo26*-pose.pt	Położenie punktów charakterystycznych obiektu lub sylwetki
Detekcja z ramkami obróconymi	yolo26*-obb.pt	Klasy i ramki ograniczające rozszerzone o orientację obiektu

2.6.3 Warianty ze względu na skalę modelu

Każdy wariant YOLO26 jest dostępny w kilku skalach różniących się od siebie liczbą parametrów oraz kosztem obliczeniowym. Wybór odpowiedniego wariantu powinien być oparty na wymaganiach systemu oraz ograniczeniach zasobów sprzętowych urządzenia końcowego. Autorzy modelu detekcyjnego przewidzieli pięć wariantów: **n** (*nano*), **s** (*small*), **m** (*medium*), **l** (*large*) oraz **x** (*extra large*). W referencyjnych wynikach każdy kolejny wariant osiąga wyższe wartości mAP_{50-95} , wymagając jednocześnie większej liczby parametrów i operacji [52].

Wyniki, które zostały przedstawione w tabeli 2.4 pokazują kompromis, typowy dla tej rodziny modeli. Wzrost rozmiaru modelu skorelowany jest ze wzrostem wartości mAP_{50-95} , liczby parametrów, liczby wykonywanych operacji, oraz czasem wykonania. Największy wzrost skuteczności pomiędzy sąsiednimi wariantami obserwujemy przy przejściu z modelu YOLO26n do YOLO26s: wartość mAP_{50-95} wzrasta o 7,7 punktu. Dalsze zwiększanie rozmiaru przynosi coraz mniejsze przyrosty względem rosnącego kosztu obliczeniowego.

Tabela 2.4: Porównanie wariantów detekcyjnych YOLO26 na zbiorze COCO dla obrazów 640×640 . Opracowanie własne na podstawie [52].

Wariant	mAP50-95	Parametry [mln]	GFLOPs	T4 TensorRT10 [ms]
YOLO26n	40,9	2,4	5,4	1,7
YOLO26s	48,6	9,5	20,7	2,5
YOLO26m	53,1	20,4	68,2	4,7
YOLO26l	55,0	24,8	86,4	6,2
YOLO26x	57,5	55,7	193,9	11,8

Podane wartości czasowe służą wyłącznie porównaniu wariantów w jednakowych warunkach testowych i nie mogą być utożsamiane z wartością rzeczywistą czasu wykonania. Ten zależy od sprzętu, rozdzielczości obrazu wejściowego, sposobu wdrożenia i innych elementów implementacji.

2.6.4 Dobór wariantów modelu

Do dalszej analizy przyjęto najnowszą wersję modeli rozwijanych przez Ultralytics – YOLO26. Wprowadza ona rozwiązania istotne z punktu projektowanego systemu, przede wszystkim domyślną detekcję end-to-end bez etapu NMS i uproszczoną regresję ramek. Te zmiany ograniczają złożoność głowicy i zakres przetwarzania, jaki był wykonywany po zakończeniu pracy sieci [20].

Ze względu na wymaganie działania modelu w RT oraz założenie wykorzystania systemu w środowisku szkolnym, dalszą analizę należy skupić na najmniejszych wariantach modelu: **n** i **s**. Wynika to z konieczności uwzględnienia ograniczeń i heterogeniczności infrastruktury sprzętowej w polskich placówkach oświatowych. Zieliński w badaniu z 2023 roku dotyczącym cyfryzacji polskich szkół wskazuje na problem przestarzałego sprzętu komputerowego, często mającego ponad 5 lat użytkowania [66].

Wariant **n** charakteryzuje się najmniejszą liczbą parametrów i najniższym kosztem obliczeniowym, co czyni go naturalnym kandydatem dla urządzeń o ograniczonych zasobach. Wariant **s** daje natomiast istotny wzrost wartości *mAP*50-95, dlatego może lepiej odpowiadać wymaganiom stawianym systemowi. Uwzględnienie obu wersji pozwoli przeanalizować wpływ poszczególnych wariantów i kombinacji parametrów na wydajność i dokładność detekcji.

2.7 Wykorzystanie danych wizualnych i przestrzennych w rozpoznawaniu obiektów

Rozpoznawanie obiektów nie musi opierać się jedynie na klasycznym obrazie RGB. Współczesne urządzenia mogą oferować również dostęp do danych opisujących geometrię obserwowanej sceny, przedstawionych w postaci chmur punktów czy map głębi. Dane przestrzenne mogą bardzo dobrze uzupełniać informacje o kolorze, teksturze i strukturze obrazu rejestrowanego przez kamerę. Każdy z wymienionych sposobów reprezentacji danych wiąże się jednak z odmiennymi ograniczeniami i wymaganiami, które należy rozpatrywać w kontekście konkretnego zastosowania.

2.7.1 Obraz RGB

Obrazy kolorowe, najczęściej reprezentowane w przestrzeni RGB, stanowią obecnie jedną z najpowszechniejszych form reprezentacji danych w dziedzinie wizji komputerowej [57]. Kamery są obecnie wykorzystywane w większości dziedzin życia: monitoringu, medycynie, przemyśle, motoryzacji czy w urządzeniach osobistych. Istotne jest więc określenie, jakie informacje zawarte w zarejestrowanym obrazie mogą zostać wykorzystane do rozpoznawania obiektów 3D.

Obraz zarejestrowany przez kamerę jest reprezentacją trójwymiarowej sceny na płaszczyźnie dwuwymiarowej. Pomimo utraty bezpośredniej informacji o głębi, z pojedynczej klatki wyodrębnić można wiele informacji użytecznych podczas analizy obiektów, m.in. o kolorze, teksturze, krawędziach, zmianach jasności czy cieniach. Te informacje mogą z powodzeniem zostać wykorzystane przez modele uczenia maszynowego do wyznaczenia charakterystycznych cech przedstawionych obiektów. Obrazy RGB stanowią również podstawową formę danych wejściowych dla wielu CNN oraz detektorów obiektów, w tym rodziny YOLO opisanej w sekcji 2.6.

Rejestracja obrazu przez kamerę cyfrową jest oparta na procesie przejścia światła odbitego przez układ optyczny i skupieniu go na światłoczułym sensorze. Współczesne kamery cyfrowe do rejestracji obrazu wykorzystują głównie sensory CMOS (ang. *complementary metal-oxide-semiconductor*), a historycznie szeroko stosowano sensory CCD (ang. *charge-coupled device*). Dane zarejestrowane przez sensor są zwykle przetwarzane dalej w celu ich odszumienia, korekty kolorów czy poprawienia balansu bieli. Wynik może zostać zapisany w formacie skompresowanym JPEG (ang. *Joint Photographic Experts Group*) lub HEIF (ang. *High Efficiency Image File Format*) [49].

Po przetworzeniu obraz RGB może być reprezentowany w postaci macierzy trójwymiarowej o wymiarach

$$M \times N \times 3, \tag{2.9}$$

gdzie M i N to liczba wierszy i kolumn obrazu, a trzeci wymiar odpowiada kanałom barwnym: czerwonemu, zielonemu i niebieskiemu. Wartość pojedynczego piksela jest więc krotką trzech wartości, które można zapisać jako

$$I(i, j) = (R, G, B). \quad (2.10)$$

Pojedynczy piksel zawiera jedynie informację o wartościach trzech składowych koloru, ale zależności przestrzenne pomiędzy wieloma pikselami pozwalają odwzorować strukturę obserwowanej sceny. Lokalne zmiany wartości pikseli pozwalają m.in. wyznaczać gradienty obrazu, które mogą posłużyć do wykrywania krawędzi i innych strukturalnie ważnych elementów [48]. Jednakże, pomimo bogactwa informacji dostępnych do odczytu w samym obrazie, reprezentacja ta ma również istotne ograniczenia. Przede wszystkim, wygląd obserwowanego obiektu zależy znacząco od warunków oświetleniowych i stopnia zasłonięcia. Zmiana tych warunków może doprowadzić do różnic w kolorach, kontraście czy widoczności jego krawędzi, tym samym wpływając na skuteczność jego rozpoznawania [49]. Brak informacji o odległości punktu od kamery jest również istotnym ograniczeniem.

Projekcja dwuwymiarowa skutkuje utratą części informacji przestrzennej, przez co różne obiekty lub ich różne orientacje mogą prowadzić do podobnych obrazów. Dodatkowym utrudnieniem może być częściowe zasłonięcie obiektów. Istnieją modele estymujące głębię, np. YOLO26*-depth, opisany w sekcji 2.6.2, jednak otrzymane z nich wartości nie stanowią pomiaru odległości, a ich dokładność zależy od wielu czynników. Z tego względu uzupełnienie obrazu o informację dotyczącą odległości punktów od kamery wydaje się przydatne podczas procesu rozpoznania brył trójwymiarowych.

2.7.2 Pozyskiwanie informacji o głębi

Informacja dotycząca głębi opisuje położenie aktualnie obserwowanych punktów względem kamery. Jeżeli jej wartości przypiszemy do poszczególnych pikseli, otrzymujemy mapę głębi, którą możemy zapisać jako

$$D(i, j) = d, \quad (2.11)$$

gdzie d to głębokość sceny odpowiadająca pikselowi (i, j) . Dzisiejsza technika pozwala na określanie głębi na kilka sposobów.

Najprostsza ze względu na wymaganą infrastrukturę jest estymacja głębi z pojedynczego obrazu RGB. Określana jest ona jako *monocular depth estimation*, a jej działanie opiera się na wyznaczeniu struktury przestrzennej na podstawie cech obrazu. Przykładami tej techniki są: rozwiązanie zaproponowane przez Ranftla et al. oraz Monodepth2 opracowane przez zespół Godarda [15, 38]. Te metody umożliwiają uzyskanie mapy głębi

bez stosowania dodatkowych urządzeń. Należy jednak pamiętać, że wynik stanowi jedynie estymację odległości, a nie jest jej pomiarem.

Kolejną metodą jest stereowizja, wykorzystująca dwa obrazy zarejestrowane z różnych położań. Znając wzajemne położenie kamer i analizując dysparcję, czyli różnicę położenia odpowiadających sobie punktów na obrazach, możliwe jest wyznaczenie informacji o głębi. Technologia stereoskopowa znalazła zastosowanie w urządzeniach konsumenckich. W 2010 roku firma Nintendo zaprezentowała nową konsolę wykorzystującą właśnie to rozwiązanie. Konsola Nintendo 3DS była wyposażona w dwie kamery, które umożliwiały rejestrację zdjęć stereoskopowych [19]. To rozwiązanie wymaga jednak znajomości parametrów kamer i ich wzajemnego położenia, a także kalibracji układu.

Ciekawym podejściem do omawianego problemu jest aktywny pomiar odległości, w którym urządzenie emituje sygnał i wyznacza odległość na bazie zarejestrowanego sygnału powrotnego. Jednym z takich urządzeń jest LiDAR, który emituje promienie lasera i analizuje sygnał odbity od obiektów. Jeżeli dane rozwiązanie wykorzystuje metodę czasu przelotu, to odległość jest wyznaczana na podstawie czasu między emisją impulsu a zarejestrowaniem jego powrotu [42].

2.7.3 Dane LiDAR i chmury punktów

Dane pozyskane z czujnika LiDAR mogą być wykorzystane do zbudowania reprezentacji przestrzennej sceny. Jako formę tej reprezentacji często stosuje się chmurę punktów (ang. *point cloud*), będącą zbiorem punktów rozmieszczonych w przestrzeni 3D. Można ją przedstawić jako zbiór

$$P = \{p_i\}_{i=1}^N, \quad (2.12)$$

gdzie każdy punkt p_i jest reprezentowany jako wektor

$$p_i = (x_i, y_i, z_i). \quad (2.13)$$

Zależnie od źródła danych, do poszczególnych punktów mogą zostać dodane dodatkowe informacje, np. intensywność sygnału, kolor lub wektor normalny. Chmura punktów, w przeciwieństwie do obrazu RGB, nie musi tworzyć regularnej siatki i zazwyczaj traktuje się ją jak nieuporządkowany zbiór punktów. Kolejność punktów w zbiorze nie powinna mieć znaczenia dla reprezentacji geometrii obiektu [4].

Brak regularnej struktury chmury punktów powoduje, że jej przetwarzanie nie może być zrealizowane w identyczny sposób jak obrazu. Z tego powodu zaproponowano wiele architektur przeznaczonych do bezpośredniego przetwarzania tego typu danych. Jedną

z nich jest PointNet, który przetwarza cechy poszczególnych punktów, a następnie wykonuje ich symetryczną agregację. Dzięki temu wynik nie jest zależny od kolejności punktów wejściowych. Ta architektura jest wykorzystywana m.in. do klasyfikacji i segmentacji obiektów trójwymiarowych [4].

Podstawowa wersja PointNet w ograniczonym stopniu uwzględnia lokalne zależności między punktami, co poskutkowało opracowaniem jej rozwinięcia, PointNet++. Wprowadzono w nim hierarchiczne grupowanie punktów oraz uczenie cech dla kolejnych obszarów przestrzennych. Umożliwia to analizę geometrii dla różnych poziomów szczegółowości oraz lepsze uwzględnienie zmiennej gęstości próbek, która występuje w chmurach punktów [37].

Chmury punktów mogą być wykorzystywane bezpośrednio do detekcji obiektów, czego przykładem jest VoteNet. Jego architektura oparta jest na PointNet++ oraz mechanizmie głosowania Hougha. Kolejne punkty wraz z wyznaczonymi cechami generują głosy wskazujące środki potencjalnych obiektów. Głosy następnie są grupowane, a na ich podstawie wyznacza się trójwymiarowe ramki ograniczające i klasy. To rozwiązanie nie wymaga wiedzy o kolorze ani teksturze obiektów, a bazuje jedynie na chmurze, bez potrzeby konwertowania jej do regularnej reprezentacji przestrzennej [35].

Skuteczność rozpoznania obiektów w chmurze punktów zależy silnie od jakości danych wejściowych i metody ich analizy. Rzeczywiste pomiary mogą zawierać w sobie losowe szумы, odstające punkty, czy zmienną gęstość próbek. Ten problem uwidacznia się szczególnie przy analizie niewielkich elementów geometrii, które przy chmurze o małej gęstości mogą być reprezentowane przez pojedyncze punkty. Jak wskazują Zhang et al., takie zakłócenia obecne w rzeczywistych chmurach punktów mogą prowadzić do pogorszenia skuteczności rozpoznawania obiektów. Odpowiedzią autorów na ten problem jest architektura R-PointNet, która zwiększa odporność modelu na zakłócenia [65].

Same dane pochodzące z LiDAR są przede wszystkim opisem geometrii obserwowanej sceny i nie przenoszą informacji o kolorze ani teksturze powierzchni. Istnieją jednak metody łączenia danych z kamer z danymi przestrzennymi. Zalety jednej reprezentacji mogą częściowo kompensować ograniczenia drugiej. Z tego powodu możliwe jest jednocześnie wykorzystanie, w procesie detekcji.

2.7.4 Fuzja danych wizualnych i przestrzennych

Zależnie od etapu, na którym następuje fuzja danych, możliwe jest podzielenie jej na kilka poziomów: fuzję danych, fuzję cech i fuzję decyzji [25].

Połączenie ze sobą danych wizualnych i przestrzennych sprawia, że otrzymujemy wspólną reprezentację sceny. Obraz z kamery dostarcza informacji o kolorze i krawędziach, a mapy głębi i chmury punktów dają nam możliwość bezpośredniego umiejscowienia punktów w przestrzeni, ukazując geometrię przestrzenną całej sceny. Ograniczenia każdej z tych modalności stanowią motywację naukowców do opracowywania metod ich

wspólnego użycia. W literaturze znajdujemy wiele metod fuzji danych. Celem tego działania jest uzyskanie reprezentacji, która będzie zawierała w sobie zarówno cechy wizualne, jak i przestrzenne obserwowanych obiektów. Takie podejście zastosowano w modelu ImVoteNet, będącym rozwinięciem architektury VoteNet. Model łączy ze sobą informacje z obrazów RGB z cechami pochodzącymi z chmury punktów. Na obrazie wyznaczane są cechy geometryczne, semantyczne i teksturowe, a następnie są one przenoszone do przestrzeni trójwymiarowej. Po połączeniu ich z cechami punktów następuje głosowanie, na wzór modelu VoteNet. Na zbiorze SUN RGB-D model osiągnął wartość $mAP@0.25$ wynoszącą 63,4%, poprawiając wynik swego poprzednika o 5,7 punktu [36].

Korzyści z łączenia danych wykazali również Liu et al. proponując model, który wykonywał równoległe detekcje na obrazie z kamery oraz mapie głębi z LiDAR. Ich rozwiązanie, łączące dane, uzyskało lepsze wyniki niż detekcje wykonywane osobno na obu modalnościach. Podczas eksperymentów model osiągnął wartość mAP równą 89,26%, podczas gdy warianty wykorzystujące pojedyncze źródło danych osiągnęły wartości 86,70% dla modelu korzystającego z obrazów i 74,27% dla modelu korzystającego z danych LiDAR. Jednocześnie wskazali, że przetworzenie pojedynczej klatki zajmowało jedynie 0,03 sekundy, co teoretycznie umożliwia działanie modelu w czasie rzeczywistym [25].

Architektury z rodziny YOLO mogą również stanowić element systemów wykorzystujących fuzję danych. Przykładami mogą być sieć YOLO-POINT⁷, w której autorzy połączyli detektor YOLOv5 z siecią PointNet++, a także sieć DTC-YOLO, która wykorzystuje jednocześnie obraz RGB oraz dane LiDAR. W DTC-YOLO zastosowano odwzorowanie informacji o głębi na obraz i ważoną fuzję obu reprezentacji. Model wykorzystuje też sieć ADF³-Net⁸, która stosuje dynamiczne bramkowanie do łączenia cech pochodzących z różnych poziomów. Eksperymenty z zastosowaniem modelu DTC-YOLO na zbiorze KITTI⁹ dały wynik mAP_{50} równy 0,911 oraz mAP_{50-95} równy 0,693, co przewyższa wyniki osiągnięte przez porównywane modele YOLOv8n, YOLOv10n oraz YOLO11n [28, 62].

Wskazuje to, że połączenie danych z różnorodnych modalności może poprawić skuteczność detekcji, a jednocześnie nie musi uniemożliwiać działania w czasie rzeczywistym. Jednym z ograniczeń, jakie należy wziąć pod uwagę, jest konieczność zastosowania dodatkowego sensora, jeżeli dane mają pochodzić z bezpośredniego pomiaru. Korzystanie z kilku źródeł danych może również wymagać ich kalibracji i synchronizacji. Ponadto przetwarzanie dodatkowego strumienia danych może zwiększyć znacząco wymagania obliczeniowe systemu. Przydatność fuzji należy więc rozpatrywać nie tylko z perspektywy potencjalnego wzrostu skuteczności detekcji, ale także z perspektywy dostępności odpowiednich sensorów, jakości pozyskiwanych danych oraz wymagań dotyczących czasu działania systemu.

⁷model łączący detekcję YOLO z informacjami pochodzącymi z obrazu i chmury punktów

⁸Adaptive Dimension-aware Focused Fusion Network

⁹Karlsruhe Institute of Technology and Toyota Technological Institute

2.8 Podsumowanie analizy tematu

Analiza literatury związanej z osobami niewidomymi uwidacznia kierunek, jaki należy przyjąć w trakcie projektowania systemu. Przede wszystkim należy skupić się na dostępności oprogramowania i wsparciu procesu nauki zarówno z perspektywy ucznia, jak i nauczyciela. Szczególną uwagę należy zwrócić na sposób przekazywania informacji użytkownikowi, tak by komunikaty były zrozumiałe, jednoznaczne i skutecznie wspierały proces nauki. Powinny również wspierać użytkownika w nauce, poprzez ułatwienie budowania reprezentacji przestrzennej danego obiektu. Dostępne systemy wspomagające często opierają się na komputerowym przetwarzaniu obrazu otoczenia i algorytmach detekcji obiektów, starając się działać w sposób uniwersalny – rozpoznając obiekty codziennego użytku, jak przeszkody czy elementy otoczenia. Obszarem słabo zagospodarowanym pozostaje wsparcie niewidomych uczniów w nauce o geometrii przestrzennej. Możliwe rozwiązania obejmują systemy wizyjne, urządzenia wyposażone w mikrokontrolery czy różnego rodzaju sensory. Przy wyborze technologii należy jednak uwzględnić jej powszechność i koszt. Ze względu na ograniczone możliwości sprzętowe szkół uzasadnione jest wykorzystanie istniejącej infrastruktury, takiej jak telefony i komputery, bez konieczności zakupu wyspecjalizowanych urządzeń. Specyfika projektowanego systemu wprowadza dodatkowe utrudnienia związane z bezpośrednim kontaktem użytkownika z obiektem. Mimo że dłonie mogą zasłaniać część bryły, detekcja powinna zachować swoją skuteczność również w tych warunkach.

Przegląd możliwych danych wejściowych ujawnia problemy mogące wystąpić przy użyciu pojedynczego źródła danych. Obraz RGB z kamery daje informację o otoczeniu, kolorze i teksturze obiektów, zaś skanowanie przestrzenne pozwala odwzorować głębię i geometrię sceny, umożliwiając tworzenie map głębi lub chmur punktów. Analizowane prace wskazują, że fuzja danych z kamery i skanera LiDAR może prowadzić do poprawy skuteczności detekcji względem korzystania z pojedynczej modalności. Należy jednak przy tym zwrócić uwagę na ograniczenia takiego podejścia, jak konieczność kalibracji sensorów czy większe wymagania obliczeniowe. Z tego względu jako główne źródło danych przyjęto obrazy kolorowe, będące domyślnym wejściem sieci YOLO. Wykorzystanie danych o głębi oraz ich fuzja z obrazem RGB wymagają dodatkowej weryfikacji eksperymentalnej.

Porównanie możliwych metod detekcji wskazuje, że w tej sytuacji najodpowiedniejszym rozwiązaniem będzie użycie detektorów jednoetapowych z rodziny YOLO, w obrębie której zdecydowano się na użycie generacji YOLO26, ze względu na usprawnienie skuteczności i wydajności względem poprzednich wersji. Spośród dostępnych, wybrano warianty *n* i *s*, ze względu na korzystny kompromis pomiędzy skutecznością detekcji a kosztem obliczeniowym. Ich porównanie pozwoli ocenić, czy użycie modelu o większej liczbie parametrów wpłynie na poprawę wyników. Nie należy kategorycznie odrzucać pozostałych

metod, jednak ich ograniczenia mogą znacznie wpłynąć na jakość i kulturę działania systemu.

Kolejne etapy pracy powinny obejmować określenie wymagań i wykonanie projektu systemu. Następnie należy przygotować zbiór danych, wyuczyć wybrane warianty sieci, porównać wyniki oraz zaimplementować mechanizm przekazywania informacji użytkownikowi.

Rozdział 3

Koncepcja systemu rozpoznawania dotykanych obiektów 3D

Przed przystąpieniem do implementacji systemu rozpoznawania dotykanych obiektów 3D, konieczne jest określenie jego koncepcji i wykonanie projektu. Analiza literatury naukowej pozwoliła nakreślić główne wymagania jakie spełniać powinien projektowany system. Najważniejszymi z nich są: rozpoznawanie dotykanych brył, również częściowo zasłoniętych, działanie w czasie zbliżonym do rzeczywistego, jednoznaczne, wspierające użytkownika informacje zwrotne i możliwość uruchomienia części obliczeniowej na powszechnie dostępnych komputerach, również tych o ograniczonych zasobach. Niniejszy rozdział przedstawia proponowaną koncepcję systemu, jego sposób działania, architekturę, sposób przepływu danych oraz najważniejsze decyzje projektowe.

System zaprojektowany jest w architekturze klient-serwer, gdzie klientem jest aplikacja mobilna, która rejestruje dane i przekazuje je do serwera uruchomionego na komputerze. Ten przetwarza dane, wykonuje detekcję obiektów i zwraca wynik. Wybór takiej architektury umożliwia użycie bardziej wymagających modeli niż byłoby to możliwe w przypadku uruchomienia ich na urządzeniu mobilnym.

3.1 Założenia funkcjonalne i нефункционалне

Na podstawie celu pracy i wniosków wynikających z analizy literatury naukowej określono założenia funkcjonalne i нефункционалне projektowanego systemu. Pierwsza grupa opisuje działania, które powinny być realizowane przez poszczególne części rozwiązania, a druga koncentruje się na oczekiwanej jakości działania oraz ograniczeniach mających wpływ na architekturę i sposób implementacji systemu.

Założenia funkcjonalne

Podstawowym zadaniem systemu jest rozpoznanie bryły eksplorowanej przez użytkownika i przekazanie mu informacji o niej oraz o jej widocznej części. W tym celu przyjęto następujące założenia funkcjonalne:

- rejestrowanie obrazu obejmującego dłoń użytkownika i eksplorowaną bryłę za pomocą kamery urządzenia mobilnego,
- ciągle przesyłanie klatek do części serwerowej przez sieć bezprzewodową,
- wykonanie detekcji obiektów dla odebranych klatek i określenie klasy obiektu, ramki ograniczającej oraz wartości pewności predykcji,
- umożliwienie użytkownikowi zażądania informacji o aktualnie eksplorowanym obiekcie za pomocą komendy głosowej,
- analiza obszaru obrazu zawierającego wykrytą bryłę, po otrzymaniu żądania informacji, w celu określenia widocznej części bryły,
- przygotowanie komunikatu na podstawie rozpoznanej klasy obiektu, analizy jego powierzchni i danych zawartych w bazie informacji o bryłach,
- przesłanie komunikatu do aplikacji mobilnej i odczytanie go użytkownikowi,
- obsługa sytuacji braku wykrycia obiektu lub niskiej wartości pewności predykcji przez przygotowanie odpowiedniego komunikatu i przekazanie go użytkownikowi.

Transmisja obrazu i detekcja powinny być kontynuowane niezależnie od pozostałych działań systemu, aby mógł pozostać gotowym do przyjęcia kolejnego żądania od użytkownika.

Założenia niefunkcjonalne

Sposób działania systemu powinien uwzględniać potrzeby użytkowników niewidomych, warunki jego wykorzystania i dostępność infrastruktury komputerowej. Na tej podstawie przyjęto następujące założenia niefunkcjonalne:

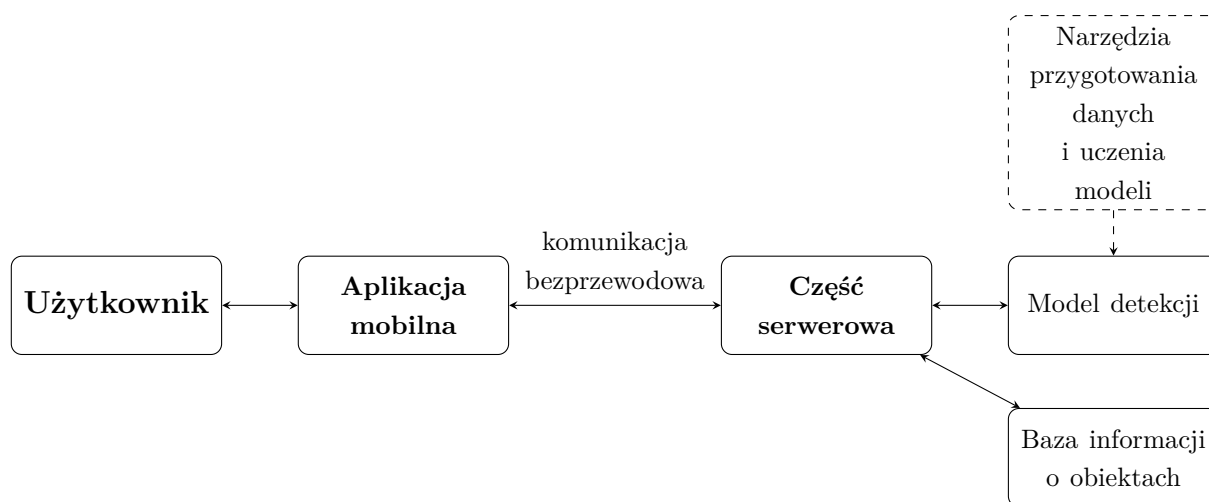
- krótki czas odpowiedzi – czas pomiędzy wydaniem komendy a odczytaniem komunikatu powinien być na tyle krótki, aby odpowiedź odnosiła się do aktualnej orientacji i położenia obiektu.
- możliwość działania na powszechnie dostępnym sprzęcie – część serwerowa powinna móc być uruchomiona na komputerze osobistym, bez stosowania specjalistycznej jednostki obliczeniowej.

- mobilność części klienckiej – urządzenie rejestrujące obraz nie powinno wymagać przewodowego połączenia z komputerem ani wyspecjalizowanego stanowiska pracy.
- odporność na sposób eksploracji – detekcja powinna uwzględniać różne orientacje brył, ułożenia dłoni i częściowe zasłonięcia obiektu.
- stabilne działanie w zmiennych warunkach – system powinien rozpoznawać bryły i działać niezależnie od zmian oświetlenia lub tła.
- modułowość – elementy systemu powinny być od siebie rozdzielone tak, aby możliwa była ich niezależna modyfikacja.
- rozszerzalność –system powinien umożliwiać dodanie kolejnych klas obiektów po przygotowaniu dodatkowych danych, wymianę modelu detekcji na inny oraz uzupełnienie bazy informacji o bryłach.

3.2 Architektura systemu

Jako system rozpoznawania dotykanych obiektów 3D rozumie się zestaw powiązanych modułów, które współpracują ze sobą w celu realizacji wyznaczonych zadań. Architektura rozwiązania została zaprojektowana w taki sposób, aby możliwe było spełnienie wyznaczonych w sekcji 3.1 wymagań projektowych. Szczególna uwaga została poświęcona ograniczeniom wynikającym z użycia urządzeń mobilnych, krótkiemu czasowi odpowiedzi oraz możliwości obsługi systemu bez użycia wzroku. System został oparty na architekturze klient-serwer, w której odpowiedzialności zostały podzielone pomiędzy dwa główne moduły. Moduł kliencki, uruchamiany na urządzeniu mobilnym, odpowiada za rejestrację obrazu i obsługę interakcji z użytkownikiem. Część serwerowa, uruchamiana przez osobę obsługującą system na komputerze, realizuje zadania związane z analizą danych, detekcją obiektów i przygotowaniem informacji zwrotnej dla użytkownika. Komunikacja pomiędzy modułami odbywa się przez sieć bezprzewodową, umożliwiając przesyłanie danych oraz dwukierunkową wymianę komunikatów. Diagram blokowy architektury systemu przedstawiono na rysunku 3.1.

Zastosowanie komunikacji bezprzewodowej umożliwia użytkownikowi swobodną eksplorację obiektu bez ograniczania jego ruchów przewodami. Pozwala to również na fizyczne rozdzielenie urządzenia rejestrującego dane od komputera wykonującego obliczenia. Komputer, na którym uruchomiono część serwerową, nie musi znajdować się przy użytkowniku, a może zostać umieszczony w dowolnym miejscu z dostępem do tej samej sieci lokalnej co klient. Umożliwia to wykorzystanie istniejącej infrastruktury komputerowej bez konieczności przygotowywania specjalistycznego stanowiska pracy dla użytkownika. Takie rozwiązanie wprowadza jednakże zależność czasu komunikacji od jakości połączenia sieciowego.



Rysunek 3.1: Diagram blokowy architektury systemu

Aplikacja mobilna jest częścią systemu, z którą użytkownik wchodzi w bezpośrednią interakcję. Jej podstawowym zadaniem jest rejestracja obrazu obejmującego dłoń użytkownika oraz eksplorowany obiekt, a następnie przekazywanie go do części serwerowej systemu. Umożliwia ona również zainicjowanie żądania informacji o aktualnie eksplorowanym obiekcie. Po otrzymaniu z serwera komunikatu informacyjnego aplikacja odczytuje go użytkownikowi za pomocą syntezy mowy. Część mobilna nie odpowiada więc za interpretację wyników ani wybieranie informacji, lecz pełni rolę pośrednika między użytkownikiem a częścią obliczeniową systemu.

Do odpowiedzialności serwera należą wszelkie zadania związane z przetwarzaniem obrazu i przygotowaniem informacji zwrotnej. W jego skład wchodzi moduły odpowiedzialne za komunikację z aplikacją, detekcję obiektów, interpretację wyników i generowanie komunikatów informacyjnych. Model detekcji analizuje przesłane klatki i określa najbardziej prawdopodobną klasę obiektu na obrazie wraz z wartością pewności predykcji i jego ramką ograniczającą. Następnie wynik detekcji jest udostępniany pozostałym modułom części serwerowej, które aktualizują informacje, przygotowują tekst na podstawie bazy informacji, a następnie przekazują go do odczytania w części klienckiej. Możliwe jest wielokrotne zapytanie o informacje o tym samym obiekcie bez konieczności zmiany jego położenia, ani orientacji względem kamery.

Projekt zakłada również utworzenie narzędzi służących do przygotowania danych treningowych i uczenia modeli detekcji. Są to elementy, które nie są niezbędne do działania systemu, ale pełnią istotną rolę w jego rozwoju. Wśród nich znajdują się skrypty tworzące zbiory danych treningowych z zarejestrowanych filmów, dzielące istniejące zbiory danych na podzbiory treningowe i walidacyjne oraz uczące modele detekcji. Wspierają one również zapis wyników poszczególnych treningów, umożliwiając ich późniejszą analizę i porównanie.

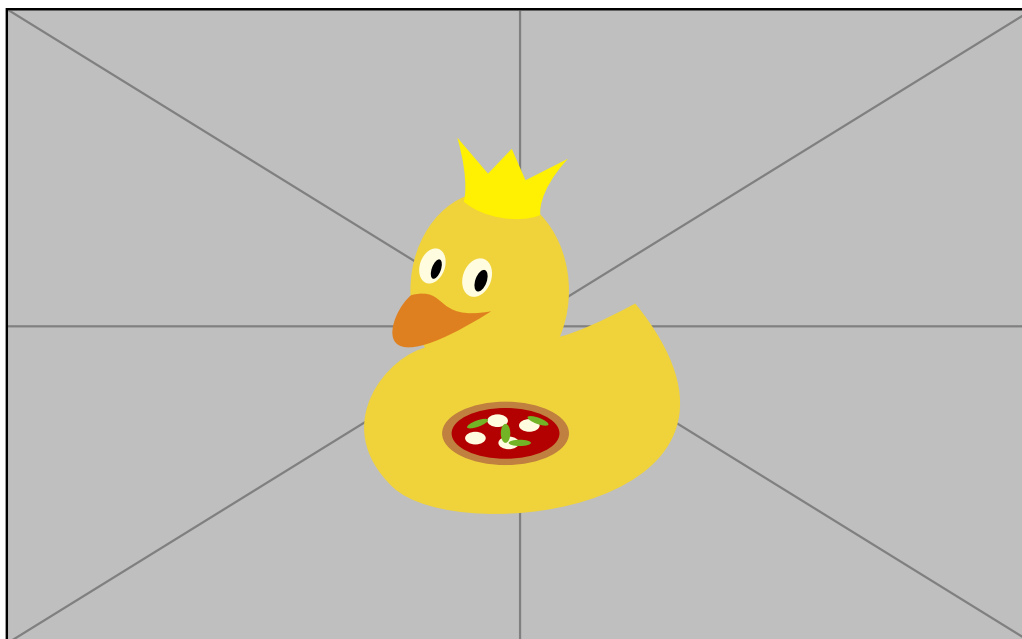
3.3 Dobór klas rozpoznawanych obiektów

System przewiduje rozpoznanie sześciu klas obiektów, będących podstawowymi bryłami geometrycznymi: sześcianu, kuli, walca, prostopadłościanu, czworościanu i stożka. Przyjęto, że wybrany zestaw klas jest wystarczający do realizacji założonego zakresu edukacyjnego oraz weryfikacji zaproponowanej koncepcji. Taki wybór klas umożliwia użytkownikowi zapoznanie się z podstawowymi pojęciami związanymi z geometrią przestrzenną, takimi jak ściana, powierzchnia boczna, krawędź czy wierzchołek. Bryły stanowią również podstawowe formy przestrzenne, z których możliwa jest budowa bardziej złożonych obiektów w wyniku ich łączenia, przekształcania i modyfikowania. Poznanie właściwości prostych brył może stanowić podstawę do rozumienia bardziej złożonych kształtów. Celem systemu nie jest więc objęcie całego zakresu nauki stereometrii, lecz wsparcie użytkownika w nauce podstawowych pojęć i zależności geometrycznych. Znaczenie stopniowego budowania bardziej złożonych brył z prostszych elementów ukazuje praca Figueiras i Arcavi [14] opisana w sekcji 2.1.

Na rysunku 3.2 przedstawiono fizyczne modele brył wykorzystanych do przygotowania danych treningowych. Ich powierzchnie zostały pomalowane przy użyciu stałej palety maksymalnie sześciu kolorów. Pozwala to na rozpoznawanie przez system poszczególnych części bryły, a także przypisanie każdej z nich informacji, która ma być przekazana użytkownikowi. Kolor jednak nie definiuje bryły, a służy jedynie rozpoznaniu jej części. W środowisku szkolnym mogą być wykorzystywane zestawy brył przygotowanych niezależnie, których kolorystyka będzie różnić się od zestawu użytego w pracach badawczych. Założeniem projektu jest więc, aby system rozpoznawał bryły na podstawie ich cech geometrycznych, a nie zastosowanej kolorystyki. Odporność modelu na zmianę kolorystyki wymaga osobnej analizy i weryfikacji eksperymentalnej.

Wybrane klasy obejmują obiekty o zróżnicowanej budowie, lecz posiadające wspólne cechy. Sześcian, prostopadłościan i czworościan posiadają ściany płaskie i wyraźne krawędzie, a kula, stożek oraz walec mają co najmniej jedną powierzchnię zakrzywioną. Te podobieństwa mogą skutkować podobnym wyglądem poszczególnych brył w zależności od ich orientacji względem kamery. Predykcja może okazać się szczególnie trudna w przypadku, gdy obiekt jest w dużej części zasłonięty przez dłoń użytkownika. Uwzględnienie takich przypadków w zbiorze uczącym jest istotne dla uczenia modelu detekcji w trudnych warunkach. Pomyślne rozpoznanie częściowo zasłoniętego obiektu pozwala ocenić odporność modelu na zasłonięcia bryły.

Każda klasa odpowiada jednemu rodzajowi bryły, niezależnie od jej orientacji oraz aktualnie widocznej powierzchni. Kolor i widoczna część obiektu nie stanowią osobnych klas, a są one analizowane dopiero po rozpoznaniu klasy przez model i zażądaniu informacji przez użytkownika. Po przeanalizowaniu wyników detekcji system przygotowuje odpowiedni komunikat informacyjny. Architektura systemu umożliwia późniejsze rozszerzenie



Rysunek 3.2: Bryły geometryczne wykorzystywane w projektowanym systemie

zbioru rozpoznawalnych klas o kolejne obiekty po przygotowaniu dodatkowych danych treningowych.

3.4 Uzasadnienie wyboru metody detekcji

Po przeprowadzeniu analizy literatury naukowej przedstawionej w rozdziale 2 zdecydowano się na wykorzystanie detektorów obiektów z rodziny YOLO. Ich sposób działania najlepiej pasuje do systemu, w którym istotne są krótki czas odpowiedzi i możliwości uruchomienia modelu detekcji na komputerach o ograniczonych zasobach obliczeniowych. Kompromis między skutecznością detekcji a szybkością działania modelu był najistotniejszym kryterium wyboru tej metody.

Na sposób funkcjonowania systemu szczególny wpływ ma szybkość działania modelu detekcji. Obraz przesyłany z aplikacji mobilnej do części serwerowej jest analizowany w sposób ciągły, gdy użytkownik eksploruje obiekt. Wynik detekcji powinien więc być dostępny w jak najkrótszym czasie, by odpowiadał bieżącej orientacji i położeniu obiektu względem kamery. Zbyt długi czas odpowiedzi może skutkować podaniem użytkownikowi informacji, która jest już nieaktualna. Jednocześnie, na czas odpowiedzi duży wpływ może mieć zdolność jednostki obliczeniowej do przetwarzania danych, a więc wymagane jest użycie modelu, który zapewni akceptowalny czas odpowiedzi również na słabszych komputerach. Zastosowanie bardziej złożonych detektorów mogłoby wydłużyć czas odpowiedzi prowadząc do przekazania nieaktualnej informacji użytkownikowi.

Istotną przesłanką wyboru detekcji obiektów w miejsce klasyfikacji jest zachodząca potrzeba dokładnego określenia położenia bryły na obrazie. Detektor podczas działania

zwraca informację o klasie, położeniu obiektu i wartości pewności predykcji. Określona ramka ograniczająca umożliwia następnie wyodrębnienie obszaru obrazu w którym znajduje się obiekt, co przy dalszej analizie ograniczy wpływ tła i innych elementów sceny. Informacja o położeniu obiektu jest następnie wykorzystywana do rozpoznania widocznej części bryły i przygotowania komunikatu.

Spośród dostępnych modeli rodziny YOLO rozwijanych przez Ultralytics zdecydowano się na wykorzystanie najnowszego wariantu YOLO26, a do badań wykorzystano jego warianty `nano` (n) i `small` (s). Różnią się one liczbą parametrów oraz kosztem obliczeniowym, co może wpływać na poprawę skuteczności detekcji, ale też wydłużenie czasu inferencji. Porównanie obu modeli pozwoli ocenić, czy zwiększenie złożoności modelu będzie rzeczywiście prowadziło do poprawy jakości wyników, uzasadniającej jego użycie w systemie, czy też wystarczające będzie użycie modelu o mniejszej liczbie parametrów. Ostateczny wybór modelu detekcji musi zostać dokonany na podstawie wyników badań eksperymentalnych, które zostaną przedstawione w rozdziale 6. Uwzględnić przy tym należy nie tylko skuteczność detekcji, ale również czas odpowiedzi i zapotrzebowanie na zasoby obliczeniowe.

3.5 Przepływ danych w systemie

Przepływ danych projektowanego rozwiązania rozpoczyna się w aplikacji mobilnej rejestrującej obraz obejmujący dłonie użytkownika i eksplorowaną bryłę. Kolejne klatki są przesyłane przez sieć bezprzewodową do części serwerowej. Transmisja realizowana jest w sposób ciągły, co sprawia, że serwer otrzymuje aktualny obraz niezależnie od działań użytkownika.

Po odebraniu klatki serwer przystępuje do jego analizy przekazując ją do modułu detekcji. Wynikiem jego przetworzenia jest przewidywana klasa obiektu, jego ramka ograniczająca oraz wartość pewności wykonanej predykcji. Wynik detekcji aktualizowany jest po przetworzeniu każdej klatki obrazu, dzięki czemu część serwerowa przechowuje możliwie aktualną informację o rozpoznanym obiekcie. Predykcja nie powoduje automatycznego przygotowania komunikatu informacyjnego, a proces ten rozpoczyna się dopiero gdy użytkownik zażąda informacji.

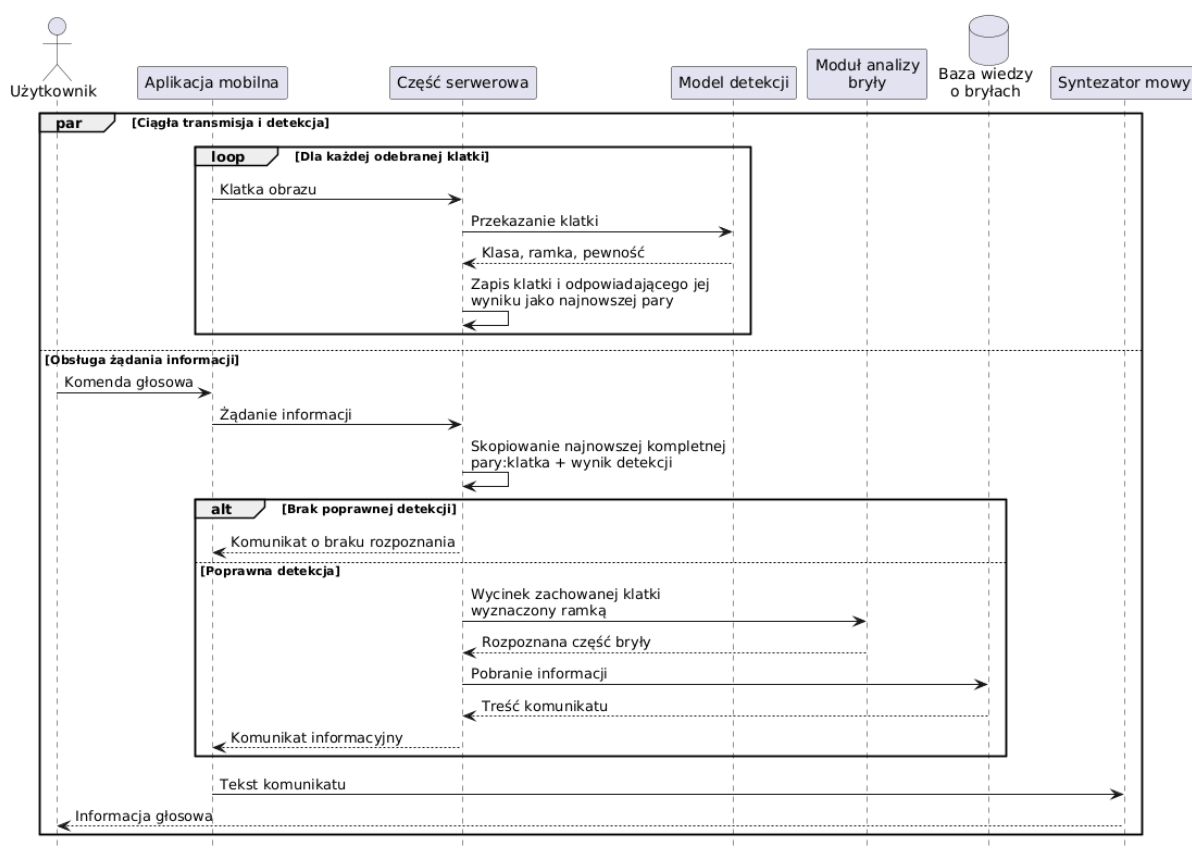
Komenda głosowa jest rozpoznawana przez aplikację mobilną, a następnie do serwera wysyłany jest komunikat sterujący, który uruchamia część interpretującą ostatnie wyniki detekcji. W momencie jego otrzymania serwer pobiera najnowszą kompletną parę obejmującą klatkę wraz z wynikiem dokonanej na niej predykcji. Dzięki temu dalsza analiza wykorzystuje obraz którego dotyczy wynik zachowanej detekcji, nawet jeżeli serwer w międzyczasie otrzymał kolejne klatki. Na czas analizy wyników detekcji i przygotowania komunikatu informacyjnego nie jest przerywana transmisja kolejnych klatek obrazu,

a serwer wciąż dokonuje detekcji obiektów. Nowsze wyniki nie wpływają na aktualnie przygotowywaną odpowiedź.

Następnie sprawdzane jest, czy ostatni wynik predykcji może zostać wykorzystany do przygotowania komunikatu informacyjnego. Jeżeli model nie wykrył żadnego przedmiotu lub wartość pewności predykcji jest niższa od przyjętego progu, zwracany jest komunikat informujący o braku możliwości rozpoznania widocznej powierzchni. Jeżeli wynik detekcji jest poprawny, system wyodrębnia z zachowanej klatki obszar obrazu wyznaczony ramką ograniczającą. Celem tego etapu jest określenie widocznej powierzchni bryły na podstawie zastosowanych oznaczeń kolorystycznych. Rozpoznana klasa obiektu ogranicza zbiór możliwych powierzchni do przypisanych do danej bryły w bazie wiedzy. Na podstawie wyniku analizy wybierany jest odpowiedni komunikat informacyjny, który następnie przekazywany jest do aplikacji mobilnej.

Przekazanie komunikatu informacyjnego użytkownikowi odbywa się poprzez odczytanie tekstu przez syntezytor mowy. W tym czasie system wciąż odbiera kolejne klatki i dokonuje detekcji obiektów, a użytkownik może kontynuować eksplorację obiektu i ponownie żądać informacji.

Równoległe działanie procesu transmisji i detekcji oraz procesu przygotowania komunikatu informacyjnego zostało przedstawione na diagramie sekwencji, na rysunku 3.3.



Rysunek 3.3: Diagram sekwencji przepływu danych w projektowanym systemie

3.6 Ograniczenia przyjętej koncepcji

Przyjęta koncepcja systemu pozwala spełnić założone wymagania funkcjonalne i nie-funkcjonalne, ale wiąże się również z pewnymi ograniczeniami wynikającymi z zakresu pracy, sposobu przygotowania danych oraz zastosowanej architektury.

Możliwości rozpoznawania brył są ograniczone wyłącznie do wcześniej zdefiniowanych klas obiektów, dla których wcześniej zostały przygotowane dane treningowe. Obiekty nie-należące do żadnej z klas nie zostaną poprawnie rozpoznane, a podobieństwo ich kształtu do jednej z istniejących klas może skutkować błędnym przypisaniem etykiety. Rozszerzenie zbioru klas wymaga zebrania i przygotowania dodatkowych danych uczących oraz ponownego wytrenowania modelu detekcji. Koncepcja zakłada również, że pojedyncza klatka obrazu przedstawia wyłącznie jedną bryłę, która aktualnie jest eksplorowana przez użytkownika. W przypadku wykrycia kilku obiektów system nie dysponuje mechanizmem pozwalającym na określenie którego z nich dotyczy żądanie informacji. Może to skutkować przygotowaniem komunikatu o niepoprawnej treści. System nie pokrywa również całego zakresu stereometrii, a koncentruje się jedynie na podstawowych bryłach geometrycznych, wykorzystywanych w edukacji i nauce pojęć związanych z geometrią przestrzenną.

Podstawowym źródłem danych jest obraz RGB rejestrowany przez kamerę urządzenia mobilnego. Jego jakość zależy od warunków oświetleniowych, tła, odległości od kamery i stopnia zasłonięcia. W pojedynczej klatce część cech charakterystycznych bryły może być niewidoczna przez zasłonięcia, co może znacznie utrudnić lub uniemożliwić poprawną detekcję. Zastosowanie wyłącznie ramek ograniczających do określenia położenia nie wyklucza udziału dłoni lub tła w analizie wycinku obrazu. Zastosowanie segmentacji umożliwiłoby dokładniejsze oddzielenie obiektu od tła i dłoni, ale wymagałoby przygotowania dodatkowych masek dla danych uczących. Należy również zwrócić uwagę na różnorodność urządzeń mobilnych, które mogą znacznie różnić się jakością rejestrowanego obrazu.

Kolejne ograniczenie wynika z przyjętych oznaczeń kolorystycznych na powierzchniach brył. Modele użyte w pracy zostały pomalowane przy użyciu stałej palety barw, natomiast w środowisku szkolnym mogą wystąpić zestawy o innej kolorystyce. Detektor powinien więc rozpoznawać bryły przede wszystkim na podstawie ich cech charakterystycznych. W przypadku pojedynczego zestawu brył w zbiorze uczącym może zaistnieć znacząca podatność polegająca na tym, że model nauczy się rozpoznawać bryły na podstawie koloru. Jednocześnie moduł analizy wycinka obrazu wymaga, aby kolory były łatwo odróżnialne, a przypisanie kolorów do powierzchni było zgodne z konfiguracją zapisaną w bazie wiedzy. Wykorzystanie zestawu brył o innej kolorystyce wymaga przygotowania dodatkowej konfiguracji, która pozwoli na przypisanie odpowiednich powierzchni do informacji znajdujących się w systemie. Zmiany palety, odcieni, połysku czy balansu bieli mogą skutkować błędami na etapie określania widocznej części bryły i przygotowywania komunikatu informacyjnego.

Przyjęta architektura klient-serwer wymaga, aby oba urządzenia miały stałe połączenie z tą samą siecią lokalną. Jakość połączenia może mieć wpływ na jakość, prędkość przesyłanych obrazów i czas odpowiedzi systemu. Rozdzielenie odpowiedzialności wymaga również uruchomienia części serwerowej, co sprawia, że system nie jest w pełni autonomiczny i wymaga obecności osoby obsługującej komputer. Aplikacja mobilna nie posiada wbudowanego modułu detekcji, a więc nie jest możliwe jej autonomiczne działanie. Żądanie głosowe uruchamiające przesłanie komunikatu sterującego do serwera wymaga od aplikacji mobilnej zdolności do rozpoznawania mowy, wymagając dostępu do mikrofonu i braku znacznego hałasu w tle. Dodatkowe opóźnienie odpowiedzi serwera może wynikać z wielu czynników, takich jak obciążenie sieci, inferencji modelu czy zdolności obliczeniowej jednostki serwerowej.

Koncepcja zakłada również, że pojedyncza instancja serwera może obsługiwać jednocześnie tylko jednego klienta. Umożliwienie obsługi wielu klientów wymagałoby równoległego przetwarzania wielu strumieni danych, przez co należałoby rozdzielić dostępne zasoby obliczeniowe na wiele procesów. To skutkowałoby wydłużeniem czasu inferencji i przygotowania odpowiedzi, a tym samym wydłużeniem czasu odpowiedzi systemu. Możliwe również jest zastosowanie architektury rozproszonej, gdzie część obliczeniowa byłaby uruchomiona na kilku komputerach, jednak wymagałoby to dodatkowej infrastruktury i zwiększenia kosztów przygotowania i wdrożenia systemu.

Wymienione ograniczenia nie wykluczają możliwości weryfikacji przyjętej koncepcji, ale wyznaczają zakres, w którym ma być zastosowana. Wpływ ograniczeń powinien zostać uwzględniony w analizie wyników badań eksperymentalnych, a wnioski powinny być uwzględnione przy formułowaniu dalszych kierunków rozwoju systemu.

Rozdział 4

Implementacja systemu

4.1 Wykorzystane technologie i organizacja projektu

Implementacja systemu obejmuje część mobilną, serwerową oraz narzędzia pomocnicze do przygotowywania danych i uczenia modeli. Podział ten odpowiada założonej architekturze klient–serwer przedstawionej w rozdziale 3.

Do przygotowania aplikacji mobilnej wykorzystano technologię Kotlin Multiplatform oraz Compose Multiplatform. Ich użycie umożliwiło stworzenie pojedynczej bazy kodu możliwej do uruchomienia zarówno na systemie Android, jak i iOS, przy jednoczesnym zachowaniu osobnych implementacji elementów zależnych od platformy. Aplikacja implementuje funkcjonalności transmisji obrazu, komunikacji z serwerem, interfejs użytkownika oraz zarządzania stanem. Komunikacja jest realizowana za pomocą biblioteki `Ktor`.

Część serwerowa została zaimplementowana w języku Python. Do obsługi połączenia sieciowego wykorzystano bibliotekę `websockets`, a do dekodowania obrazu i jego przetwarzania bibliotekę `OpenCV`. Inferencja modeli sieci neuronowych odbywa się przy użyciu biblioteki `Ultralytics`, która działa w środowisku `PyTorch`. Interfejs graficzny przygotowano przy użyciu biblioteki `PySide6`.

Najważniejsze technologie wykorzystane w implementacji poszczególnych komponentów systemu zostały zestawione w tabeli 4.1.

Repozytorium rozwiązania składa się z pięciu katalogów:

- `KMP` – aplikacja mobilna dla systemów Android i iOS;
- `server` – część serwerowa systemu;
- `training` – katalog narzędzi do trenowania modeli sieci neuronowych;
- `dataset_splitter` – skrypt do podziału zbioru danych;
- `video_to_dataset` – narzędzie do ekstrakcji klatek z nagrań wideo.

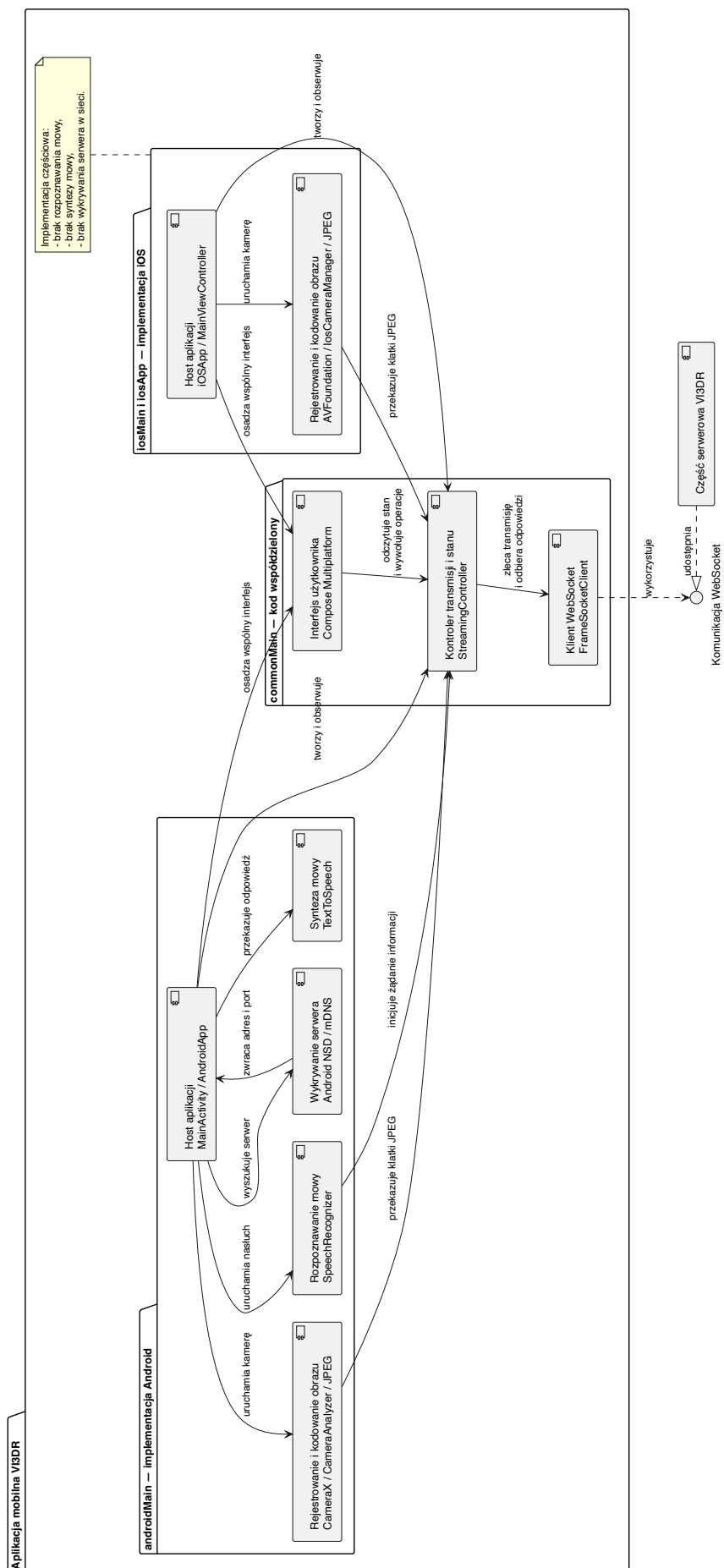
Rozdzielenie kodu poszczególnych części umożliwia niezależny rozwój i testowanie komponentów systemu. Szczególnie istotne jest rozdzielenie narzędzi związanych z przygotowaniem danych i trenowaniem modeli od kodu aplikacji mobilnej i serwera. Wyznacza to wyraźne granice między częścią wykonawczą systemu i jego komponentami badawczymi.

Tabela 4.1: Najważniejsze technologie wykorzystane w poszczególnych częściach systemu

Część systemu	Technologie i biblioteki	Zastosowanie
Aplikacja mobilna	Kotlin Multiplatform, Compose Multiplatform, Ktor	Współdzielona logika aplikacji, interfejs użytkownika i komunikacja WebSocket
Android	CameraX, SpeechRecognizer, TextToSpeech, Android NSD	Rejestrowanie obrazu, rozpoznawanie komend głosowych, synteza mowy i wykrywanie serwera w sieci lokalnej
iOS	AVFoundation, SwiftUI	Rejestrowanie obrazu, podgląd kamery i osadzenie współdzielonego interfejsu aplikacji
Część serwerowa	Python, websockets, OpenCV, Ultralytics, PyTorch, webcolors	Obsługa komunikacji, dekodowanie i przetwarzanie obrazu, wykonywanie detekcji obiektów i dopasowanie kolorów powierzchni
Interfejs serwera	PySide6, QWebChannel	Prezentowanie obrazu, stanu połączenia i bieżących wyników detekcji
Narzędziobadawcze	Ultralytics, OpenCV, NumPy, scikit-learn, matplotlib	Przygotowanie zbioru danych, uczenie, testowanie i analiza wyników modeli

4.2 Implementacja aplikacji mobilnej

Aplikacja mobilna stanowi część kliencką systemu. Do jej zadań należy rejestrowanie obrazu z kamery urządzenia, przekazywanie kolejnych klatek do serwera, oraz obsługa bezdotykowa systemu za pomocą komend głosowych. Podział aplikacji przedstawiono na diagramie komponentów, na rysunku 4.1.



Rysunek 4.1: Diagram komponentów aplikacji mobilnej z podziałem na kod współdzielony i implementacje platformowe

4.2.1 Organizacja kodu aplikacji

Implementacja aplikacji mobilnej została zrealizowana z wykorzystaniem technologii Kotlin Multiplatform i Compose Multiplatform. Kod aplikacji umieszczono w module `composeApp` i podzielono na pakiety odpowiadające częściom wspólnym i zależnym od platformy. W katalogu `commonMain` znajduje się kod współdzielony obsługujący logikę zarządzania stanem aplikacji, komunikację z serwerem, obsługę połączenia oraz wspólne elementy interfejsu użytkownika. Katalogi `androidMain` i `iosMain` zawierają implementacje korzystające z natywnych funkcji systemów Android i iOS, takich jak rejestrowanie obrazu, obsługa komend głosowych i odczytywanie komunikatów z serwera.

Centralny punkt aplikacji stanowi klasa `StreamingController`, zarządzająca stanem połączenia i transmisji. Kontroluje ona również częstotliwość przesyłania kolejnych ramek oraz obsługuje żądania użytkownika i odpowiedzi serwera. Stan kontrolera wykorzystywany jest przez interfejs użytkownika przygotowany w technologii Compose Multiplatform. Do współdzielonego interfejsu użytkownika dołączany jest natywny podgląd z kamery urządzenia.

Zastosowanie technologii Kotlin Multiplatform umożliwiło rozpoczęcie prac nad aplikacją mobilną jednocześnie dla systemów Android i iOS. Wstępne testy implementacji ukazały jednak istotne problemy ze stabilnością i wydajnością aplikacji na systemie iOS. Z tego względu dalsze prace skupiono wyłącznie na aplikacji dla systemu Android, dla której zaimplementowano wszystkie funkcjonalności systemu. Kod aplikacji dla systemu iOS pozostawiono w repozytorium aby umożliwić dalszy rozwój tej gałęzi w przyszłości.

4.2.2 Rejestrowanie i przygotowanie obrazu

Punktem wejścia do aplikacji jest klasa `MainActivity`, która uruchamia interfejs użytkownika i konfiguruje elementy wymagające dostępu do natywnych mechanizmów systemu: kamery, mikrofonu i syntezy mowy. Rejestrowanie obrazu z kamery realizowane jest przy użyciu biblioteki `CameraX`, która udostępnia strumień kolejnych ramek. Ten obsługiwany jest mechanizmem `ImageAnalysis`, korzystając ze strategii `STRATEGY_KEEP_ONLY_LATEST`. Przyjęcie jej sprawia, że w przypadku w którym aplikacja nie zdąży przetworzyć wszystkich ramek, zostaną one pominięte, a do przetwarzania zostanie wybrana najnowsza z nich. To podejście sprawia, że aplikacja nie generuje dodatkowego narzutu związanego z rosnącą długością kolejki obrazów i ogranicza opóźnienia pomiędzy pozycją bryły, a danymi przesyłanymi do serwera.

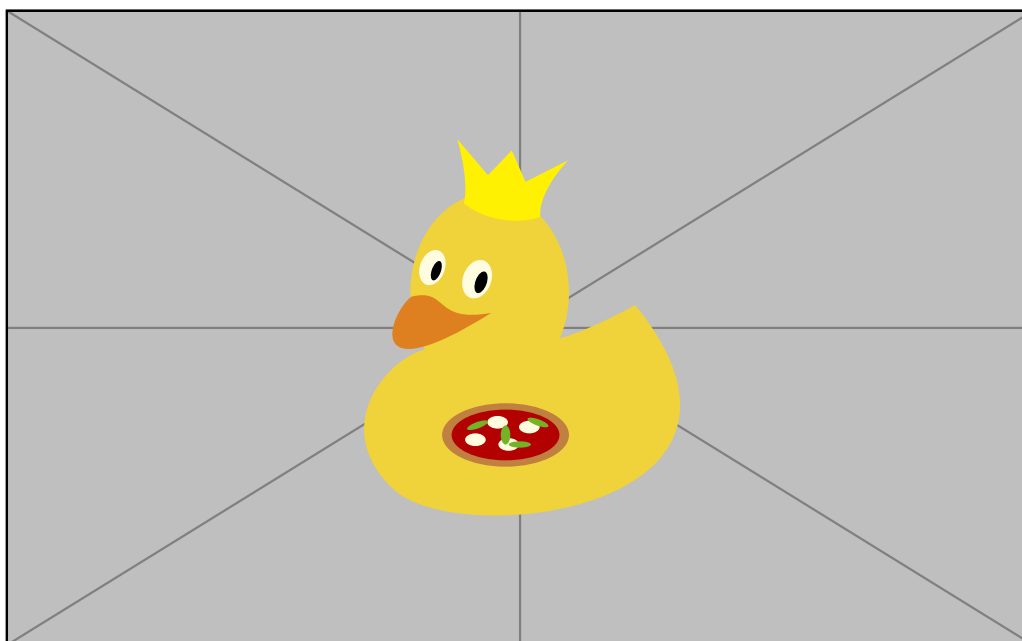
Klatki zarejestrowane z kamery są przetwarzane w klasie `CameraAnalyzer`. Dane przekazywane przez `CameraX` są konwertowane z formatu `YUV_420_888` do układu `NV21`, a następnie kompresowane do formatu `JPEG`. Wykorzystanie sieci bezprzewodowej do przesyłania danych premiuje kompresję danych, ponieważ obniża to wymagania dotyczące przepustowości łącza i zmniejsza opóźnienia w transmisji. Po zakończeniu przygotowania

klatki jest ona przekazywana do dalszej obsługi, a obiekt `ImageProxy` jest zwalniany, by możliwe było przyjęcie kolejnych danych.

4.2.3 Zarządzanie transmisją

Po przygotowaniu klatki, jest ona przekazywana do części wspólnej aplikacji. Każde wysłanie poprzedzone jest sprawdzeniem stanu połączenia: czy transmisja została aktywowana, czy połączenie z serwerem pozostaje otwarte, oraz czy wysłanie nie spowoduje przekroczenia ustalonego limitu FPS. Odbywa się to w klasie `StreamingController`, która przekazuje przygotowaną tablicę bajtów do wydzielonego klienta komunikacji. Rozdzielenie warstwy komunikacji od logiki aplikacji sprawia, że moduł kamery nie musi znać sposobu komunikacji z serwerem, a klient może być w przyszłości wymieniony na inny, np. w wypadku zmiany protokołu komunikacyjnego.

Interfejs użytkownika umożliwia włączanie i wyłączanie transmisji obrazu oraz zmianę parametrów połączenia. Są to: adres i port serwera, częstotliwość przesyłania klatek oraz jakość przesyłanego obrazu. Zrzut ekranu 4.2 przedstawia wygląd interfejsu aplikacji mobilnej. Komunikaty informujące o zmianie stanu połączenia są wyświetlane w formie toastów. Szczegóły protokołu komunikacyjnego, format wymienianych wiadomości oraz sposób nawiązywania połączenia są opisane w sekcji 4.4 poświęconej komunikacji klient–serwer.



Rysunek 4.2: Interfejs aplikacji mobilnej

4.2.4 Obsługa interakcji głosowej

Interakcja z systemem została zrealizowana w sposób, który nie wymaga od użytkownika niewidomego korzystania z ekranu dotykowego. Wersja aplikacji dla systemu Android wykorzystuje wbudowane mechanizmy rozpoznawania mowy za pomocą mechanizmu `SpeechRecognizer` w klasie `VoiceCommandRecognizer`. Rozpoznany tekst analizowany jest pod kątem występowania komendy głosowej, a w przypadku jej wykrycia, kontroler aplikacji wysyła do serwera odpowiednie żądanie. Obecna wersja aplikacji obsługuje jedynie komendy żądania informacji i nie przewiduje możliwości sterowania transmisją obrazu. Przyszłe wersje systemu mogą zostać rozbudowane o te funkcjonalności, ale założenie projektowe przewiduje, że osoba niewidoma będzie korzystała z systemu w towarzystwie osoby widzącej, mogącej sterować transmisją obrazu.

Odpowiedzi otrzymywane z serwera są zapisywane w stanie aplikacji, a następnie przekazywane do mechanizmu `TextToSpeech`. Syntezator mowy odczytuje komunikaty z serwera w języku urządzenia. Jego wykorzystanie uniezależnia użytkownika od konieczności wejścia w interakcję z ekranem dotykowym podczas żądania i odbierania informacji.

Mechanizmy rozpoznawania i syntezy mowy są silnie zależne od platformy i systemu operacyjnego. Inicjacja żądania i obsługa treści odpowiedzi odbywa się jednak w części wspólnej aplikacji. Umożliwia to w przyszłości rozszerzenie aplikacji o kolejne systemy, bez konieczności ponownej implementacji sposobu komunikacji z serwerem.

4.3 Implementacja części serwerowej

Część serwerowa odpowiada za odbieranie danych z aplikacji mobilnej, dekodowanie ich, wykonywanie detekcji obiektów i prezentowanie stanu systemu. Do jej przygotowania wykorzystano język Python i można ją uruchomić w formie aplikacji konsolowej lub z interfejsem graficznym.

4.3.1 Organizacja i uruchamianie serwera

Głównym punktem wejścia do serwera jest klasa `ApplicationRuntime`, której zadaniem jest utworzenie i połączenie modułów: serwera `WebSocket`, modułu detekcji obiektów oraz komponentu publikującego usługę w sieci lokalnej. W procesie uruchamiania serwera wczytywany jest model detekcji, funkcja wykonująca inferencję przekazywana jest do modułu obsługi obrazów i uruchamiane są mechanizmy komunikacji i publikacji usługi. W przypadku uruchomienia serwera z interfejsem graficznym, główny wątek aplikacji obsługuje interfejs Qt, a osobny wątek z własną pętlą zdarzeń `asyncio` obsługuje właściwy serwer. Dzięki temu, że oba sposoby uruchamiania serwera korzystają z tej samej klasy `ApplicationRuntime`, sposób działania nie jest zależny od wyboru interfejsu.

Kod odpowiedzialny za połączenia został rozdzielony na dwa poziomy. Serwer WebSocket jest zarządzany przez klasę `CaptureServer`, która odpowiada również za kontrolę aktywnej sesji. Dla każdego klienta jest natomiast tworzona instancja klasy `CaptureSession`, odpowiadająca za obsługę wiadomości i przetwarzanie przychodzących obrazów.

Ustawienia serwera zgromadzono w pliku `settings.py`, gdzie możliwe jest wprowadzenie zmian dotyczących adresu i portu serwera, parametrów kolejki, konfiguracji mDNS oraz modelu detekcji. Ustawienia dotyczące bezpośrednio detekcji, jak jej próg pewności, rozmiar obrazu wejściowego i urządzenie wykorzystywane do obliczeń są przekazywane do modułu detekcji w momencie jego tworzenia. Zmiana tych parametrów wymaga ponownego uruchomienia serwera.

Możliwa jest wymiana modelu detekcji na inny bez konieczności modyfikacji pozostałych elementów systemu. Wariant z interfejsem użytkownika umożliwia wybranie modelu z pliku `.pt`, korzystając z systemowego okna dialogowego. Jest on następnie ładowany przez klasę `YOLODetector`. Obiekt detektora nie jest zmieniany, więc sesje odwołują się do tej samej funkcji przetwarzającej, ale kolejne iteracje detekcji wykonywane są na nowym modelu. Wariant konsolowy wymaga podania ścieżki do modelu w pliku konfiguracyjnym i ponownego uruchomienia serwera.

Podczas tworzenia obiektu `ApplicationRuntime` wczytywana jest również przygotowana wcześniej baza wiedzy zawierająca informacje o powierzchniach brył oraz odpowiadające im informacje. Dane przechowywane są w pojedynczym pliku `.json` o ustalonej strukturze. Loader przekształca zawartość pliku do struktur danych reprezentujących bryły, powierzchnie i parametry dopasowania kolorów. Baza może zostać przeładowana w trakcie działania serwera, a przy obsłudze kolejnych żądań informacji wykorzystywana jest nowa konfiguracja.

4.3.2 Odbieranie i przygotowanie obrazu

Po nawiązaniu połączenia z klientem klasa `CaptureServer` tworzy obiekt `CaptureSession` do którego przekazuje funkcję wykonującą detekcję. Uruchomiona zostaje asynchroniczna pętla odbioru wiadomości. Jeżeli serwer rozpozna wiadomość binarną, jest ona traktowana jako klatka obrazu i przekazywana do dalszej obsługi.

Aby ograniczyć liczbę zaległych klatek, rozmiar kolejki wejściowej ustawiono na pojedynczą wiadomość. Metoda `_drain_to_latest_frame` sprawdza, czy w kolejce znajdują się kolejne dane binarne, a jeżeli tak, to zastępuje starszy obraz nowszym. Taki mechanizm nie gwarantuje zachowania wszystkich klatek, ale zmniejsza ryzyko wykonania detekcji na obrazie, który nie jest już aktualny i nie odpowiada aktualnemu położeniu bryły. Jeżeli serwer nie zdąży przetworzyć starszych klatek są one pomijane, a do detekcji przekazywana jest najnowsza z nich.

Wiadomości binarne są wstępnie sprawdzane pod kątem występowania plików JPEG na podstawie dwóch pierwszych bajtów FF D8. Następnie są one przekazywane do tablicy NumPy, gdzie dekodowane są do formatu z kanałami BGR korzystając z funkcji `cv2.imdecode`. W przypadku niepowodzenia dana wiadomość zostaje pominięta bez przerwania sesji. Poprawnie zdekodowany obraz jest przekazywany do modelu detekcji. Po zakończeniu predykcji wynik zostaje zapisany razem z kopią odpowiadającej mu klatki obrazu w strukturze `DetectionSnapshot`. Dzięki temu przechowywana jest spójna para obrazu i wyniku wykonanej na nim detekcji. Dostęp do aktualnego snapshotu chroniony jest blokadą wątkową, aby nie było możliwe jednoczesne zapisywanie i odczytywanie danych.

W chwili otrzymania żądania informacji pobierany jest najnowszy kompletny `DetectionSnapshot`. Dalsza analiza oparta jest na informacjach w nim zawartych, a kolejne obrazy mogą być niezależnie odbierane i przetwarzane przez serwer.

Wynikiem przetwarzania, zwracanym przez model, jest między innymi klatka z naniesionymi ramkami detekcji, która wykorzystywana jest w oknie podglądu. Częstotliwość z jaką aktualizowany jest podgląd nie jest zależna od inferencji. Brak aktualizacji podglądu nie oznacza, że nie została wykonana detekcja, a jedynie, że nie zdołała ona zostać wyświetlona w interfejsie.

4.3.3 Moduł detekcji obiektów

Ładowaniem i wykonywaniem detekcji obiektów zajmuje się klasa `YOLODetector`. W momencie jej inicjalizacji otrzymuje ona ścieżkę do modelu, minimalną wartość pewności detekcji, rozmiar obrazu wejściowego oraz urządzenie wykorzystywane do obliczeń. Model jest ładowany przy użyciu klasy `YOLO` z biblioteki `Ultralytics`. Wśród dostępnych urządzeń do obliczeń można wybrać warianty udostępniane przez bibliotekę `PyTorch`, w tym CPU, GPU i MPS. Domyślnie, w przypadku braku wyboru urządzenia, ten zostaje wybrany automatycznie przez bibliotekę. Inferencja modelu odbywa się poprzez uruchomienie metody `annotate` w której obraz przekazywany jest do funkcji `model.predict` razem z parametrami wynikającymi z konfiguracji.

Istnieje możliwość, że w wyniku detekcji na obrazie zostanie oznaczonych wiele obiektów. W takim przypadku wykorzystywana jest metoda `_extract_best_detection`, która wybiera detekcję o najwyższej wartości pewności. Następnie pobiera z niej informacje o klasie obiektu oraz jego współrzędnych na obrazie w postaci

$$B = (x_1, y_1, x_2, y_2) \quad (4.1)$$

ograniczonych do wymiarów obrazu. Eliminuje to możliwość wyznaczenia obszaru znajdującego się poza jego granicami.

Informacje o obiekcie są przechowywane w strukturze `DetectionResult`, która zawiera nazwę przewidywanej klasy, wartość pewności detekcji i obiekt `DetectionBox` z informacjami o położeniu ramki ograniczającej. Struktura ta posiada metodę, która umożliwia wyodrębnienie z obrazu fragmentu odpowiadającego zapisanej ramce. Po zakończeniu predykcji tworzony jest obiekt `DetectionSnapshot`, zawierający kopię przetworzonej klatki oraz odpowiadający jej `DetectionResult`. Zapisu i odczyt są chronione blokadą wątkową, aby zapobiec sytuacji, w której podczas przygotowywania odpowiedzi wykorzystana zostałaby klatka pochodząca z innej iteracji modelu detekcji. Snapshot zostaje zaktualizowany po każdej zakończonej predykcji i może zostać pobrany w chwili żądania informacji.

Ładowanie modelu i wykonywanie detekcji również są operacjami chronionymi blokadą wątkową, aby uniknąć sytuacji, w której model mógłby zostać nadpisany w trakcie wykonywania predykcji.

4.3.4 Analiza wycinka i przygotowanie informacji zwrotnej

Szczegółowa analiza obiektu jest wykonywana wyłącznie po otrzymaniu żądania informacji od klienta. Dzięki temu detekcja pozostaje oddzielona od pozostałych operacji wymaganych jedynie do przygotowania informacji zwrotnej. Odebranie komunikatu `info-request` skutkuje utworzeniem obiektu `AnalysisContext`, przechowującego najnowszy kompletny `DetectionSnapshot` oraz referencję do bazy wiedzy. Dzięki takiemu mechanizmowi późniejsze procesy nie wpływają na już rozpoczętą analizę.

Analiza wykonywana jest używając klasy `AnalysisWorker`. Wykorzystuje ona osobny wątek w obiekcie `ThreadPoolExecutor`, dzięki czemu może wykonywać obliczenia poza główną pętlą zdarzeń serwera. W ramach zadania, na podstawie wyniku detekcji w kontekście, wyodrębniany jest fragment obrazu ograniczony ramką detekcji. Jeżeli snapshot nie istnieje, model nie wykrył obiektu lub wycinek jest niepoprawny, dalsza analiza nie jest wykonywana.

Właściwe przetwarzanie wycinka obrazu odbywa się w klasie `SliceAnalyzer`. Jako parametry otrzymuje ona wycinek obrazu, nazwę rozpoznanej klasy obiektu oraz bazę wiedzy. Klasa służy jedynie do pobrania odpowiednich informacji o powierzchniach bryły i dopasowaniu kolorów. Niweluje to sposobność powielania obliczeń dla obiektów, których nie dotyczy analiza.

Rozpoznanie widocznej powierzchni odbywa się w module `SurfaceMatcher`. Do każdej powierzchni w bazie wiedzy dopasowywany jest kolor referencyjny zapisany jako nazwa koloru CSS oraz komunikat informacyjny. Nazwa koloru jest przekształcana do przestrzeni RGB, a następnie do układu BGR i przestrzeni Lab. Wycinek obrazu również

jest przekształcany do przestrzeni Lab, a następnie wykonywane jest dopasowanie kolorów. Odbywa się ono dla każdego piksela wycinka poprzez obliczenie kwadratu odległości euklidesowej od kolorów referencyjnych wszystkich powierzchni. Piksel jest przypisywany do koloru wyłącznie wtedy, gdy uzyskana odległość nie przekracza ustalonego progu `max_distance` znajdującego się w konfiguracji bryły. Następnie zliczana jest liczba pikseli należących do każdej z powierzchni. Wynikiem jest informacja o powierzchni, której kolor został dopasowany do największej liczby pikseli wycinka.

Po określeniu powierzchni `SliceAnalyzer` zwraca komunikat informacyjny, który następnie jest przekazywany do klienta. Jeżeli zajdzie błąd w trakcie analizy, np. nie zostanie wykryta powierzchnia, klient otrzymuje komunikat: "Nie udało się rozpoznać widocznej powierzchni".

4.3.5 Organizacja przetwarzania współbieżnego

Obsługa serwera i sesji odbywa się w pętli zdarzeń biblioteki `asyncio`. Operacje sieciowe realizowane są w sposób asynchroniczny, dzięki czemu oczekiwanie na nie nie blokuje procesu serwera.

Operacje, których wykonanie może trwać dłużej od pozostałych, a tym samym mogące blokować pętlę zdarzeń są wykonywane poza głównym wątkiem serwera. Wykorzystują one mechanizm `asyncio.to_thread`, a należą do nich: ładowanie modelu, inferencja, rejestrowanie i wyrejestrowanie usługi w sieci lokalnej. Inferencja, mimo oddzielnego wątku, pozostaje operacją sekwencyjnego przepływu obrazu – sesja oczekuje na jej zakończenie zanim rozpocznie kolejną iterację.

Odmienne podejście przyjęto w przypadku analizy wycinka obrazu uruchamianej na żądanie klienta. Tutaj, po otrzymaniu żądania, synchronicznie tworzony jest obiekt kontekstu analizy. Następnie tworzone jest nowe zadanie `asyncio`, a obliczenia są przekazywane do klasy workera. Wykorzystuje on obiekt `ThreadPoolExecutor` z pojedynczym wątkiem, co nie blokuje głównej pętli zdarzeń serwera. W czasie wykonywania analizy, serwer może odbierać kolejne klatki i wykonywać na nich detekcję obiektów. Nie wpływa to na rozpoczętą analizę, ponieważ korzysta ona ze snapshotu zapisanego w kontekście w momencie otrzymania żądania. Po zakończeniu pracy wynik powraca do pętli zdarzeń, gdzie następuje jego dalsza obsługa. Zamknięcie sesji skutkuje anulowaniem wszystkich zadań analizy, które nie zostały jeszcze zakończone.

Wariant desktopowy dzieli przetwarzanie na dwa wątki wykonania. Główny wątek obsługuje interfejs graficzny, a backend uruchamiany jest w osobnym wątku z własną pętlą zdarzeń. Operacje wykonywane na interfejsie graficznym przez użytkownika są przekazywane do backendu poprzez mechanizm korutyn biblioteki `asyncio`. Dane przekazywane z backendu do interfejsu graficznego są emitowane w formie sygnałów Qt. Stan współdzielony jest chroniony blokadą wątkową.

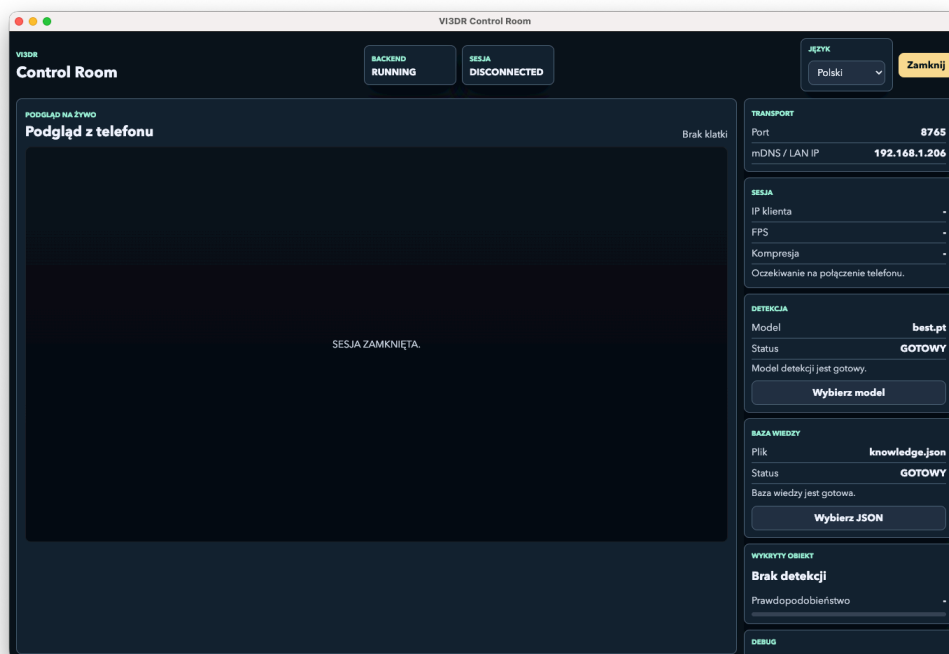
4.3.6 Interfejs graficzny

Do przygotowania interfejsu graficznego wykorzystano bibliotekę PySide6, Qt WebEngine i mechanizm `QWebChannel`. Warstwa prezentacji przygotowana jest w technologiach webowych: HTML, CSS i JavaScript. Komunikacja między interfejsem a backendem jest realizowana przez obiekt `DesktopBridge`.

Klasą utrzymującą informacje dotyczące działania serwera jest `BackendController`, która pośredniczy w wymianie danych między interfejsem Qt a backendem. Do jej zadań należy odbiór obrazu podglądu, klasy obiektu, wartości pewności detekcji, stanu sesji i metryk transmisji. Dane, które mają zostać zaprezentowane użytkownikowi są następnie przekazywane do warstwy JavaScript z wykorzystaniem sygnałów Qt.

Obraz podglądu jest kodowany do JPEG i przekazywany do interfejsu w formie adresu danych w formacie Base64. Takie podejście niweluje konieczność zapisu danych na dysku. Informacje o wynikach detekcji są przekazywane oddzielnie, a częstotliwość ich aktualizacji została ograniczona w celu zmniejszenia narzutu związanego z renderowaniem interfejsu.

Aplikacja umożliwia obserwowanie stanu backendu, aktywnej sesji, parametrów obrazu i metryk transmisji. Udostępnia również możliwość zmiany modelu detekcji, a także wysłania wiadomości do klienta wykorzystując metodę służącą do przesyłania informacji. Zrzut ekranu 4.3 przedstawia wygląd interfejsu graficznego serwera.



Rysunek 4.3: Interfejs graficzny serwera

4.3.7 Obsługa błędów i cykl życia

Podczas uruchomienia serwera, przed udostępnieniem możliwości nawiązania połączenia, ładowany jest model detekcji i rejestrowana jest usługa sieciowa mDNS. Niepowodzenie wykonania jednego z tych procesów skutkuje wyświetleniem komunikatu o błędzie i zakończeniem uruchamiania backendu. W wariancie desktopowym, komunikat o błędzie jest wyświetlany na interfejsie użytkownika.

Problemy dotyczące pojedynczej klatki nie są przyczyną przerwania sesji. W przypadku zaistnienia nieobsługiwanych danych przesyłanych przez klienta, serwer ignoruje je i kontynuuje działanie. Przy błędzie modułu detekcji do interfejsu przekazywana jest klatka bez naniesionych ramek detekcji, komunikat o błędzie pojawia się w logach serwera, a serwer kontynuuje pracę i obsługę kolejnych klatek.

Obsługa żądania informacji przewiduje typowe sytuacje uniemożliwiające wykonanie analizy wycinka. Wystąpienie błędu nie zatrzymuje działania serwera, a skutkuje jedynie przygotowaniem komunikatu zastępczego. Wyjątki związane bezpośrednio z odczytem bazy wiedzy i dopasowaniem powierzchni są obsługiwane w module analizy. Nieprzewidziane wyjątki w trakcie zadania analizy są odnotowane w logach serwera, ale w tym wypadku do klienta nie jest wysyłany żaden komunikat o wystąpieniu błędu.

Rozłączeniu klienta towarzyszy zamknięcie podglądu w interfejsie serwera, wyzerowanie metryk sesji i usunięcie ostatniego obrazu z interfejsu. Anulowane zostają aktywne zadania analizy, a przypisany worker zostaje zatrzymany. Informacje o aktywnej sesji są zwalniane i możliwe jest nawiązanie połączenia z kolejnym klientem. Zatrzymanie serwera skutkuje zamknięciem mechanizmu WebSocket i wyrejestrowaniem usługi w sieci lokalnej. Następnie następuje zakończenie wszystkich wątków i zamknięcie procesu aplikacji. Wariant desktopowy w pierwszej kolejności kończy działanie wątku backendu, a następnie zamyka interfejs graficzny Qt.

4.4 Komunikacja klient–serwer

Komunikacja z serwerem, zgodnie z założeniami projektowymi, odbywa się w architekturze klient–serwer. Do jej realizacji wykorzystano protokół WebSocket, umożliwiający stałą komunikację dwukierunkową. Dzięki wykorzystaniu tego protokołu, możliwe jest przysyłanie kolejnych klatek oraz komunikatów sterujących pojedynczym kanałem komunikacyjnym, bez konieczności ponownego nawiązywania połączenia dla każdej wiadomości.

Do implementacji komunikacji aplikacja mobilna wykorzystuje bibliotekę `Ktor`, a część serwerowa bibliotekę `websockets`. Szczegóły implementacyjne przygotowania i przetwarzania obrazu przedstawiono w sekcjach 4.2 i 4.3. Niniejsza sekcja skupia się na opisie wymiany danych pomiędzy rozproszonymi modułami systemu.

4.4.1 Nawiązywanie połączenia i wykrywanie serwera

Aby możliwe było połączenie z serwerem konieczna jest znajomość jego adresu IP i portu. Można je wprowadzić ręcznie korzystając z pól interfejsu użytkownika, lub skorzystać z mechanizmu wykrywania usług w sieci lokalnej.

Aplikacja przeznaczona na system Android wykorzystuje mechanizm Network Service Discovery (NSD), oparty na protokole mDNS. Serwer publikuje usługę typu `_vi3dr._tcp.local.` w sieci lokalnej pod nazwą **VI3DR Server**. Po uruchomieniu wyszukiwania w aplikacji mobilnej i odnalezieniu usługi, pobrane zostają przypisane do niej adres IP i numer portu, które następnie przekazane do kontrolera transmisji automatycznie uzupełniają konfigurację połączenia.

Publikacji usługi po stronie serwera realizowana jest za pomocą biblioteki **zeroconf**. W rekordzie zawarte są również dodatkowe informacje dotyczące protokołu, wersji i ścieżki połączenia, aczkolwiek aktualna implementacja nie wykorzystuje ich w żaden sposób. W przyszłości mogą one zostać użyte do weryfikacji zgodności wersji i umożliwienia współpracy różnych wersji poszczególnych komponentów systemu.

Na podstawie odczytanych parametrów tworzony jest adres serwera w postaci `ws://<adres_ip>:<port>`. Po rozpoczęciu transmisji aplikacja nawiązuje połączenie z serwerem i tworzy sesję obsługiwaną w klasie **FrameSocketClient**. Poprawne ustanowienie sesji skutkuje zmianą stanu na *połączony* i od tego momentu możliwe jest wysyłanie kolejnych ramek obrazu oraz równoległa wymiana komunikatów sterujących w postaci wiadomości tekstowych.

Serwer domyślnie nasłuchuje na porcie 8765, na wszystkich dostępnych interfejsach sieciowych, obsługując jednocześnie pojedynczą sesję. W przypadku próby nawiązania połączenia przez kolejnego klienta, serwer odrzuca je wysyłając komunikat `client_stop` i zamykając połączenie (por. 3.6).

4.4.2 Protokół i format przesyłanych danych

W ramach połączenia, pomiędzy klientem a serwerem, wymieniane są wiadomości w formacie binarnym i tekstowym. Podstawowe wiadomości protokołu zostały przedstawione w tabeli 4.2.

Wiadomość binarna zawiera wyłącznie bajty obrazu zakodowanego w formacie JPEG. Zrezygnowano z dodatkowego nagłówka warstwy aplikacji i metadanych, mogących zawierać informacje o rozdzielczości obrazu, numerze klatki czy znaczniku czasu, ponieważ nie są one wymagane do prawidłowego działania systemu. Typ wiadomości protokołu WebSocket pozwala na odróżnienie danych obrazu od komunikatów sterujących, jednakże po stronie serwera sprawdzane są dwa początkowe bajty obrazu – charakterystyczne dla formatu JPEG. Zawartość wiadomości jest następnie przekazywana do dalszego przetwarzania.

Tabela 4.2: Podstawowe komunikaty protokołu klient-serwer

Kierunek	Typ	Format	Znaczenie
Klient → serwer	binarny	dane JPEG	Klatka obrazu zarejestrowana przez kamerę urządzenia mobilnego.
Klient → serwer	tekstowy	info-request	Żądanie przygotowania informacji dotyczącej aktualnie eksplorowanego obiektu.
Klient → serwer	tekstowy	stop	Żądanie zakończenia transmisji i zamknięcia aktywnej sesji.
Serwer → klient	tekstowy	info-response: <tekst>	Tekst przeznaczony do odczytania użytkownikowi, zawierający informację o obiekcie albo komunikat o problemie z przygotowaniem odpowiedzi.
Serwer → klient	tekstowy	client_stop	Polecenie zakończenia połączenia wysyłane przez część serwerową.

Komunikaty sterujące mają postać tekstową i są przesyłane w formacie UTF-8. Rozwiązanie to ogranicza złożoność protokołu i pozwala na łatwe rozróżnienie poszczególnych operacji.

4.4.3 Przesyłanie klatek obrazu

Po przygotowaniu obrazu przez aplikację mobilną, jest on przechowywany w formie tablicy bajtów JPEG. Przed wysłaniem kontroler sprawdza, czy transmisja jest aktywna, czy połączenie z serwerem pozostaje otwarte, oraz czy wysłanie nie spowoduje przekroczenia ustalonego limitu FPS. Jeżeli wszystkie warunki są spełnione, obraz jest przekazywany do serwera w formie wiadomości binarnej (por. 4.2.3).

Serwer, po odebraniu wiadomości binarnej, przekazuje ją do obiektu `CaptureSession`. W przypadku nagromadzenia wielu klatek preferowana jest najnowsza z nich. Szczegóły implementacyjne tego mechanizmu zostały opisane w sekcji 4.3.2.

Warstwa aplikacji nie implementuje ponownego przetwarzania pominiętych klatek – nie jest to wymagane, ponieważ nowsze klatki zastępują starsze. Protokół WebSocket korzysta z transportu TCP, zapewniającego uporządkowanie dostarczenia przesyłanych wiadomości.

4.4.4 Obsługa komunikatów sterujących

Poza wiadomościami binarnymi w systemie wymieniane są również komunikaty sterujące w formie tekstowej. Odpowiadają one za obsługę żądań użytkownika oraz cyklu życia transmisji.

Po rozpoznaniu komendy głosowej aplikacja przekazuje do serwera wiadomość `info-request`. Jej odebranie przez serwer skutkuje zakolejkowaniem zadania przygotowania tekstu zawierającego informacje zwrotne. Obsługa kończy się bez oczekiwania na rezultat, aby nie blokować potoku i by sesja mogła kontynuować swoje działanie.

Końcowym etapem analizy obiektu jest przekazanie przygotowanego przez moduł tekstu z powrotem do sesji, a następnie jego wysłanie w postaci `info-response:<tekst>`. Aplikacja kliencka rozpoznaje prefiks odpowiedzi i przekazuje zawartość do dalszej obsługi w kontrolerze. Ostatecznie tekst zostaje przekazany do mechanizmu syntezy mowy opisanego w sekcji 4.2.4.

4.4.5 Cykl życia połączenia i obsługa błędów

Po ustanowieniu sesji możliwe jest rozpoczęcie komunikacji aplikacji z serwerem, poprzez przekaz kolejnych ramek obrazu i wymianę komunikatów sterujących. Jeżeli użytkownik pragnie zakończyć transmisję, aplikacja wysyła do serwera wiadomość `stop` inicjując zamknięcie sesji. Połączenie może również zostać zakończone przez serwer poprzez wysłanie do aplikacji komunikatu `client_stop`. Stosuje się go w przypadku próby połączenia kolejnego klienta lub zamknięcia procesu serwera. W obu przypadkach aplikacja mobilna zmienia stan na *rozłączony* i wyświetla komunikat o zakończeniu sesji.

Błąd podczas łączenia z serwerem powoduje przejście aplikacji do stanu błędu i wyświetlenie komunikatu. Nie są wykonywane automatyczne próby ponownego połączenia, więc wznowienie transmisji wymaga ręcznej interwencji użytkownika.

Błędy pojedynczych wiadomości są izolowane i nie powodują przerwania sesji. Niepoprawne dane binarne są pomijane, a szczegółowe informacje dotyczące zachowania serwera w przypadku błędów zostały opisane w sekcji 4.3.7.

Komunikacja odbywa się przy użyciu nieszyfrowanego wariantu protokołu WebSocket i nie wykorzystuje mechanizmu uwierzytelniania klienta. Takie podejście zostało uznane za wystarczające w kontekście pracy w zaufanej sieci lokalnej. W przyszłości możliwe jest rozszerzenie protokołu o mechanizmy szyfrowania i uwierzytelniania, gdyby system miał zostać wykorzystany w sieciach publicznych.

4.5 Narzędzia pomocnicze

Oprócz aplikacji mobilnej i serwera, aby możliwe było przygotowanie zbioru danych, uczenie modeli i przeprowadzenie eksperymentów, przygotowano zestaw narzędzi pomocniczych. Nie uczestniczą one bezpośrednio w działaniu systemu, ale umożliwiają przygotowanie jego najważniejszego elementu – modelu detekcji obiektów. Zostały one przygotowane w języku Python i podzielone pomiędzy katalogi `video_to_dataset`,

`dataset_splitter` oraz `training`. Większość z nich udostępnia interfejs wiersza poleceń z możliwością podania parametrów działania bez konieczności modyfikacji kodu źródłowego.

4.5.1 Ekstrakcja klatek z nagrań

Do ekstrakcji klatek z nagrań wideo przygotowano skrypt znajdujący się w katalogu `video_to_dataset`. Jego zadaniem jest załadowanie pliku wideo i zapisanie kolejnych klatek w postaci osobnych plików graficznych. Do realizacji wykorzystano bibliotekę `OpenCV`.

Nagranie otwierane jest przez klasę `VideoCapture`, a następnie odczytywane są jego kolejne klatki. Użytkownik może określić częstotliwość zapisu klatek za pomocą parametru wejściowego. Dla wartości n zapisywana jest pierwsza klatka, a następnie co n -ta klatka. Jeżeli nie zostanie podana wartość parametru, zachowana zostaje każda klatka filmu.

Obrazy zapisywane są zgodnie ze schematem `frame_<n>.jpeg` w formacie JPEG. Jeżeli w katalogu docelowym znajdują się już jakieś pliki, narzędzie sprawdza ich nazwy i kontynuuje numerację od najwyższego numeru. Umożliwia to wielokrotne uruchamianie ekstrakcji z różnych nagrań w tym samym katalogu, bez ryzyka nadpisania istniejących plików.

Narzędzie nie modyfikuje w żaden sposób ekstrahowanych klatek, a zwraca obrazy bezpośrednio odczytane przez bibliotekę. Nie generuje również żadnych adnotacji, ponieważ ich przygotowanie odbywa się w osobnym procesie.

4.5.2 Narzędzie do podziału zbioru danych

Do organizacji zbioru danych przygotowano narzędzie `dataset_splitter`. Jego zadaniem jest podział zbioru danych na dwie części: treningową i walidacyjną. Jako dane wejściowe przyjmowany jest katalog z obrazami i odpowiadającymi im adnotacjami w formacie YOLO. Dla każdego obrazu narzędzie wyszukuje odpowiadający mu plik z adnotacją, a jeżeli nie zostanie on odnaleziony obraz traktowany jest jako tło i tworzony jest dla niego pusty plik adnotacji.

Podział realizowany jest za pomocą funkcji `train_test_split` z biblioteki `scikit-learn`. Dane są losowane z zachowaniem stratyfikacji względem klasy, a wartość ziarna generatora liczb pseudolosowych może zostać określona przez użytkownika. Wynik działania narzędzia to drzewo katalogów odpowiadające strukturze w formacie YOLO:

- `images/train`,
- `images/val`,
- `labels/train`,
- `labels/val`.

Domyślny sposób działania narzędzia przewiduje kopiowanie plików do katalogów docelowych, ale możliwe jest również jego uruchomienie w trybie przenoszenia plików.

4.5.3 Narzędzia do treningu i testowania modeli

Najbardziej rozbudowanym narzędziem pomocniczym jest zestaw skryptów do treningu i testowania modeli detekcji, znajdujący się w katalogu `training`. Jego głównym punktem wejścia jest skrypt `train.py`, który uruchamia proces uczenia modelu wykorzystując bibliotekę `Ultralytics`.

Skrypt umożliwia wskazanie pliku konfiguracji zbioru danych oraz modelu używanego jako punkt początkowy. Możliwe jest użycie istniejących wag modelu, zapisanych w pliku `.pt`, lub rozpoczęcie treningu od zera, na podstawie architektury opisanej w pliku `.yaml`. Istnieje również możliwość kontynuowania treningu na podstawie wcześniej zapisanego punktu kontrolnego. Parametry uczenia mogą zostać zmienione przy uruchomieniu skryptu korzystając z argumentów wiersza poleceń. Zmianie podlegają między innymi: rozmiar obrazu wejściowego, rozmiar batcha, liczba epok, liczba procesów roboczych, wartość `patience`, urządzenie obliczeniowe oraz ścieżka katalogu wynikowego. Brak podania parametrów skutkuje uruchomieniem treningu z wartościami domyślnymi. Urządzenie wykorzystywane do obliczeń jest wybierane na podstawie dostępności technologii: CUDA, MPS, a następnie CPU.

Przed rozpoczęciem treningu sprawdzana jest poprawność konfiguracji zbioru danych w której wymagane są informacje o klasach oraz ścieżkach do katalogów z obrazami i adnotacjami dla zbiorów treningowego i walidacyjnego. Na potrzeby biblioteki tworzona jest następnie znormalizowana wersja konfiguracji, zapisywana w katalogu wynikowym.

Nieprawidłowości przy konfiguracji skutkują przerwaniem skryptu uniemożliwiając rozpoczęcie treningu. Możliwe jest sprawdzenie poprawności konfiguracji bez uruchamiania procesu uczenia korzystając z flagi `--dry-run`.

Uczenie modelu wykonywane jest przez metodę `model.train` z biblioteki `Ultralytics`. Odpowiada ona za kolejne epoki treningowe, walidację, tworzenie punktów kontrolnych i zapisywanie wyników. Główne pliki tworzone podczas uczenia to `results.csv`, wagi `best.pt` i `last.pt` oraz wykresy przebiegu uczenia.

Ocena modelu na wydzielonym zbiorze testowym jest realizowana przez skrypt `test.py`. Wykorzystuje on metodę walidacji z biblioteki `Ultralytics` z ustawieniem `split=test`. Wynikiem działania skryptu jest katalog `test` zawierający artefakty generowane przez bibliotekę, wśród których znajdują się między innymi macierze pomyłek i krzywe zależności.

Narzędzie udostępnia również skrypt `predict.py`, który umożliwia wykonanie predykcji na pojedynczym obrazie. Służy on przede wszystkim do wizualnej kontroli wyników detekcji i nie wykonuje pełnej oceny modelu.

4.5.4 Filtry wejściowe i zapis wyników

Mechanizm treningu umożliwia wprowadzenie dodatkowego przetwarzania obrazów przed ich przekazaniem do modelu. Moduł `filtered_dataset` tworzy przetworzoną wersję zbioru danych, w której obrazy poddawane zostały filtrowaniu. Projekt zawiera dwa filtry: `GrayscaleInputFilter` i `SobelInputFilter`, jednakże możliwe jest przygotowanie własnych filtrów. Działają one jako niezależne moduły, przez co możliwe jest ich wskazanie przez narzędzie treningowe bez wprowadzania zmian w jego kodzie źródłowym. Istnieje również narzędzie umożliwiające podgląd działania filtrów na wybranych obrazach zbioru danych, bez ich zapisywania.

Uczenie odbywa się na przetworzonych obrazach, a po zakończeniu treningu zapisywany jest dodatkowy plik wag zawierający warstwę filtrującą na wejściu modelu. Dzięki temu nie jest konieczne przetwarzanie obrazów w aplikacji mobilnej ani w serwerze, a model może być używany wprost na obrazach kolorowych.

Oprócz plików domyślnie zapisywanych przez bibliotekę `Ultralytics` projekt generuje również dodatkowe artefakty, podsumowujące przebieg uczenia. Odpowiedni moduł odczytuje z wyników wartości metryk, na których podstawie oblicza dodatkowo wartość *F1-score*. Ponadto skanuje plik z wynikami poszczególnych epok i zapisuje wyniki najlepszej epoki w formie pliku `run_stats.json`.

Aktualna implementacja nie umożliwia wykonania automatycznego zestawienia wyników wielu niezależnych eksperymentów. Każde uruchomienie jest zapisywane w osobnym katalogu, którego zawartość może następnie zostać wykorzystana do ręcznej analizy. W przyszłości możliwe jest przygotowanie dodatkowego narzędzia, dedykowanego do tego celu.

Rozdział 5

Zbiór danych i metodyka badań

Specyfika problemu rozpoznawania trzymanyh obiektów trójwymiarowych wymaga przygotowania własnego zbioru danych. Niniejszy rozdział opisuje proces pozyskiwania danych, ich oznaczania, przyjęty sposób podziału na zbiory oraz sposób, w jaki przeprowadzono eksperymenty.

5.1 Zbiór danych

Stworzenie odpowiedniego zbioru danych jest kluczowe dla przeprowadzenia eksperymentów i oceny jakości detekcji. Kluczowe jest, aby odpowiadał on rzeczywistym zastosowaniom systemu, a także aby był wystarczająco zróżnicowany i reprezentatywny dla problemu.

5.1.1 Charakterystyka zbioru danych

Przygotowany zbiór danych składa się z 4666 obrazów JPEG o rozdzielczości 1920×1440 podzielonych na zbiory treningowy, walidacyjny i testowy. Przedstawiają one osobę siedzącą przed kamerą, manipulującą w dłoniach bryłami należącymi do jednej z 6 klas obiektów:

- sześćcian,
- prostopadłościan,
- kula,
- walec,
- stożek,
- czworościan.

Tabela 5.1: Podział zbioru danych na zbiory treningowy, walidacyjny i testowy.

Klasa	Zbiór treningowy	Zbiór walidacyjny	Zbiór testowy
Sześcian	493	54	90
Kula	334	30	51
Walec	377	54	68
Prostopadłościan	563	47	86
Czworościan	923	85	18
Stożek	424	48	52
Tło	668	89	112
Razem	3782	407	477

Obrazy w zbiorze zostały pozyskane z nagrań wideo, z zachowaniem występowania wszystkich klas w każdym nagraniu.

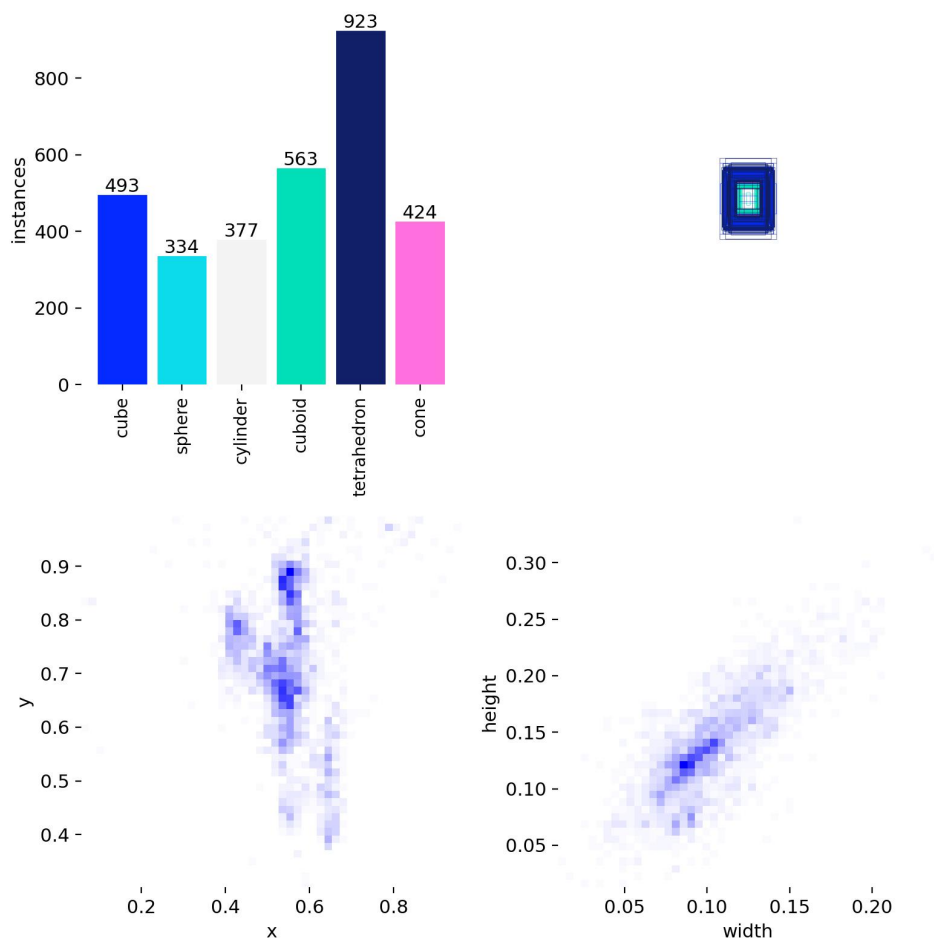
Liczbę obrazów w poszczególnych zbiorach z podziałem na klasy i tła przedstawiono w tabeli 5.1. Podział danych został wykonany w taki sposób, aby dane z jednego nagrania nie znalazły się w kilku zbiorach. Ogranicza to ryzyko przecieku danych oraz możliwość zawyżenia wyników poprzez rozpoznanie obiektu bazując jedynie na wyglądzie sceny, a nie na faktycznym kształcie bryły.

Pełen zbiór danych wraz z adnotacjami zawiera 4666 obrazów i 4666 odpowiadających im plików adnotacji w formacie YOLO, a także 4 pliki tekstowe. Łączny rozmiar zbioru danych to 3,19 GB.

Dodatkowa charakterystyka zbioru uczącego została przedstawiona na rysunku 5.1. Oprócz liczności poszczególnych klas przedstawiono na nim również rozkład położenia ramek ograniczających oraz ich znormalizowane wymiary. Większość z nich znajduje się w centralnej części obrazu, co jest skutkiem przyjętego sposobu rejestrowania materiałów wideo.

5.1.2 Proces pozyskiwania i oznaczania danych

Pierwszym etapem przygotowania zbioru danych było nagranie materiałów wideo zawierających wspomniane wcześniej widoki osoby manipulującej bryłami. Zarejestrowano je przy pomocy nieruchomego urządzenia mobilnego Samsung Galaxy S25 w rozdzielczości 1920×1440 i 30 klatkach na sekundę. Zostały one zrealizowane w różnych warunkach oświetleniowych z wykorzystaniem światła dziennego i sztucznego, z różnymi osobami i w różnych ubraniach. Na każdym nagraniu osoba manipuluje każdą z przygotowanych brył w sposób odpowiadający przewidywanemu scenariuszowi użycia systemu. Ostatecznie nagrano 9 filmów o łącznej długości 14 minut i 16 sekund. Następnie, korzystając



Rysunek 5.1: Rozkład klas oraz położenia ramek ograniczających w zbiorze treningowym.

z przygotowanego skryptu (por. 4.5.1), z wszystkich nagrań wyekstrahowano 4666 obrazów w formacie JPEG.

W następnym kroku obrazy należało oznaczyć, czyli przygotować adnotacje obiektów, które się na nich znajdują. Do tego zadania wykorzystano narzędzie CVAT [6] rozwijane przez firmę cvat.ai corporation. Zostało ono wybrane ze względu na możliwość pracy w przeglądarce internetowej, a także półautomatyczne oznaczanie obiektów korzystając z wcześniej wytrenowanego modelu detekcji. Niestety, w trakcie oznaczania napotkano problemy z uruchomieniem modelu w narzędziu, co skutkowało koniecznością ręcznego oznaczania wszystkich obiektów i rezygnacji z rozszerzenia zbioru o dodatkowe obrazy. Podczas pracy z narzędziem korzystano również z serwisu Amazon S3¹ do przechowywania danych w chmurze. Dzięki niemu nie zaistniała konieczność przesyłania dużych ilości danych pomiędzy wykorzystywanymi platformami.

¹Amazon Simple Storage Service

5.1.3 Podział danych na zbiory treningowy, walidacyjny i testowy

Po oznaczeniu danych należało je podzielić na zbiory treningowy, walidacyjny i testowy. Wykonano ręczny podział danych tak, aby obrazy z pojedynczego nagrania nie były rozdzielone pomiędzy różne zbiory. Zadbano równocześnie, aby zbiór treningowy zawierał zróżnicowane obrazy, a zbiory walidacyjny i testowy zawierały wszystkie klasy oraz możliwie zróżnicowane przykłady scen. Nagrania, z których obrazy znalazły się w zbiorach treningowym i walidacyjnym zostały zarejestrowane pojedynczego dnia, a nagrania dla zbioru testowego zostały zarejestrowane kilka dni później. Ostateczny udział obrazów w pełnym zbiorze danych wynosił 81% dla zbioru treningowego, 8,7% dla zbioru walidacyjnego i 10,2% dla zbioru testowego. Zrezygnowano z wykorzystania skryptu `split_dataset.py` do podziału danych, aby nie mieszać obrazów, a tym samym wykluczyć możliwość rozpoznania obiektów na podstawie wyglądu sceny. Szczegółowy udział filmów w poszczególnych zbiorach przedstawiono w tabeli 5.2.

Tabela 5.2: Podział nagrań wideo pomiędzy zbiory treningowy, walidacyjny i testowy.

Data nagrania	Długość nagrania mm:ss	Liczba obrazów	Zbiór	Udział
13-05-2026 12:39	04:07	1481	treningowy	całościowy: 31,7% w zbiorze: 39,2%
13-05-2026 12:45	02:21	843	treningowy	całościowy: 18,1% w zbiorze: 22,3%
13-05-2026 12:50	01:37	581	treningowy	całościowy: 12,5% w zbiorze: 15,4%
13-05-2026 12:53	01:22	493	treningowy	całościowy: 10,6% w zbiorze: 13,0%
13-05-2026 12:58	01:04	384	treningowy	całościowy: 8,2% w zbiorze: 10,2%
13-05-2026 13:02	01:08	407	walidacyjny	całościowy: 8,7% w zbiorze: 100%
19-05-2026 14:03	0:54	164	testowy	całościowy: 3,5% w zbiorze: 34,4%
19-05-2026 14:06	0:54	164	testowy	całościowy: 3,5% w zbiorze: 34,4%
19-05-2026 14:10	0:49	149	testowy	całościowy: 3,2% w zbiorze: 31,2%

Przeznaczenie poszczególnych zbiorów danych jest zgodne z przyjętymi w literaturze standardami oraz nie występuje nakładanie się ich zastosowań. Zbiór treningowy służy wyłącznie do trenowania modeli, zbiór walidacyjny do oceny jakości modeli w trakcie

treningu, a zbiór testowy jest używany wyłącznie do oceny jakości modeli po zakończeniu treningu.

5.1.4 Warianty danych wejściowych

Podstawowym wariantem danych wejściowych jest obraz kolorowy w przestrzeni barw RGB. Stanowi on punkt odniesienia dla wszystkich eksperymentów.

Aby sprawdzić eksperymentalnie wpływ koloru na jakość detekcji, postanowiono przetestować zdolność modeli na danych wejściowych w dwóch wariantach: kolorowym (RGB) oraz w odcieniach szarości (grayscale). Konwersja na odcienie szarości została wykonana obliczając wartość ważonej luminancji z wykorzystaniem wzoru:

$$Y = 0,299 \cdot R + 0,587 \cdot G + 0,114 \cdot B. \quad (5.1)$$

Taki dobór wag najlepiej odzwierciedla wpływ poszczególnych kanałów na postrzeganą jasność obrazu przez ludzkie oko. Następnie wartość luminancji została powielona na wszystkie trzy kanały, aby zachować zgodność z oczekiwanym przez modele wejściem.

Zastosowanie w tym miejscu średniej wartości kolorów:

$$Y = \frac{R + G + B}{3} \quad (5.2)$$

nie zostało uznane za właściwe, ponieważ eksperyment ma na celu sprawdzenie, czy usunięcie informacji o barwie będzie miało wpływ na jakość detekcji, a średnia wartość kolorów nie odzwierciedla rzeczywistego wpływu poszczególnych kolorów na postrzeganą jasność obrazu.

Repozytorium z projektem zawiera również filtr Sobela, jednakże nie został on wykorzystany w eksperymentach ponieważ oczekujemy, że model będzie w stanie samodzielnie wyznaczyć krawędzie obiektów. Użycie klasycznego, deterministycznego operatora detekcji krawędzi skutkuje zatraceniem informacji o teksturze i rozkładzie jasności w obrębie obiektów. Ich utrata może skutkować obniżeniem jakości detekcji, szczególnie w przypadku obiektów, które nie mają wyraźnych krawędzi, np. kuli.

Porównanie obrazów w wariacie kolorowym z obrazami na których zastosowano opisane filtry przedstawiono na rysunku 5.2.

5.2 Metodyka eksperymentalna

Poprawne przeprowadzenie badań wymaga ustalenia kryteriów oceny poszczególnych modeli, a także przyjęcia procedury eksperymentalnej, która umożliwi porównanie jakości detekcji pomiędzy nimi. W tym celu należy ustalić metryki oceny, a także parametry treningu modeli.



(a) Wariant kolorowy.

(b) Wariant w odcieniach szarości.

Rysunek 5.2: Warianty danych wejściowych.

5.2.1 Konfiguracja treningu modeli

Skrypt treningowy `train.py` umożliwia konfigurację wielu parametrów treningu modelu. W eksperymentach nie wykorzystano wszystkich dostępnych opcji, a jedynie te, które uwydatnią różnice w jakości detekcji pomiędzy poszczególnymi wariantami modeli. Podczas treningu zmieniano trzy elementy konfiguracji:

- **model** – wariant modelu: YOLO26n, YOLO26s,
- **zbiór danych** – warianty zbioru danych: RGB, odcienie szarości,
- **rozmiar obrazu** – rozdzielczość obrazu wejściowego: 640, 1024.

Łącznie wytrenowano 7 modeli, które pozwoliły na przeprowadzenie eksperymentów i porównanie jakości detekcji pomiędzy nimi.

Urządzenia wykorzystane do treningu były wybierane według dostępności w danym momencie, a nie według ich wydajności. Trening na karcie graficznej NVIDIA RTX 4060 był wykonywany na lokalnym komputerze z systemem Windows 11, a treningi na pozostałych kartach odbywały się w chmurze za pomocą platformy RunPod z systemem Ubuntu 22.04. Urządzenie obliczeniowe i system operacyjny nie są zmiennymi badanymi, a ich wpływ na jakość uzyskanych modeli nie jest przedmiotem badań.

Augmentacja danych została włączona w celu zwiększenia różnorodności danych treningowych i poprawy zdolności generalizacji modeli. Wykorzystano w tym celu wbudowane w narzędzie Ultralytics YOLO mechanizmy augmentacji pozostawiając ich parametry domyślne. Obejmują one modyfikacje składowych HSV, translację, skalowanie, odbicia poziome oraz augmentację mozaikową.

Trening modeli był prowadzony do momentu braku poprawy kryterium walidacyjnego przez 50 kolejnych epok (ang. *early stop*). Po tym czasie trening był przerywany i uznawano, że model osiągnął swoje plateau i dalszy trening nie przyniesie znaczącej

poprawy jakości detekcji. Przyjęta wartość parametru `patience` jest kompromisem pomiędzy przedwczesnym zakończeniem treningu a niepotrzebnym wydłużeniem jego czasu [34].

Każdy trening uruchomiony został z ustawieniem ziarna generatora liczb pseudolosowych na wartość 42. Zwiększa to powtarzalność wyników oraz ułatwia odtworzenie eksperymentów w przyszłości. Wartość ta została wybrana ze względu na jej popularność w literaturze i kulturze komputerowej².

Do treningu wykorzystano następujące wersje bibliotek i narzędzi:

- **Python** – 3.11.11,
- **Ultralytics** – 8.4.63,
- **PyTorch** – 2.12.0,
- **matplotlib** – 3.10.9,
- **opencv-python** – 4.13.0.92,
- **CUDA** – 13.2.

5.2.2 Metryki oceny

Do oceny poszczególnych modeli wykorzystano metryki zwracane przez bibliotekę Ultralytics: precision, recall, $mAP50$, $mAP50-95$ oraz macierze pomyłek. Dodatkowo obliczano wartość $F1$ -score dla najlepszego modelu w każdym eksperymencie. Szczegółowe opisy każdej z użytych metryk znajdują się w sekcji 2.5.

Dokładne przebiegi każdego treningu znajdują się w pliku `results.csv`, a porównanie najlepszej epoki treningu z ostatnią przedstawia plik `run_stats.json`.

Najważniejszą z używanych metryk jest $mAP50-95$, ponieważ uwzględnia ona jakość detekcji dla wielu progów IoU i pozwala na ocenę zarówno jakości lokalizacji, jak i klasyfikacji obiektów [51]. Jako metryka pomocnicza użyto siostrzaną metrykę $mAP50$, która jest łatwiejsza do interpretacji i ukazuje skuteczność detekcji przy mniej rygorystycznych kryteriach. Kolejno, metryki precision i recall umożliwiają ocenę jakości detekcji w kontekście liczby wykrytych obiektów, a ich średnia harmoniczna $F1$ -score opisuje kompromis pomiędzy nimi. Niezwykle przydatne okazują się również macierze pomyłek, na których podstawie można określić, które klasy są najtrudniejsze do wykrycia i które są najczęściej mylone z innymi.

Do porównania należy wykorzystać metryki, które zostały wyznaczone na zbiorze testowym, a nie na zbiorze walidacyjnym. Zbiór walidacyjny służy wyłącznie do oceny jakości predykcji podczas treningu do wyznaczenia najlepszej epoki treningu, podczas gdy

²Wartość 42 jest nawiązaniem do powieści Douglasa Adamsa *Autostopem przez Galaktykę*, gdzie stanowi odpowiedź na "Wielkie Pytanie o Życie, Wszechświat i całą resztę"

zbiór testowy jest wykorzystywany do końcowej oceny modelu. Ważne jest również, aby przy każdym eksperymencie używać dokładnie tego samego zbioru testowego, aby zachować miarodajne porównanie poszczególnych modeli. Wyniki tej oceny stanowią podstawę do porównania modeli przedstawionego w rozdziale 6.

Oprócz metryk jakości detekcji przy końcowym porównaniu modeli uwzględniono liczbę ich parametrów, koszt obliczeniowy wyrażony w GFLOPs, rozmiar pliku modelu oraz czas inferencji i wynikającą z niego liczbę przetwarzanych klatek na sekundę. Wartości te zebrano w dwóch niezależnych seriach pomiarowych na pojedynczym urządzeniu.

Nie porównywano pełnego czasu odpowiedzi ponieważ jest on silnie zależny od używanego urządzenia obliczeniowego, przepustowości sieci oraz obciążenia systemu operacyjnego. Każdy przypadek wdrożenia systemu należy oceniać indywidualnie, na podstawie wyników eksperymentów przeprowadzonych na docelowym sprzęcie i w docelowych warunkach pracy.

5.2.3 Procedura eksperymentalna

Aby móc dokonać porównania jakości detekcji pomiędzy poszczególnymi wariantami modeli należy podzielić je na eksperymenty. Każdy z nich polegał na wytrenowaniu kilku, porównywalnych modeli, które różniły się pojedynczym parametrem. Wykonano następujące eksperymenty:

Eksperyment A: Porównanie jakości detekcji pomiędzy rozmiarami modelu YOLO26 **n** i **s** dla danych wejściowych w wariacie kolorowym. Pozwoli to ocenić wpływ skali modelu i liczby jego parametrów na jakość detekcji.

Eksperyment B: Porównanie jakości detekcji pomiędzy wariantami z różnymi rozdzielczościami obrazu wejściowego: 640 oraz 1024. Pozwoli on ocenić wpływ rozdzielczości obrazu na jakość detekcji.

Eksperyment C: Porównanie jakości detekcji pomiędzy wariantami danych: kolorowym i w odcieniach szarości. Pozwoli to ocenić, czy usunięcie informacji o barwie wpływa na jakość detekcji i czy model jest w stanie skutecznie rozpoznać obiekty bez wykorzystania informacji o kolorze.

Przedstawione eksperymenty pozwalają ocenić, czy możliwe jest skuteczne rozpoznawanie trzymanych obiektów trójwymiarowych bez informacji o kolorze, a także czy możliwe jest uzyskanie zadowalających wyników przy użyciu modeli o mniejszej liczbie parametrów. Wyniki poszczególnych eksperymentów zostały przedstawione w rozdziale 6.

Rozdział 6

Wyniki badań i ich analiza

Wyniki, jakie otrzymano z przeprowadzonych eksperymentów pozwalają określić, czy badany problem rozpoznawania brył może zostać rozwiązany z użyciem metod uczenia głębokiego. Niniejszy rozdział prezentuje uzyskane wyniki oraz ich analizę. Przeprowadzono w nim również analizę najczęstszych błędów modeli i wysnuto wnioski dlaczego zachodzą.

Do przeprowadzenia eksperymentów wytrenowano siedem sieci o architekturze YOLO, wykorzystując ten sam zbiór danych, korzystając z procesorów graficznych firmy NVIDIA. W części eksperymentów obrazy przed treningiem przekształcono do odcieni szarości zgodnie z procedurą opisaną w 4.5.4. Macierz wariantów modeli została przedstawiona w tabeli 6.1. Taka liczba modeli pozwala na przeprowadzenie eksperymentów założonych w sekcji 5.2.3.

Tabela 6.1: Macierz modeli wytrenowanych w ramach badań.

Model	RGB 640	RGB 1024	grayscale 640	grayscale 1024
YOLO26n	✓	✓	✓	✓
YOLO26s	✓	✓	✓	

6.1 Zestawienie wyników treningów

Każdy z wytrenowanych modeli został oceniony pod kątem jakości detekcji na zbiorze testowym. Do oceny jakości detekcji za każdym razem wykorzystano model, który w trakcie uczenia osiągnął najlepsze wyniki na zbiorze walidacyjnym. Kryterium wyboru najlepszego modelu była wartość mAP_{50-95} . Po przeprowadzeniu treningu każdy z modeli został oceniony na zbiorze testowym, a wyniki zostały zestawione w tabeli 6.2.

Prawie każdy z treningów zakończył się jego wcześniejszym zatrzymaniem, co oznacza, że w trakcie uczenia model osiągnął wartość mAP_{50-95} na zbiorze walidacyjnym,

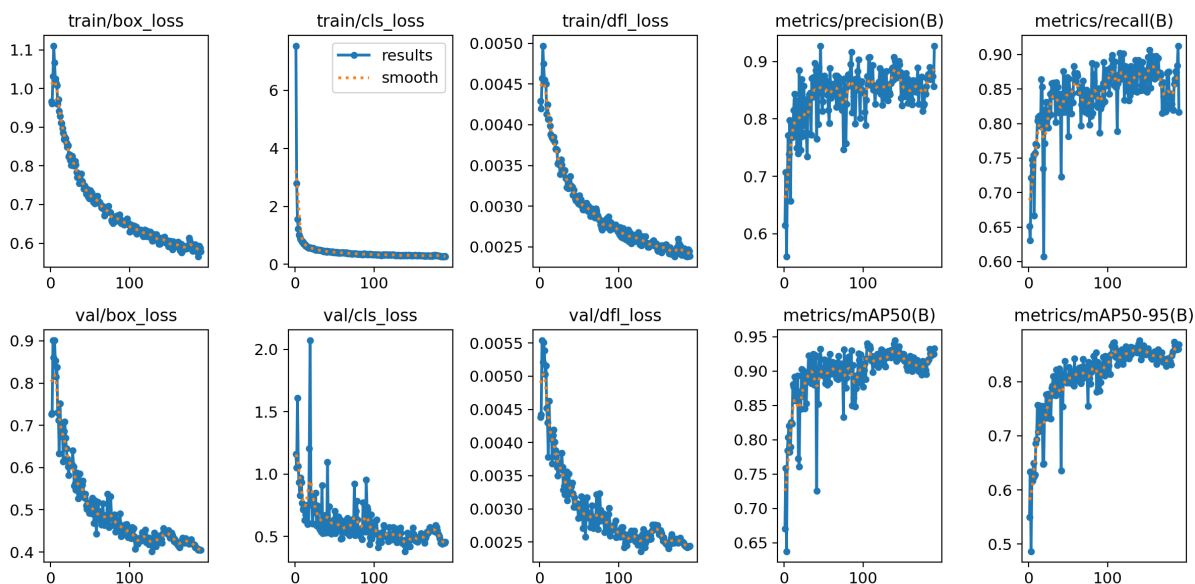
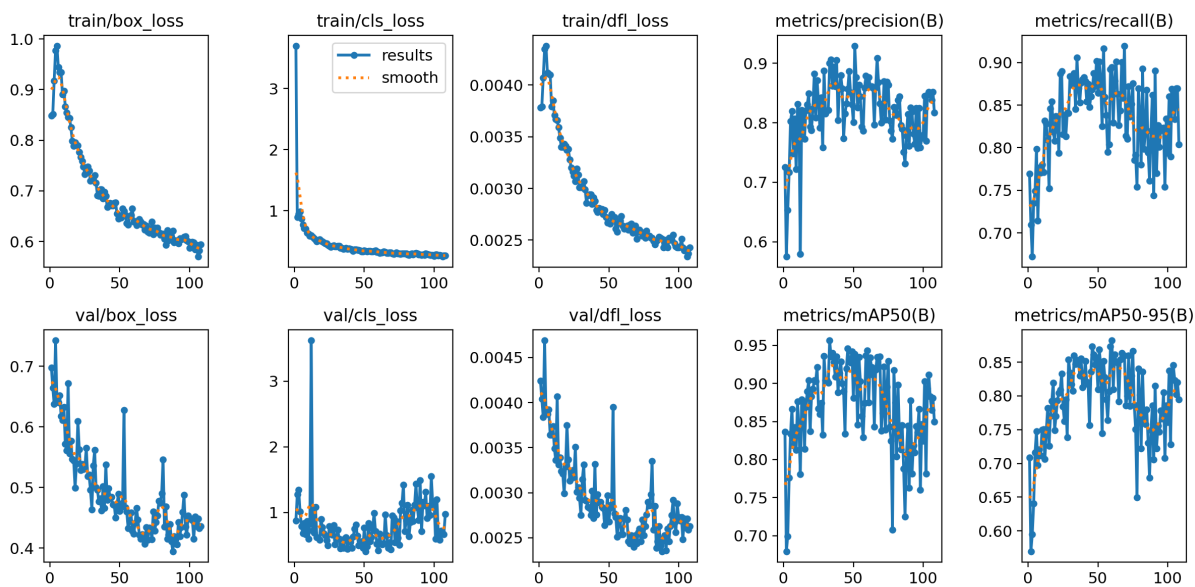
Tabela 6.2: Zestawienie wyników detekcji modeli na zbiorze testowym.

ID	E_1	E_2	E_3	E_4	E_5	E_6	E_7
Model	YOLO26n	YOLO26s	YOLO26n	YOLO26s	YOLO26n	YOLO26s	YOLO26n
Dane	RGB	RGB	RGB	RGB	grayscale	grayscale	grayscale
Rozmiar obrazu	640	640	1024	1024	640	640	1024
Ostatnia epoka	190	108	211	114	203	280	230
Najlepsza epoka	140	60	161	64	153	230	180
Precision	0.714	0.701	0.794	0.846	0.591	0.630	0.649
Recall	0.678	0.760	0.825	0.736	0.475	0.601	0.601
<i>mAP</i> 50	0.716	0.786	0.862	0.844	0.488	0.560	0.621
<i>mAP</i> 50-95	0.599	0.641	0.719	0.729	0.360	0.449	0.495
F1-score	0.696	0.730	0.809	0.787	0.526	0.615	0.624
GPU	RTX 4060	RTX 4060	L40S	L40S	A100	A100	A100

która nie uległa poprawie przez 50 kolejnych epok. Uczenie modelu E_2 zakończono po 48 epokach bez poprawy wartości ze względu na ograniczenia czasowe dostępu do jednostki obliczeniowej i nie podjęto decyzji o kontynuacji treningu. Trening wariantu **s** na obrazach kolorowych w obu przypadkach (E_2 , E_4) potrzebował mniejszej liczby epok do osiągnięcia najlepszej wartości kryterium walidacyjnego niż wariant **n** (E_1 , E_3).

Większość treningów modeli miała stabilny przebieg, charakteryzujący się szybkim wzrostem wartości metryk mAP, a następnie ich wypłaszczenie. Jest to typowe zachowanie w procesie uczenia modeli YOLO, które w początkowej fazie szybko dopasowują się do danych treningowych, a następnie optymalizują wagi w celu poprawy jakości detekcji. Przebieg treningu modelu E_1 wraz ze zmianami wartości jego metryk został przedstawiony na rysunku 6.1a. Możemy na nim zaobserwować, że wartości funkcji straty dla zbioru uczącego systematycznie maleją, co świadczy o tym, że model uczy się coraz lepiej dopasowywać do danych treningowych. Podobne tendencje, choć nie tak wyraźne, obserwujemy w przypadku wartości funkcji straty dla zbioru walidacyjnego. Jednocześnie wartości metryk jakości detekcji szybko rosną świadcząc o tym, że model coraz lepiej dopasowuje etykiety do obiektów na obrazach.

Ciekawym przypadkiem jest trening modelu E_2 , który osiągnął najlepszą wartość metryki w 60. epoce. Analiza przebiegu uczenia, ukazanego na rysunku 6.1b, pokazuje, że w trakcie treningu wystąpiło nagłe załamanie wartości metryk jakości detekcji. Pomimo systematycznego spadku wartości funkcji straty, od około połowy treningu funkcja straty na zbiorze walidacyjnym zaczęła wzrastać. W końcowej fazie treningu widzimy jednak powrót do klasycznego przebiegu uczenia, co może skłaniać do przeprowadzenia dalszych badań nad tym przypadkiem. Obserwowany przebieg uzasadniałby ponowienie treningu modelu E_2 w celu sprawdzenia, czy wzrost metryk walidacyjnych utrzymałby się w kolejnych epokach.

(a) Przebieg treningu wariantu E_1 .(b) Przebieg treningu wariantu E_2 .

Rysunek 6.1: Przebiegi treningu modeli YOLO26 na obrazach kolorowych o rozdzielczości 640 pikseli.

6.2 Wpływ skali modelu na jakość detekcji

Aby ocenić wpływ skali modelu na jakość detekcji, w eksperymencie A porównano modele YOLO26n i YOLO26s. W tym celu wykorzystano modele wytrenowane na obrazach kolorowych o rozdzielczościach 640 (E_1 , E_2) oraz 1024 pikseli (E_3 , E_4). Wyniki zestawiono parami, tak aby w każdym porównaniu jedyną zmienną była skala modelu. Wartości metryk dla poszczególnych porównań przedstawiono w tabeli 6.3.

Tabela 6.3: Porównanie jakości wariantów modeli YOLO26 n i s w eksperymencie A.

Metryka	RGB 640			RGB 1024		
	YOLO26n	YOLO26s	Różnica (s-n)	YOLO26n	YOLO26s	Różnica (s-n)
$mAP50$	0.71595	0.78551	+0.06957	0.86231	0.84409	-0.01823
$mAP50-95$	0.59866	0.64119	+0.04253	0.71856	0.72932	+0.01077
$F1-score$	0.69568	0.73041	+0.03473	0.80908	0.78712	-0.02195

Dla obrazów o rozdzielczości 640 pikseli wyższą wartość analizowanych metryk osiągnął wariant s. Wartość $mAP50$ uległa poprawie o 6, 96 p.p. (punktu procentowego), a dla bardziej rygorystycznej $mAP50-95$ różnica wyniosła około 4, 25 p.p. Wartość wskaźnika $F1-score$ również wzrosła, co wskazuje, że w takiej konfiguracji większy model wykazuje się lepszą jakością detekcji na zbiorze testowym.

Inną zależność obserwujemy przy większych obrazach wejściowych o rozdzielczości 1024 pikseli. Model YOLO26s uzyskał nieznacznie wyższą wartość $mAP50-95$, o 1, 08 p.p. Jednocześnie, mniejszy wariant osiągnął wyższe wartości $mAP50$ i wskaźnika $F1-score$. Oznacza to, że przy większym obrazie wejściowym wzrost skali modelu nie przekłada się bezpośrednio na poprawę jakości detekcji.

Porównanie obu rozdzielczości wskazuje, że wpływ zwiększenia skali modelu zależy od rozmiaru obrazu wejściowego. Dla mniejszych obrazów lepszą jakość detekcji wykazuje model YOLO26s. Przy obrazach o rozdzielczości 1024 pikseli korzyść użycia większego modelu widoczna jest jedynie w wartości metryki $mAP50-95$. Można więc zauważyć, że zwiększenie skali modelu ma większe znaczenie przy korzystaniu z mniejszych obrazów wejściowych, natomiast przy większej ilości danych na wejściu przewaga większego wariantu ulega znacznemu ograniczeniu.

Wyniki eksperymentu A nie wskazują więc na stałą przewagę jednego wariantu, a raczej na zależność jakości detekcji od ilości informacji dostarczanych modelowi. YOLO26s przynosi znaczną poprawę dla mniejszych obrazów, lecz dla wariantu 1024 oba modele zachowują podobne właściwości detekcji. Ocena zasadności użycia większego wariantu modelu wymaga więc uwzględnienia również innych czynników, takich jak koszt obliczeniowy i czas inferencji, które zostaną zestawione w kolejnych sekcjach rozdziału.

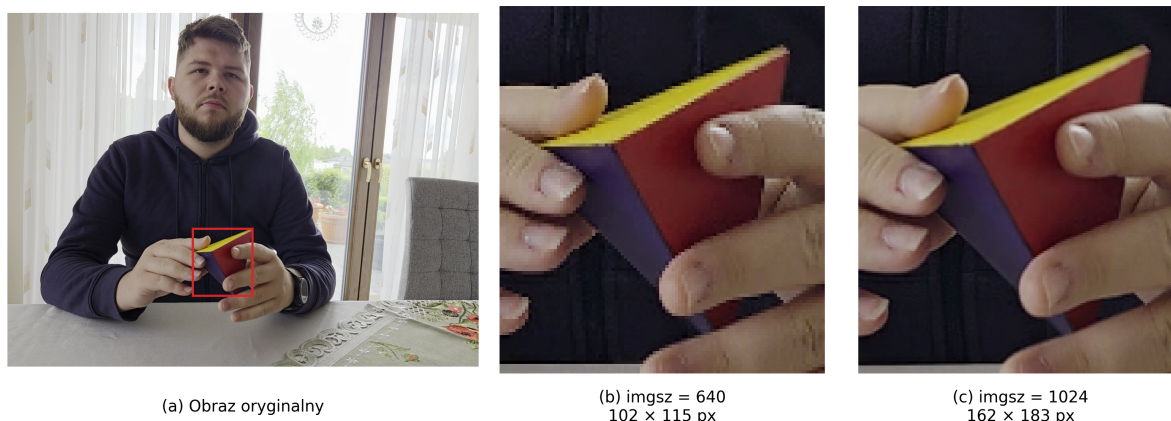
6.3 Wpływ rozdzielczości obrazu wejściowego na jakość detekcji

W kolejnym eksperymencie badano wpływ rozmiaru wejściowego obrazu na jakość detekcji, porównując ze sobą modele trenowane na obrazach kolorowych z parametrem `imgsz` ustawionym na 640 i 1024 piksele. Do porównania wykorzystano pary modeli o tej samej skali, tak aby możliwe było wyizolowanie wpływu rozdzielczości obrazu wejściowego na jakość detekcji. Wyniki porównania zestawiono w tabeli 6.4.

Przykład wpływu rozdzielczości obrazu wejściowego przedstawiono na rysunku 6.2. W prezentowanym przypadku w wariancie 640 bryła jest reprezentowana przez 11730 pikseli, a w wariancie 1024 przez 29646 pikseli. Stanowi to wzrost o 2,53 razy i może wpływać na zdolność modelu do wykrywania obiektów, szczególnie w przypadku mniejszych brył. Warto jednak zauważyć, że zwiększenie rozdzielczości obrazu wejściowego niesie za sobą również zwiększony koszt obliczeniowy oraz wydłużenie czasu inferencji.

Tabela 6.4: Porównanie jakości modeli dla rozmiarów obrazu wejściowego 640 i 1024 pikseli w eksperymencie B.

Metryka	YOLO26n			YOLO26s		
	640	1024	Różnica	640	1024	Różnica
<i>mAP50</i>	0.71595	0.86231	+0.14636	0.78551	0.84409	+0.05857
<i>mAP50-95</i>	0.59866	0.71856	+0.11990	0.64119	0.72932	+0.08813
<i>F1-score</i>	0.69568	0.80908	+0.11340	0.73041	0.78712	+0.05672



Rysunek 6.2: Wizualizacja różnicy rozmiaru obiektu na obrazie w zależności od rozdzielczości obrazu wejściowego.

Zwiększenie rozdzielczości obrazu wejściowego przyniosło poprawę wszystkich analizowanych wartości metryk dla obu wariantów modeli. Największe zmiany można zauważyć w przypadku YOLO26n. Wartość metryki *mAP50* wzrosła o 14,64 p.p., a *mAP50-95*

o około 12 p.p. Jednocześnie wskaźnik $F1$ -score wzrósł o 11,34 p.p. Wyniki te wskazują na wyraźny wzrost jakości detekcji wraz z zwiększeniem szczegółowości obrazu wejściowego poprzez zmianę jego rozdzielczości.

Poprawa występuje również dla wariantu **s**, jednak nie osiąga ona takiego poziomu jak w przypadku YOLO26n. Zwiększenie rozmiaru wejścia do 1024 pikseli podniosło wartość $mAP50$ o 5,86 p.p., $mAP50-95$ o 8,81 p.p., a $F1$ -score o 5,67 p.p. W przeciwieństwie do wariantu **n**, największy wzrost obserwuje się w przypadku bardziej rygorystycznej metryki $mAP50-95$.

Wyniki porównań pokazują, że rozmiar obrazu wejściowego ma istotny wpływ na jakość detekcji. Zależność jest szczególnie widoczna dla YOLO26n, dla którego zwiększenie rozdzielczości przyniosło poprawę wszystkich analizowanych metryk o ponad 11 p.p. Może mieć to związek z większą liczbą pikseli reprezentujących obiekty na obrazie, dzięki czemu zachowywane jest więcej informacji o ich krawędziach, powierzchniach, cieniach i innych cechach widocznych na obrazie.

Wyniki eksperymentu B wskazują, że zwiększenie rozmiaru obrazu wejściowego do 1024 pikseli poprawia znacząco jakość detekcji niezależnie od zastosowanej skali modelu. Jednocześnie, konieczność przetworzenia większej ilości danych wejściowych zwiększa koszt obliczeniowy mogąc prowadzić do wydłużenia czasu inferencji. W przeciwieństwie do eksperymentu A, w którym zwiększenie modelu nie musiało przynieść korzyści, eksperyment B wykazuje, że zwiększenie rozdzielczości obrazu wejściowego prowadzi do poprawy działania modeli. Ocena przydatności zwiększenia rozdzielczości musi zostać jednak zestawiona z kosztami obliczeniowymi, jakie nakłada na system przetwarzanie większej ilości danych.

6.4 Wpływ reprezentacji obrazu na jakość detekcji

Największe różnice pomiędzy badanymi modelami wystąpiły w eksperymencie C, który polegał na usunięciu informacji o kolorze z obrazów wejściowych. Przeciwnie do poprzednich eksperymentów, w tym przypadku porównywano modele o jednakowej konfiguracji, uczonych na zbiorach danych o identycznych obrazach i adnotacjach, ale pracujących w reprezentacji kolorowej i w odcieniach szarości. Do porównania wykorzystano trzy dostępne pary modeli: YOLO26n 640 (E_1 , E_5), YOLO26s 640 (E_2 , E_6) oraz YOLO26n 1024 (E_3 , E_7). Wyniki przedstawiono w tabeli 6.5.

Każda para wykazuje spadek skuteczności detekcji dla wszystkich metryk po usunięciu informacji o kolorze. Największą różnicę odnotowano w przypadku modelu YOLO26n przy rozmiarze wejściowym 640 pikseli. Wartość $mAP50-95$ dla tego wariantu spadła o 23,9 p.p. osiągając wartość 36%. Podobne zachowanie obserwujemy w przypadku pozostałych par modeli, gdzie spadek wartości wyniósł 19,2 p.p. dla wariantu YOLO26s 640 oraz 22,4

Tabela 6.5: Porównanie jakości modeli pracujących na obrazach RGB i w odcieniach szarości w eksperymencie C.

Konfiguracja	Reprezentacja	$mAP50$	$mAP50-95$	$F1-score$
YOLO26n 640	RGB	0.71595	0.59866	0.69568
	grayscale	0.48831	0.36001	0.52635
	różnica	-0.22764	-0.23866	-0.16933
YOLO26s 640	RGB	0.78551	0.64119	0.73041
	grayscale	0.55971	0.44940	0.61526
	różnica	-0.22581	-0.19179	-0.11515
YOLO26n 1024	RGB	0.86231	0.71856	0.80908
	grayscale	0.62098	0.49469	0.62426
	różnica	-0.24133	-0.22387	-0.18482

p.p. dla wariantu YOLO26n 1024. Tak duże różnice wskazują, że informacja o kolorze odgrywa istotną rolę w procesie detekcji.

Silny spadek wartości zauważamy też w pozostałych metrykach. Spadki wartości $mAP50$ pomiędzy wariantami wahają się od 22,6 do 24,1 p.p., a dla wskaźnika $F1-score$ od 11,5 do 18,5 p.p. Widzimy tutaj, że usunięcie informacji o kolorze ma wpływ nie tylko na rozpoznanie klasy obiektu, ale prowadzi do ogólnego pogorszenia skuteczności detekcji we wszystkich badanych przypadkach.

Jednocześnie, modelem najlepiej radzącym sobie w środowisku, w którym wykonywana jest detekcja na obrazach w odcieniach szarości, jest wariant E_7 , czyli YOLO26n operujące na obrazach o rozdzielczości 1024 pikseli. Wartość metryki $mAP50-95$ dla tego wariantu wyniosła 49,5%, a dla pozostałych wariantów E_5 i E_6 wyniosła około 36% i 44,9%. Wyniki te zgadzają się z obserwacjami z eksperymentu B, zgodnie z którymi, w badanym problemie, zwiększenie rozdzielczości skutkuje poprawą jakości detekcji. Nie niweluje to jednak ograniczeń wynikających z braku informacji o kolorze, które prowadzą do bardzo dużego spadku zdolności modelu do wykrywania obiektów.

Eksperyment nie pozwala jednoznacznie określić, że modele RGB rozpoznają bryły wyłącznie na podstawie ich koloru. Modele pracujące na odcieniach szarości nadal zachowują zdolność do wykrywania obiektów, choć popełniają więcej błędów. Wskazuje to, że podczas detekcji modele korzystają również z innych cech wykrywanych obiektów, jak krawędzie, powierzchnie i wierzchołki. Uzyskane wyniki wskazują jednak, że informacja o barwie stanowi ważne źródło informacji, a jej usunięcie prowadzi do znacznego pogorszenia jakości detekcji.

W kontekście docelowego zastosowania systemu to porównanie jest szczególnie ważne. Zakłada się, że bryły używane w środowisku docelowym mogą znacznie różnić się kolorystyką od zestawu wykorzystywanego do stworzenia zbioru uczącego. Wariant pracujący w grayscale ogranicza możliwość korzystania modelu z informacji o kolorze, jednakże wyniki eksperymentu C pokazują, że usunięcie kanału koloru prowadzi do znacznego pogorszenia jakości detekcji. Przeprowadzone porównanie nie pozwala stwierdzić, że badane warianty zapewniają odporność na zmiany kolorystyki. Ocena tego podejścia wymaga dalszych badań na zbiorach danych zawierających inne obiekty odpowiadające wykrywanym klasom niż te, które zostały użyte do treningu.

Na podstawie wyników można jednoznacznie stwierdzić, że usunięcie informacji barwnej z obrazów znacznie pogarsza jakość detekcji wszystkich badanych konfiguracji. Najlepszy wariant nie przekroczył wartości 50% dla metryki mAP_{50-95} , co jest wynikiem niesatysfakcjonującym w kontekście zastosowania systemu w środowisku docelowym. Wskazuje to jednak, że informacja o kolorze nie jest niezbędna do wykonania detekcji. Pomimo tego, rozwój modeli pracujących w odcieniach szarości może stanowić interesującą gałąź badań, szczególnie w kontekście zmiennych warunków oświetleniowych czy różnorodności kolorystycznej obiektów w środowisku docelowym.

6.5 Porównanie najlepszych konfiguracji

Przeprowadzone eksperymenty nie pozwoliły na jednoznaczne wyłonienie wariantu modelu, który osiągnąłby najlepsze wyniki we wszystkich analizowanych aspektach. Uwidoczniły jednak znaczące różnice w skuteczności detekcji, w zależności od konfiguracji danego modelu. Z punktu widzenia zastosowania systemu w środowisku docelowym, sama jakość detekcji nie jest jednak jedynym kryterium wyboru. Ponieważ przetwarzanie obrazu ma odbywać się w czasie rzeczywistym, istotne są również koszt obliczeniowy oraz czas potrzebny na wykonanie predykcji.

Do końcowego porównania wybrano cztery konfiguracje reprezentujące najważniejsze kompromisy ukazane w wcześniejszych badaniach: E_1 , E_3 , E_4 oraz E_7 . Pomiar czasu inferencji i liczby klatek na sekundę wykonano na urządzeniu Apple MacBook Air z procesorem Apple M1 i 8GB pamięci RAM, używając MPS (ang. *Metal Performance Shaders*). Zestawienie jakości detekcji i kosztu obliczeniowego przedstawiono w tabeli 6.6.

Spośród przedstawionych konfiguracji, najwyższą wartość mAP_{50-95} osiągnął wariant E_4 , będący modelem YOLO26s pracującym na obrazach kolorowych o rozdzielczości 1024 pikseli. Jest to jednocześnie największy model, który wymaga najwięcej zasobów obliczeniowych. Bardzo podobne wyniki osiągnął wariant E_3 , który pracuje na obrazach tej samej rozdzielczości, ale jest mniejszym modelem YOLO26n. Różnica pomiędzy tymi wariantami w wartości metryki mAP_{50-95} wyniosła 1,08 p.p. Oba modele osiągają więc

Tabela 6.6: Porównanie wybranych konfiguracji modeli pod względem jakości i kosztu obliczeniowego.

Konfiguracja	mAP_{50-95}	F1-score	L. Param.	GFLOPs	Czas	FPS
E_1 : n RGB 640	0.59866	0.69568	2.51 M	5.78	16.55ms	60.4
E_3 : n RGB 1024	0.71856	0.80908	2.51 M	14.80	37.72ms	26.5
E_4 : s RGB 1024	0.72932	0.78712	9.95 M	57.65	85.26ms	11.7
E_7 : n grayscale 1024	0.49469	0.62426	2.51 M	14.80	36.15ms	27.7

zbliżoną skuteczność detekcji, a porównanie metryk nie pozwala jednoznacznie wskazać, który z nich jest lepszy.

Koszt zastosowania większego wariantu jest jednak czynnikiem, który może dyskwalifikować jego użycie w systemie. Model E_4 zawiera około 9,95 mln parametrów wobec 2,51 mln znajdujących się w wariacie E_3 , a liczba operacji wynosi odpowiednio 57,65 GFLOPs wobec 14,80 GFLOPs. Oznacza to, że użycie wariantu E_4 prowadzi do niemal czterokrotnego zwiększenia tych wartości mimo że główną korzyścią jest minimalny wzrost metryki mAP_{50-95} . Nie jest to wystarczający powód, aby wybrać wariant E_4 jako najlepszy do zastosowania w systemie.

Różnice są widoczne również w czasie predykcji. Wariant E_3 jest w stanie przetworzyć obraz wejściowy w czasie o ponad połowę krótszym niż większy model, osiągając możliwość predykcji 26,5 klatek na sekundę. Warto jednak zaznaczyć, że mierzony czas uwzględnia jedynie działanie mechanizmu predykcji modelu, a nie czas odpowiedzi systemu. Na ten wpływają również transmisja obrazu, jego dekodowanie i pozostałe etapy przetwarzania. Zmierzone wartości służą jedynie do porównania modeli uruchomionych w tych samych warunkach.

Odmiennej kierunku optymalizacji zauważamy w przypadku wariantu E_1 , który działa na obrazach o mniejszej rozdzielczości. Reprezentuje on podejście nastawione przede wszystkim na optymalizację kosztów, przez co osiąga wyniki gorsze od wariantu E_3 o około 12 p.p. w metryce mAP_{50-95} . Jednocześnie obniża koszt predykcji do poziomu 5,78 GFLOPs, co stanowi wartość niemal trzykrotnie mniejszą względem wariantu E_3 . Wpływa to również na płynniejsze działanie, będąc w stanie przetworzyć pojedynczy obraz wejściowy w czasie krótszym niż 17 ms. Ten wariant może być rozważany w przypadku jednostek o bardzo ograniczonej mocy obliczeniowej, jednak nie ukazuje tak dobrego kompromisu pomiędzy skutecznością detekcji a kosztem obliczeniowym.

Osobną rolę w porównaniu stanowi model E_7 pracujący na obrazach w odcieniach szarości. Jest on najlepszym spośród trzech wariantów wykorzystujących dodatkowy filtr przekształcający obrazy. Jego koszt obliczeniowy pozostaje zbliżony do wariantu E_3 – oba wykorzystują tę samą architekturę i ten sam rozmiar obrazu wejściowego. Znacznie niższa skuteczność detekcji w porównaniu do pozostałych wariantów jest jednak istotnym

ograniczeniem, które może dyskwalifikować go w zastosowaniach wymagających wysokiej skuteczności detekcji. Może jednak stanowić istotny punkt odniesienia przy dalszych badaniach nad zależnością detekcji od informacji o barwie.

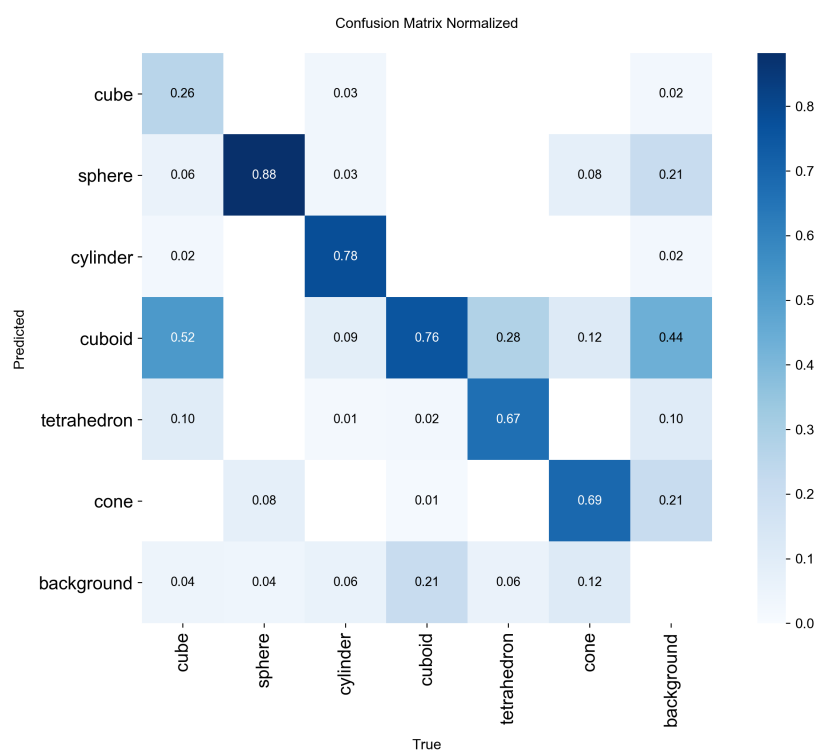
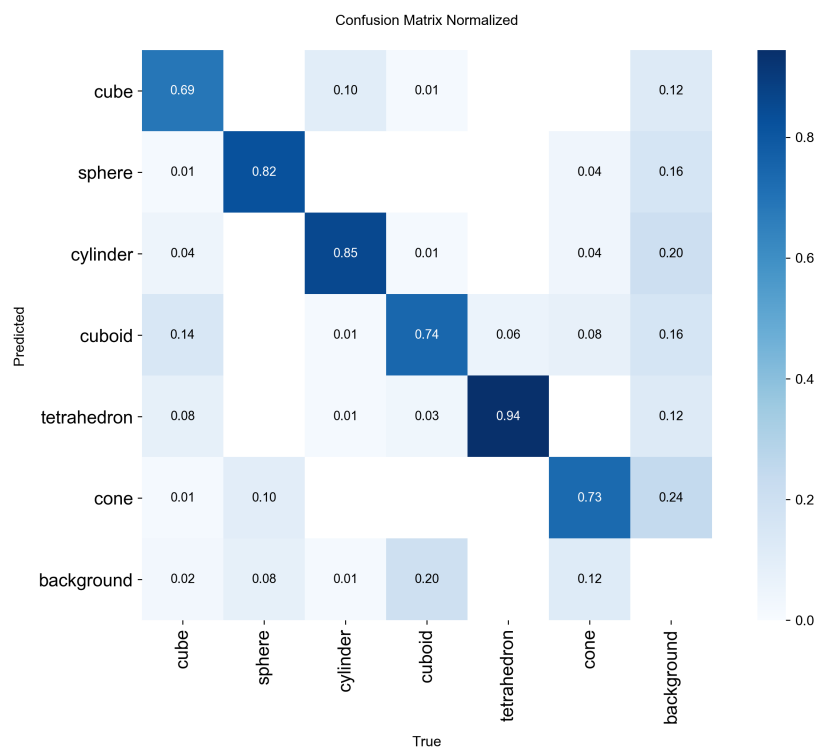
Z punktu widzenia kompromisu pomiędzy skutecznością a kosztem obliczeniowym, najkorzystniejszym wariantem pozostaje konfiguracja E_3 : YOLO26n pracujący na obrazach kolorowych o rozdzielczości 1024 pikseli. Osiąga on gorsze wartości metryki mAP_{50-95} , jednocześnie osiągając lepsze wartości wskaźnika $F1$ -score niż wariant E_4 . Jednocześnie jest od niego znacznie szybszy i powinien być pierwszym wyborem do zastosowania w systemie. W badaniach dodatkowy koszt wynikający z użycia wariantu YOLO26s nie przynosi poprawy jakości wystarczającej do uzasadnienia wyboru większego wariantu jako podstawowego modelu systemu.

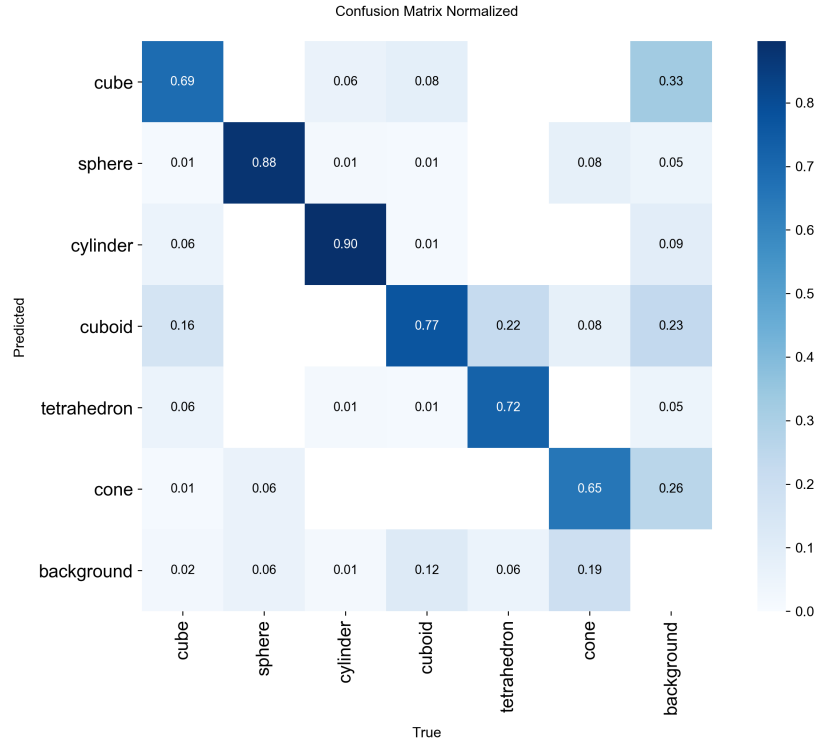
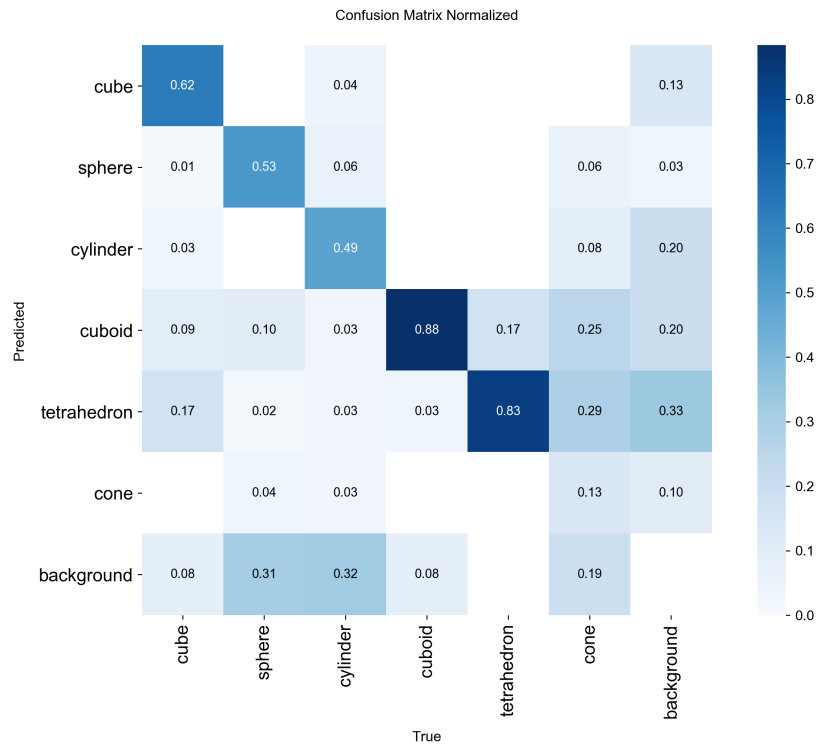
6.6 Analiza błędów detekcji

Globalne wartości mAP i $F1$ -score pozwalają wykonać ogólną ocenę skuteczności modeli, jednakże nie ukazują one szczegółowego obrazu błędów popełnianych przez system. Dlatego, w celu ich dokładniejszego określenia należy przyjrzeć się macierzom pomyłek, które zostały uzyskane podczas pracy modeli na zbiorze testowym. Wybrano te same cztery konfiguracje, które zostały poddane ocenie w poprzedniej sekcji. Za punkt odniesienia przyjęto wariant E_3 , który został wskazany jako najkorzystniejszy pod względem ocenianego kompromisu. Pozostałe warianty pozwalają natomiast ocenić, w jaki sposób zmienia się charakter błędów w zależności od czynników treningu.

Znormalizowane macierze pomyłek ukazują procentowy udział predykcji modelu. Mocno zarysowana główna przekątna wskazuje, że model poprawnie klasyfikuje większość obiektów, a wartości poza nią wskazują na błędy detekcji. Kolumny macierzy odpowiadają rzeczywistym klasom obiektów, natomiast wiersze klasom przewidywanym przez model. Wiersz *background* oznacza przypadki, w których model nie wykrył żadnego obiektu, a kolumna *background* przypadki, w których na obrazie nie było żadnego obiektu. Macierze analizowanych modeli przedstawiono na rysunku 6.3.

Szczególnie wyraźną różnicę zauważyć można porównując modele E_1 i E_3 , które różnią się jedynie rozmiarem obrazu wejściowego. Pierwszy z nich ma problemy z wykrywaniem sześciątów, które bardzo często klasyfikuje jako prostopadłościany. Poprawnie sklasyfikowano jedynie 26% z nich, a ponad połowa została przypisana do klasy prostopadłościanów. Zauważamy też, że model często znajduje prostopadłościany w miejscach, w których ich nie ma, co może wskazywać na ich nadmierne przypisywanie obiektów do tej klasy. Zwiększenie rozmiaru obrazu wejściowego w wariantcie E_3 znacznie poprawiło skuteczność odróżnienia sześciątów od prostopadłościanów, co może wynikać z zachowania większej ilości szczegółów obrazu, ułatwiających rozróżnienie proporcji, ścian i krawędzi obu brył.

(a) Wariant E_1 : YOLO26n 640 RGB.(b) Wariant E_3 : YOLO26n 1024 RGB.

(c) Wariant E_4 : YOLO26s 1024 RGB.(d) Wariant E_7 : YOLO26n 1024 grayscale.

Rysunek 6.3: Wybrane znormalizowane macierze pomyłek badanych modeli.

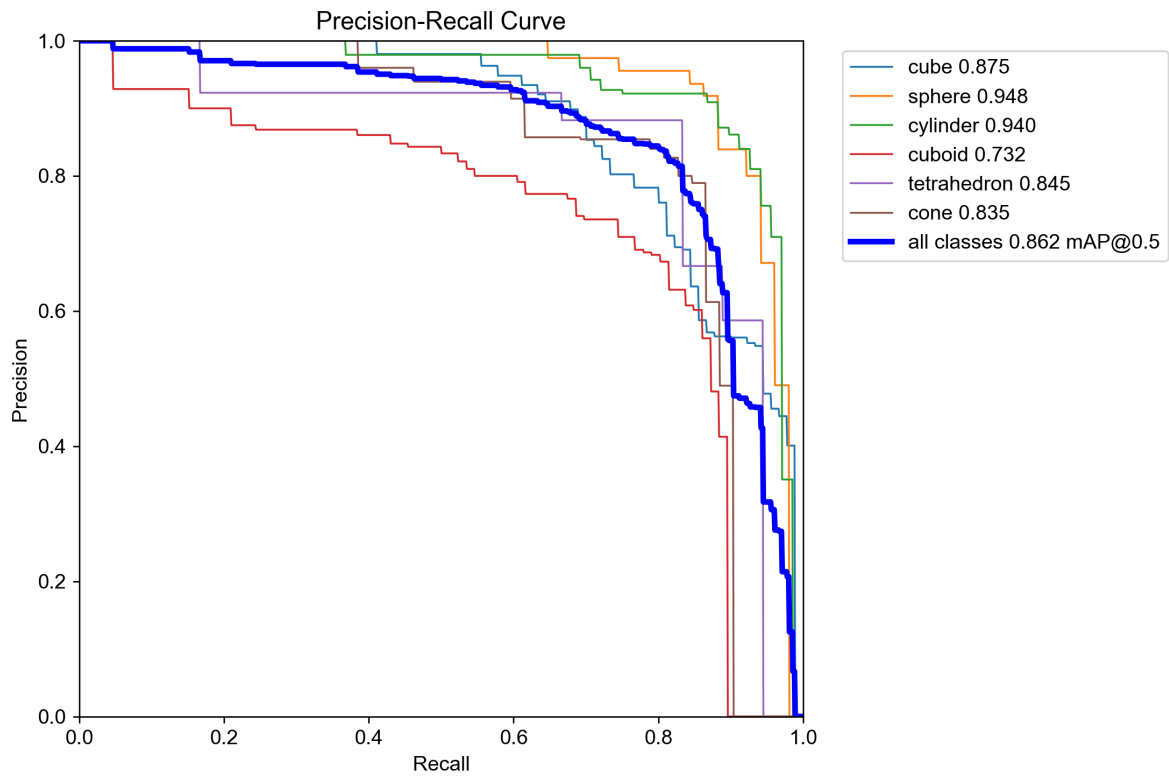
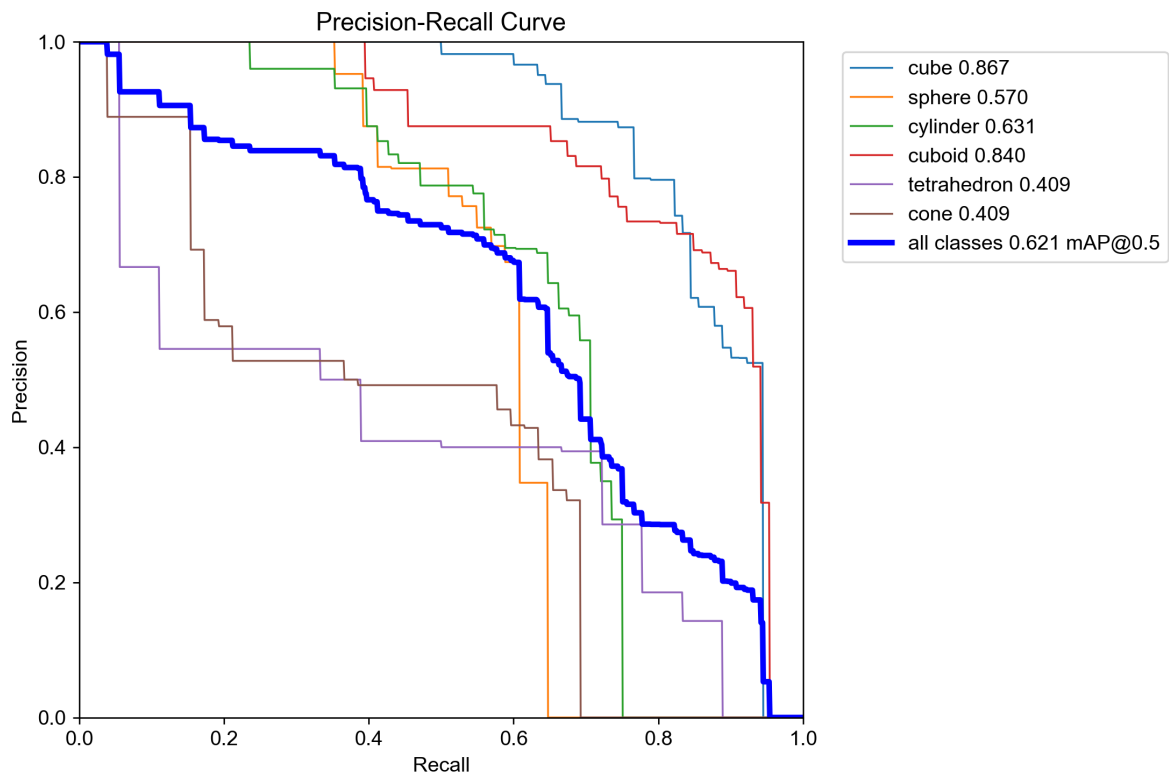
Podobne zachowanie obserwujemy dla innych klas. Zwiększenie rozdzielczości skutkuje wzrostem poprawnych klasyfikacji z 78% do 85% dla walca i z 67% do 94% dla czworościanu. Nie wszystkie klasy jednak reagują w ten sam sposób – dla kuli ilość poprawnych predykcji spada o 6 punktów procentowych, a dla prostopadłościanu pozostaje na zbliżonym poziomie. Te wyniki uzupełniają wnioski płynące z eksperymentu B: zwiększenie rozdzielczości obrazu poprawia globalną zdolność modelu do wykrywania obiektów, ale nie dla każdej klasy osobno.

Przy zestawieniu modeli E_3 i E_4 widzimy zgoła inny obraz. Zwiększenie skali modelu poprawia udział poprawnych detekcji dla kuli, walca i prostopadłościanu, ale osiąga znacząco gorsze wyniki dla czworościanu i stożka. Dla pierwszego z nich ilość poprawnych detekcji spada o 22 punkty procentowe, a dla drugiego o 8 punktów procentowych. Niewielka przewaga w mAP_{50-95} wariantu E_4 nie wynika więc z równomiernej poprawy detekcji wszystkich klas, a raczej z ograniczenia części błędów. Jednocześnie, większy model popełnia więcej błędów w innych przypadkach, co stanowi kolejny argument za wyborem wariantu E_3 do zastosowania w systemie.

Największą zmianę w charakterze i ilości błędów detekcji obserwujemy w przypadku wariantu E_7 , korzystającego z obrazów w odcieniach szarości. Usunięcie informacji o kolorze nie wpływa równomiernie na zdolność rozpoznawania poszczególnych klas. Na wysokim poziomie pozostaje wykrywanie prostopadłościanów i czworościanów, osiągając wyniki powyżej 83%. W porównaniu z modelem E_3 zdecydowanie gorzej radzi sobie z wykrywaniem kuli i walca – w obu przypadkach spadek wyniósł ponad 29 punktów procentowych. Najgorszy wynik został zanotowany dla klasy stożka, którego poprawne wykrycie odbyło się tylko w 13% przypadków, a 29% z nich zostało sklasyfikowane jako czworościany. Wskazuje to, że usunięcie informacji o barwie szczególnie wpływa na zdolność modelu do rozróżniania tej klasy od innych.

Model pracujący na obrazach w odcieniach szarości wykazuje również większą liczbę pominiętych obiektów. Wiersz *background* macierzy odpowiada obiektom referencyjnym, do których nie dopasowano żadnej predykcji. Dla modelu E_7 udział takich przypadków wynosi 31% dla kuli i 32% dla walca, podczas gdy model E_3 wartości te osiąga odpowiednio 8% i 1%. Dla pozostałych klas wyniki pozostają na zbliżonym poziomie. Pogorszenie wyników wynika więc nie tylko z błędnego przypisania klas, ale również ze zwiększonej liczby niewykrytych obiektów.

Macierze pomyłek nie są jednak jedynym źródłem informacji o błędach detekcji. Warto również przyjrzeć się krzywym precyzji i czułości, które pozwalają ocenić jak zmienia się kompromis pomiędzy tymi dwoma metrykami w zależności od progu decyzyjnego. Z tego względu analizę uzupełniono o wykresy krzywych precision–recall. Na rysunku 6.4 zestawiono je dla wariantów E_3 oraz E_7 . Reprezentują one wybraną konfigurację docelową oraz wpływ ograniczenia koloru na jakość detekcji.

(a) Wariant E_3 : YOLO26n 1024 RGB.(b) Wariant E_7 : YOLO26n 1024 grayscale.

Rysunek 6.4: Krzywe precision–recall modeli E_3 i E_7 uzyskane na zbiorze testowym.

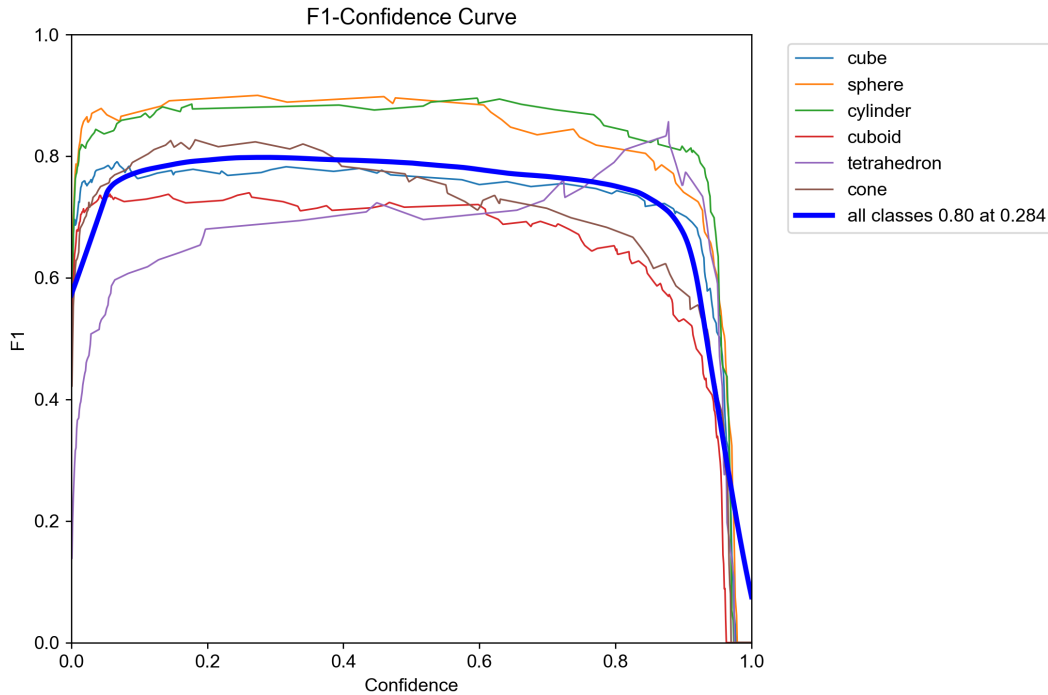
Krzywe PR umożliwiają ocenę, w jaki sposób wzrost czułości oddziałuje na precyzję detekcji poszczególnych klas. Ich przebieg ukazuje rozwinięcie informacji przekazywanych przez metryki AP i mAP , ponieważ pozwala ocenić zachowanie modelu w całym zakresie kompromisu pomiędzy tymi dwoma metrykami. Porównanie wariantów E_3 i E_7 pozwala sprawdzić, czy pogorszenie jakości spowodowane usunięciem informacji o kolorze dotyczy pojedynczego progu działania, czy też jest widoczne w szerszym zakresie pracy modelu.

Dla modelu E_3 średnia wartość AP przy progu IoU równym 0,5 wyniosła 0,862. Najwyższe wartości uzyskano dla klas kuli i walca, które osiągnęły wartości powyżej 0,94, a także dla sześciangu, którego AP50 wyniosło 0,875. Najsłabszym wynikiem spośród wszystkich klas charakteryzuje się prostopadłościan, dla którego wartość wyniosła 0,732. Mimo obserwowanej na macierzach pomyłek tendencji do mylenia sześciangów z prostopadłościanami, model prezentuje wysoką precyzję dla większości klas, również po zwiększeniu progu czułości.

Znacznie bardziej zróżnicowany przebieg krzywych PR obserwujemy w przypadku wariantu E_7 . W jego przypadku $mAP50$ spada do wartości 0,621. Model osiąga najwyższe wyniki dla prostopadłościanu i sześciangu, dla których wartości AP50 są powyżej 0,84, pozostając zbliżonymi do modelu pracującego w przestrzeni kolorowej. Pozostałe klasy osiągają jednak znacznie niższe wyniki. Najniższe wartości uzyskano dla stożka i czworoscianu, dla których AP50 wyniosło około 0,41. Ma to swoje odzwierciedlenie w macierzy pomyłek, w której widać, że model ma największe problemy z wykrywaniem stożków.

Porównanie krzywych pochodzących z tych dwóch modeli potwierdza więc, że pogorszenie wyników po usunięciu informacji o kolorze nie jest ograniczone do jednego progu czułości, a dotyczy całego zakresu pracy modelu. Różnice utrzymują się w szerokim zakresie kompromisu, a ich wielkość jest silnie zależna od klasy obiektu. Szczególnie istotny jest fakt, że oba modele zachowują wysokie wartości AP50 dla sześciangu i prostopadłościanu niezależnie od wykorzystania koloru do detekcji. Niestety, dla pozostałych klas wartość ta ulega znacznemu pogorszeniu. Wskazuje to, że model wykorzystuje informacje o kolorze w procesie detekcji, lecz jego znaczenie nie jest jednakowe dla wszystkich klas.

Poza zależnościami precision-recall z punktu widzenia systemu istotny jest również wybór progu pewności predykcji. Jego zbyt niskie ustawienie może zwiększyć liczbę wykrywanych obiektów, jednak może również prowadzić do zwiększenia ilości fałszywych predykcji. Z drugiej strony, zbyt rygorystyczne ustawienie progu ograniczy udział błędnych wskazań kosztem pominięcia części obiektów. Wybór optymalnej wartości progu, w którym zachowany jest najlepszy kompromis pomiędzy precyzją a czułością, może zostać dokonany na podstawie krzywej $F1$. Zależność $F1$ -score od progu pewności predykcji dla wariantu E_3 przedstawiono na rysunku 6.5.

Rysunek 6.5: Krzywa $F1$ dla wariantu E_3 .

Dobór odpowiedniej wartości progu jest szczególnie ważny w kontekście docelowego systemu, ponieważ wpływa on bezpośrednio na to, które detekcje zostaną uznane za wystarczająco pewne i przekazane do dalszego przetwarzania. Wynik benchmarku jakościowego może więc posłużyć nie tylko do porównania modeli, ale przede wszystkim do doboru parametrów pracy wybranego modelu E_3 w systemie docelowym.

Dla modelu YOLO26n 1024 RGB, który został wskazany jako wariant docelowy, najwyższa wartość krzywej $F1$ jest osiągana przy progu pewności predykcji równym około 0,284 i wynosi około 0,8. W tym obszarze zachowany jest najlepszy kompromis precyzji i czułości. Przy zwiększaniu progu możemy zauważyć stopniowy spadek wartości wskaźnika, a wysokie wartości pewności – powyżej 0,9 – prowadzą do załamania się krzywej i nagłego spadku wartości $F1$ -score. Wartość progu około 0,284 może więc zostać przyjęta jako punkt odniesienia przy konfiguracji systemu docelowego, ale nie należy jej traktować jako wartości ostatecznej. Koszty poniesione przez błędne detekcje i pominięte obiekty nie muszą być jednakowe w każdym przypadku, dlatego ostateczny próg powinien zostać dobrany zależnie od oczekiwanego działania systemu. W szczególności zasadne może być jego zwiększenie celem ograniczenia błędnych informacji przekazywanych użytkownikowi końcowemu, kosztem częstszego pomijania obiektów.

Analiza macierzy pomyłek i krzywych pokazuje, że różnice poszczególnych modeli nie sprowadzają się wyłącznie do zmian parametrów i wartości metryk globalnych. Poszczególne zmiany wpływają na charakter i częstość popełnianych błędów. Zwiększenie rozdzielczości obrazu wejściowego w największym stopniu ułatwiło modelowi odróżnienie

sześcianów od prostopadłościanów, a zwiększenie skali modelu poprawiło wyniki tylko części klas. Największy wpływ miało usunięcie informacji o kolorze, ale ono również nie wpłynęło równomiernie na wszystkie klasy. Zdolność rozpoznawania sześciianu i prostopadłościanu pozostała na wysokim poziomie.

Podczas interpretacji wyników należy jednak pamiętać, że w zbiorze testowym nie występowała równa liczba klas obiektów. W przypadku klas reprezentowanych przez małą liczbę obiektów, pojedynczy błąd może mieć znaczący wpływ na wartości widoczne w znormalizowanej macierzy. Takie zachowanie można zaobserwować w przypadku klasy czworoscianu, która występowała w zbiorze testowym jedynie 18 razy. Z tego względu nie należy wyciągać zbyt daleko idących wniosków na podstawie różnic obserwowanych w przypadku tej klasy ani formułować wniosków o statystycznie istotnych różnicach w skuteczności detekcji.

Macierze pomyłek i krzywe PR nie stanowią również pełnego przeglądu wszystkich aspektów detekcji. Model może poprawnie wskazywać klasę obiektu, ale nieprawidłowo określać jego położenie. Taki błąd będzie widoczny w metryce mAP_{50-95} , ale nie musi być odzwierciedlony w analizie klas. Z tego powodu wyniki przedstawione w tej sekcji należy rozpatrywać łącznie z wynikami metryk globalnych oraz z porównaniem wydajności modeli. Dopiero w ten sposób możliwe jest uzyskanie pełnego obrazu skuteczności detekcji i charakteru popełnianych błędów.

6.7 Dyskusja wyników i ograniczeń badań

Przeprowadzone badania pozwoliły ocenić wpływ kilku czynników na jakość detekcji dotykanych obiektów trójwymiarowych. Analiza wykazuje, że w badanym problemie nie wystarczy jedynie zwiększyć złożoności modelu, aby poprawić jego skuteczność. Znaczący wpływ na wyniki ma również rozdzielczość obrazu wejściowego oraz informacja o kolorze. Szczególnie uwidacznia się to przy porównaniu rozmiaru modelu i rozdzielczości wejścia. Zwiększenie skali z YOLO26n do YOLO26s nie przynosiło jednoznacznej poprawy wszystkich metryk, a w niektórych przypadkach pogarszało wyniki. Zwiększenie rozdzielczości z 640 do 1024 pikseli przynosiło natomiast znaczącą poprawę wszystkich analizowanych wartości niezależnie od wybranego wariantu modelu. Może to wskazywać, że w badanym problemie bardzo ważna jest możliwość uchwycenia szczegółów obrazu, które pozwalają modelowi odróżnić poszczególne klasy obiektów.

Wielkość adnotowanych obiektów w zbiorze danych może częściowo wyjaśniać poprawę wyników obserwowaną po zwiększeniu rozdzielczości obrazu wejściowego. W przygotowanym zbiorze danych obiekty zajmowały średnio około 1,51% powierzchni obrazu, a największy z nich zajął około 7,31% powierzchni. Różnice w wielkościach wynikają przede wszystkim z różnej odległości obiektów od kamery, a także z ich zasłonięć palcami. W takich warunkach zwiększenie rozdzielczości obrazu wejściowego pozwala zachować znacznie

więcej szczegółów, które mogą być istotne w procesie detekcji. Wniosek ten dotyczy jednak wyłącznie modeli uczonych na przygotowanym zbiorze danych i nie może być uogólniany na inne zadania detekcji obiektów.

Bardzo istotnym wynikiem jest duży spadek skuteczności detekcji po usunięciu informacji o barwie. W każdym z badanych przypadków zaobserwowano, że modele pracujące na obrazach grayscale osiągają wyniki gorsze od wariantów wykorzystujących obrazy RGB. Nie oznacza to jednak, że modele rozpoznają obiekty wyłącznie na podstawie koloru. Powierzchnie poszczególnych brył z zastosowanego zestawu zostały pokryte różnymi kolorami należącymi do wspólnej palety barw, więc nie występuje jednoznaczne powiązanie koloru z klasą obiektu. Informacja o barwie może stanowić dodatkową cechę wykorzystywaną przez model, na przykład przez charakterystyczne kombinacje kolorów, ich rozmieszczenie lub udział w widocznej części bryły. Przeprowadzone badania nie pozwalają jednoznacznie określić, w jakim stopniu modele korzystają z tych zależności. Ograniczenie to wynika ze sposobu przygotowania zbioru danych, w którym zastosowano pojedynczy zestaw fizycznych brył o stałym przypisaniu kolorów do ich powierzchni. Nie przeprowadzono eksperymentu z innym zestawem brył, w którym kolory powierzchni byłyby przypisane w inny sposób. Nie jest więc możliwe określenie, jaki wpływ na działanie modeli RGB miałyby zmiana przypisania barw do powierzchni obiektów. Aby można było to ocenić, konieczne byłoby przygotowanie dodatkowych wariantów brył o zmienionym układzie kolorów i sprawdzenie zachowania modeli w takich warunkach.

Kolejne ograniczenia wynikają z samego sposobu przygotowania zbioru danych. W badaniach wykorzystano obrazy pochodzące z kilku nagrań wideo z których wyodrębniono pojedyncze klatki. Takie podejście umożliwiło pozyskanie dużej liczby obrazów, ale przedstawiających podobną scenę i obiekty w podobnych pozycjach. Zastosowany podział według nagrań pozwolił na ograniczenie możliwości przecieku podobnych klatek pomiędzy zbiorami, ale nie zwiększył różnorodności warunków. Liczba różnych scen, osób, sposobów ułożenia kamery i warunków otoczenia pozostaje więc ograniczona. Sam zbiór testowy obejmuje 477 obrazów, co nie pozwala na pełną ocenę generalizacji modeli na inne pomieszczenia, urządzenia rejestrujące, warunki oświetleniowe czy sposób eksploracji brył. Dodatkowo, użycie pojedynczego urządzenia do rejestracji nagrań nie pozwala na ocenę wpływu różnych parametrów kamery na pracę modeli.

Duże znaczenie może również odgrywać nierównomierna liczebność klas. Widać ją w zbiorze uczącym, gdzie występuje znaczna przewaga czworościanów, oraz w zbiorze testowym, gdzie obecne są klasy reprezentowane przez bardzo małą liczbę obiektów. W przypadku klas o niewielkiej liczebności, pojedynczy błąd może mieć bardzo duży wpływ na wyniki klasowe. Z tego powodu wyniki macierzy pomyłek powinny być raczej interpretowane w kontekście wskazania charakteru błędów, a nie jako podstawa do stwierdzenia przewagi jednego wariantu nad drugim.

Ograniczenie procedury eksperymentalnej stanowi również liczba wykonanych treningów. Dla każdego badanego wariantu przeprowadzono pojedynczy trening z ustalonym ziarnem generatora liczb pseudolosowych. Umożliwia to zwiększenie powtarzalności warunków eksperymentów, ale nie pozwala na ocenę wpływu zmienności wyników pomiędzy niezależnymi treningami tej samej konfiguracji. Niewielkich różnic pomiędzy modelami nie można na tej podstawie interpretować jako istotnych statystycznie. Powtórzenie treningów i ponowna ocena wytrenowanych modeli umożliwiłaby wyznaczenie dodatkowych własności statystycznych i bardziej wiarygodną ocenę różnic pomiędzy wariantami.

Spośród ośmiu możliwych wariantów, będących kombinacjami skali modelu, rozdzielczości obrazu wejściowego i przestrzeni barwnej, wytrenowano jedynie siedem. Brakujący wariant, który nie został poddany treningowi, to YOLO26s 1024 grayscale. Nie został on uwzględniony w przyjętym zakresie badań. Jego brak nie uniemożliwił wykonania pozostałych porównań parami, ale nie pozwala na pełną ocenę wpływu wszystkich czynników na skuteczność detekcji.

Do treningu wykorzystano różne jednostki obliczeniowe, bazując na ich dostępności. Sam sprzęt i systemy operacyjne nie stanowiły zmiennych badanych w eksperymentach, dlatego nie można na ich podstawie formułować wniosków o wpływie na przebieg i czas uczenia.

Porównanie kosztu inferencji przeprowadzono w jednakowych, kontrolowanych warunkach sprzętowych, dzięki czemu uzyskane wyniki mogły posłużyć porównaniu modeli.

Zakres badań obejmował jedynie skuteczność modułu detekcji obiektów. Nie przeprowadzono badań z udziałem osób niewidomych ani eksperymentów oceniających ergonomię i użyteczność systemu w środowisku docelowym. Uzyskane wartości metryk nie mogą stanowić więc podstawy do oceny jakości działania całego systemu, jakości przekazywanych komunikatów, ani rzeczywistego wpływu na proces nauki geometrii przestrzennej. Są one jedynie oceną technicznej zdolności modeli detekcji obiektów w warunkach laboratoryjnych.

Przedstawione ograniczenia wskazują potencjalne kierunki kolejnych badań i wyznaczają w jakim zakresie należy interpretować wyniki. Pokazują, że skuteczne wykorzystanie metod uczenia maszynowego w systemach wspomagających jest możliwe oraz pozwalają określić wpływ wybranych parametrów na jakość działania modeli. Nie stanowią jednak dowodu na pełną odporność systemu na zmiany warunków, środowiska, kolorystyki czy sposobu eksploracji obiektów. Wnioski płynące z badań należy więc odnosić do warunków eksperymentalnych w których zostały przeprowadzone.

Rozdział 7

Podsumowanie i wnioski końcowe

Po zakończeniu badań i eksperymentów przeprowadzonych w ramach niniejszej pracy można sformułować kilka istotnych wniosków i refleksji dotyczących skuteczności zastosowania metod detekcji opartych na uczeniu głębokim w kontekście rozpoznawania obiektów trzymanych przez osoby niewidome i słabowidzące. Praca objęła część projektowo-implementacyjną oraz badawczą. Ten rozdział stanowi podsumowanie wykonanych prac, ich ocenę oraz wskazanie potencjalnych kierunków dalszego rozwoju systemu.

7.1 Podsumowanie wykonanych prac

Pierwszym etapem pracy było poszerzone studium literatury, które uwidoczniło kierunki rozwoju technologii wspomagających osoby niewidome i słabowidzące. Pośród nich znajdujemy wiele rozwiązań opartych na przetwarzaniu obrazu, które umożliwiają wsparcie w codziennych czynnościach. Nie zidentyfikowano w nim jednak żadnej pracy, która opisywałaby wsparcie osób niewidomych w nauce geometrii przestrzennej poprzez interakcję z fizycznymi modelami trójwymiarowymi. W związku z tym, należało opracować własny system, który byłby odpowiedzią na zidentyfikowaną lukę w literaturze.

Rozwiązanie oparto na architekturze klient-serwer i metodach detekcji obiektów. Konceptyjnie rozdzielono odpowiedzialności pomiędzy urządzeniem mobilnym, pełniącym rolę klienta, a serwerem, który przetwarzał dostarczone informacje i zwracał wyniki detekcji. Wyznaczono również wymagania funkcjonalne i нефункционалне systemu, które stanowiły podstawę do dalszych prac.

Założenia, które przyjęto dla systemu, zostały następnie zrealizowane poprzez stworzenie funkcjonalnego prototypu. W jego ramach powstały aplikacja kliencka przeznaczona na urządzenia mobilne z systemem Android, serwer przetwarzający dane oraz zestaw modeli detekcji obiektów. Zadana architektura systemu umożliwiła połączenie różnych komponentów w spójną całość, co sprawiło, że prototyp może zostać uznany za funkcjonalny i gotowy do dalszego rozwoju. Interakcja z użytkownikiem końcowym została

oparta na komunikacji głosowej, co wykluczyło konieczność korzystania z ekranów dotykowych i odrywania rąk od trzymanyh obiektów. Stworzono również zestaw narzędzi do przygotowania danych treningowych i treningu modeli.

Aby możliwe było użycie modeli detekcji, konieczne było przygotowanie odpowiedniego zbioru danych. W tym celu nagrano i przetworzono zestaw filmów z których wyodrębniono obrazy zawierające osoby trzymające w dłoniach różne bryły geometryczne. Łącznie na zbiór składa się 4666 ręcznie adnotowanych obrazów, które zostały następnie podzielone na zbiory treningowy, walidacyjny i testowy.

Część badawcza pracy obejmowała eksperymenty z udziałem siedmiu modeli YOLO26 trenowanych w różnych konfiguracjach. Celem tej części było zbadanie możliwości ich użycia w implementowanym systemie. Badano wpływ skali modelu, rozdzielczości wejściowej i reprezentacji obrazu. Ostatecznie wskazano model, który wykazywał najlepszy kompromis jakości i kosztu obliczeniowego, charakteryzując się krótkim czasem inferencji przy zachowaniu korzystnej skuteczności detekcji.

7.2 Ocena realizacji celu pracy

Celem pracy było opracowanie systemu wspomagającego osoby niewidome i słabowidzące w nauce geometrii przestrzennej za pomocą fizycznych modeli 3D. Cel ten zrealizowano poprzez stworzenie funkcjonalnego prototypu systemu, który umożliwia rejestrowanie i przesyłanie obrazu, detekcję brył trzymanyh przez użytkownika, analizę widocznej części obiektu oraz przekazywanie informacji w formie głosowej. Zaimplementowane rozwiązanie realizuje więc podstawowy scenariusz użytkowy przyjęty na etapie projektowania.

Przeprowadzono również eksperymentalną ocenę wytrenowanych modeli detekcji. Uzyskane wyniki potwierdziły, że modele były w stanie wykrywać wszystkie sześć klas obiektów uwzględnionych w pracy. Badania umożliwiły wskazanie jednego z modeli jako najkorzystniejszego kompromisu pomiędzy skutecznością detekcji a kosztem obliczeniowym i czasem inferencji. Nie oznacza to jednak, że jest on w pełni odporny na zmiany środowiska, ponieważ to nie było przedmiotem osobnego eksperymentu i nie zostało zweryfikowane w warunkach docelowych.

Zaimplementowany prototyp pozwala na jego działanie w kontrolowanych warunkach oraz przeprowadzenie dalszych testów z udziałem osób niewidomyh i słabowidzących. Wdrożenie go w środowisku docelowym wymaga jednak dalszych badań. W szczególności należy przeprowadzić ankiety wśród użytkowników końcowych, które pozwolą na ocenę jego użyteczności i wpływu na proces nauki. Może to wskazać obszary wymagające poprawy i dalszego rozwoju.

Pełny zakres funkcjonalny aplikacji mobilnej został zrealizowany wyłącznie dla wersji przeznaczonej na system Android. Szkielet implementacyjny dla systemu iOS został

przygotowany, jednak nie zawiera ważnych funkcjonalności, co uniemożliwia jego użycie w przyjętym scenariuszu działania bez dalszych prac.

Cel pracy został osiągnięty w przyjętym zakresie. Stworzono działający prototyp systemu i przygotowano i oceniono jego kluczowy moduł detekcji obiektów. Jednocześnie zidentyfikowano obszary wymagające dalszych badań i rozwoju przed jego ewentualnym wdrożeniem w środowisku docelowym.

7.3 Najważniejsze wnioski

Realizacja pracy pozwoliła na sformułowanie kilku istotnych wniosków, które mogą stanowić podstawę do dalszych badań i rozwoju systemu.

Wpływ rozdzielczości obrazu wejściowego okazał się jednym z najistotniejszych czynników wpływających na skuteczność detekcji. Modele trenowane na obrazach o wyższej rozdzielczości wykazywały ogólną poprawę wyników względem wariantów używających obrazów o niższej rozdzielczości. Dla modelu YOLO26n poprawie uległy wszystkie badane metryki, natomiast dla modelu YOLO26s poprawa dotyczyła wyłącznie metryk precyzji, $mAP50$ i $mAP50 - 95$ i $F1$ -score przy jednoczesnym spadku czułości. Warto jednak zauważyć, że zwiększenie tego parametru wiąże się ściśle ze zwiększeniem wymagań obliczeniowych, prowadząc do wydłużenia czasu inferencji, dlatego jej wybór powinien uwzględnić również kompromis pomiędzy jakością detekcji a wydajnością systemu.

Zwiększenie skali modelu nie przyniosło jednoznacznej poprawy skuteczności detekcji. Przy mniejszych badanych obrazach wyższe warianty modeli wykazywały lepsze wyniki, wskazując na korzyści z zastosowania bardziej złożonych modeli. Dla obrazów o wyższej rozdzielczości ta zależność nie była już tak wyraźna: występowały wahania poszczególnych metryk, a w niektórych przypadkach mniejsze modele osiągały lepsze wyniki. Zmiana skali modelu nie zapewnia więc stałej przewagi, a zasadność jej stosowania powinna być oceniana w kontekście innych czynników.

Usunięcie informacji o kolorze z obrazów wejściowych poskutkowało znacznym spadkiem skuteczności detekcji badanych modeli. Analiza wyników poszczególnych klas wykazała jednak, że nie było ono jednakowe dla wszystkich klas. Wartości porównywalne z modelami pracującymi na obrazach kolorowych uzyskano dla klas sześcianu i prostopadłościanu. Najgorsze wyniki uzyskano dla klasy stożka. Wskazuje to na istotną rolę informacji o barwie, ale jej znaczenie może być różne w zależności od klasy obiektu.

Poszczególne zmiany konfiguracji modeli wpływały w różnym stopniu na zdolność rozróżniania klas obiektów. Szczególnie często mylone były obiekty o zbliżonym kształcie, jak sześcian i prostopadłościan, które w wielu przypadkach zostawały błędnie przypisane do siebie.

Uruchomienie detekcji na zbiorach testowych skutkowało osiągnięciem niższych wyników niż w przypadku zbiorów walidacyjnych. Ukazuje to znaczenie wykonywania testów

na danych, które nie uczestniczyły w procesie treningu. Warto również zauważyć, że dane w zbiorze testowym były niezależne od zbioru treningowego i walidacyjnego, przez co stanowiły bardziej niezależny materiał do oceny. Zaobserwowana różnica wskazuje również, że wyniki walidacyjne nie odzwierciedlały w pełni zdolności modeli do generalizacji.

Podczas wyboru modelu przeznaczonego do zastosowania w systemie należy zwrócić uwagę nie tylko na jego jakość detekcji, ale również na koszt obliczeniowy i czas inferencji. Wysoka wartość metryk detekcji nie musi oznaczać, że dany model będzie najlepszym wyborem w kontekście całego rozwiązania. Model, który został wybrany do implementacji w systemie, nie osiągał najwyższych wyników, ale wymagał mniejszej liczby zasobów obliczeniowych i charakteryzował się krótszym czasem inferencji od pozostałych.

Wnioski, jakie płyną z przeprowadzonych badań wskazują, że możliwe jest wykorzystanie metod detekcji obiektów w kontekście tematu pracy. Wskazują również na obszary wymagające dalszych badań i rozwoju, które mogą przyczynić się do zwiększenia skuteczności, a w przyszłości, być może, wdrożenia systemu w środowisku docelowym.

7.4 Możliwe kierunki rozwoju

W toku prac zidentyfikowano kilka obszarów, które mogą stanowić kierunki dalszego rozwoju systemu.

Najważniejszym z nich jest zdecydowanie powiększenie i zróżnicowanie zbioru danych. Rozmiary obecnego zbioru zostały ograniczone przez czas i możliwości przeprowadzenia nagrań, a także konieczność ręcznej adnotacji danych. W przyszłości warto rozważyć wykorzystanie w nagraniach większej liczby osób, pomieszczeń i warunków, co pozwoli na zwiększenie różnorodności materiałów. Warto również rozważyć wprowadzenie dodatkowych klas obiektów, jak np. pozostałych wielościanów foremnych, których własności geometryczne są często omawiane w ramach nauki geometrii przestrzennej. Zwiększenie liczby klas obiektów może również przyczynić się do zwiększenia użyteczności systemu, poprzez rozszerzenie dostępnych możliwości wsparcia.

Kolejnym kierunkiem, jaki należy podjąć jest rozszerzenie badań dotyczących wykorzystania modeli pracujących w odcieniach szarości. Warto przeprowadzić eksperymenty z dodatkowymi zestawami brył, aby zbadać wpływ ich kolorystyki na skuteczność detekcji. Niewykluczone, że dodanie do zbioru danych brył o różnych kolorach, mających ten sam kształt, pozwoliłby modelom na lepszą generalizację i zmniejszenie ich zależności od informacji o kolorze.

Następnie, warto wykonać dodatkowe eksperymenty z wykorzystaniem innych architektur sieci neuronowych, a także powtórzyć treningi z wykorzystaniem innych metod augmentacji danych i wartościami ziarna. Pozwoli to ocenić zmienność wyników i sprawdzić powtarzalność uzyskanych rezultatów. Warto również uzupełnić zbiór modeli o wariant YOLO26s pracujący na obrazach w odcieniach szarości i rozdzielczości 1024 pikseli. Takie

rozszerzenie badań pozwoliłoby dokładniej ocenić wpływ tych czynników na skuteczność detekcji.

Innym kierunkiem rozwoju jest zmiana sposobu analizy obrazów. Obecna wersja traktuje każdy obraz niezależnie. Zmiana tego podejścia na analizę sekwencji obrazów mogłaby ograniczyć wpływ chwilowego zasłonięcia obiektu, a także zwiększyć skuteczność detekcji poprzez wykorzystanie informacji zawartej w kilku kolejnych klatkach.

Odchodząc od samej detekcji, uwagę zwrócić należy na użytkowników końcowych. Badania przeprowadzone z osobami niewidomymi i słabowidzącymi pozwolą na ocenę użyteczności systemu oraz wskażą obszary wymagające poprawy. Zmiany interfejsu użytkownika mogłyby ograniczyć lub wyeliminować konieczność obecności operatora w trakcie korzystania z systemu, zwiększając jego samodzielność i komfort użytkowania. Warto również rozważyć wprowadzenie dodatkowych komend głosowych, które pozwoliłyby na sterowanie całym systemem w sposób bezdotykowy.

Kolejne kierunki rozwoju systemu obejmują integrację z innymi technologiami, takimi jak sensory LiDAR lub kamery RGB-D, jednakże ich użyteczność w kontekście pracy wymaga dalszych badań. Należy również rozwinąć moduł aplikacji klienckiej dla systemu iOS, aby umożliwić korzystanie z systemu jak największej liczbie użytkowników. Możliwe również, że w przyszłości, inferencja będzie mogła odbywać się lokalnie na urządzeniu mobilnym, co pozwoliłoby na zwiększenie prywatności użytkowników i odejście od konieczności stosowania jednostki serwerowej.

Wymienione kierunki rozwoju systemu stanowią jedynie propozycje, które mogą przyczynić się do zwiększenia jego skuteczności i użyteczności. Nie są to wszystkie możliwe kierunki, a jedynie te które zostały zidentyfikowane w toku realizacji pracy i mogą przynieść wymierne korzyści w przyszłości. Warto również podkreślić, że dalsze badania i rozwój powinny odbywać się w ścisłej współpracy z ośrodkami zajmującymi się wsparciem osób niewidomych i słabowidzących, aby zapewnić, że system będzie odpowiadał na rzeczywiste potrzeby użytkowników końcowych.

Bibliografia

- [1] Mirza Samad Ahmed Baig, Syeda Anshrah Gillani, Shahid Munir Shah, Mahmoud Aljawarneh, Abdul Akbar Khan i Muhammad Hamzah Siddiqui. *AI-based Wearable Vision Assistance System for the Visually Impaired: Integrating Real-Time Object Recognition and Contextual Understanding Using Large Vision-Language Models*. 2024. arXiv: 2412.20059 [cs.CV]. URL: <https://arxiv.org/abs/2412.20059>.
- [2] Fabiola Cando-Guanoluisa, Lesly Claudio, Dylan Cevallo, Carlos Colcha, Silvia Taípe i Grecia Pilatas. „Visually Impaired Students’ and Their Teacher’s Perceptions of the English Teaching and Learning Process”. W: *Mertesol Journal* 46 (sty. 2023), s. 1–14. DOI: 10.61871/mj.v46n4-3.
- [3] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov i Sergey Zagoruyko. *End-to-End Object Detection with Transformers*. 2020. arXiv: 2005.12872 [cs.CV]. URL: <https://arxiv.org/abs/2005.12872>.
- [4] R. Qi Charles, Hao Su, Mo Kaichun i Leonidas J. Guibas. „PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation”. W: *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, s. 77–85. DOI: 10.1109/CVPR.2017.16.
- [5] João Francisco Staffa Costa, V. M. R. Lima i M. S. Biembengut. „Spatial perception of a blind person: interactions and experiences”. W: *Brazilian Journal of Science Teaching and Technology, Ponta Grossa* 16 (2023), s. 1–20. DOI: 10.3895/rbect.v16n1.13944.
- [6] CVAT.ai Corporation. *CVAT: Complete Data Labeling Suite For Teams Building Real-World AI*. URL: <https://www.cvat.ai/>.
- [7] N. Dalal i B. Triggs. „Histograms of oriented gradients for human detection”. W: *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR’05)*. T. 1. 2005, 886–893 vol. 1. DOI: 10.1109/CVPR.2005.177.
- [8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit i Neil Houlsby. *An Image is Worth 16x16 Words:*

- Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV]. URL: <https://arxiv.org/abs/2010.11929>.
- [9] Ewa Dzierzgowska, Michał Maćkowski, Mateusz Kawulok, Piotr Brzoza, Stella Maćkowska i Dominik Spinczyk. „Alternative audio-graphic method for presenting structural information in mathematical graphs designed for low-vision users”. W: *Scientific Reports* 15.1 (2025), s. 21972. ISSN: 2045-2322. DOI: 10.1038/s41598-025-07710-2. URL: <https://doi.org/10.1038/s41598-025-07710-2>.
- [10] ETSI, CEN and CENELEC. *EN 301 549 V3.2.1: Accessibility Requirements for ICT Products and Services*. European Harmonised Standard. Mar. 2021. URL: https://www.etsi.org/deliver/etsi_en/301500_301599/301549/03.02.01_60/en_301549v030201p.pdf (term. wiz. 11.05.2026).
- [11] European Parliament and Council of the European Union. *Directive (EU) 2016/2102 of the European Parliament and of the Council of 26 October 2016 on the accessibility of the websites and mobile applications of public sector bodies*. Official Journal of the European Union, L 327, 2 December 2016, pp. 1–15. 2016. URL: <https://eur-lex.europa.eu/eli/dir/2016/2102/oj> (term. wiz. 11.05.2026).
- [12] European Parliament and Council of the European Union. *Directive (EU) 2019/882 of the European Parliament and of the Council of 17 April 2019 on the Accessibility Requirements for Products and Services*. Official Journal of the European Union, L 151, 7 June 2019, pp. 70–115. 2019. URL: <https://eur-lex.europa.eu/eli/dir/2019/882/oj> (term. wiz. 11.05.2026).
- [13] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn i Andrew Zisserman. „The PASCAL Visual Object Classes (VOC) Challenge.” W: *International Journal of Computer Vision* 88.2 (2010), s. 303–338. DOI: 10.1007/s11263-009-0275-4.
- [14] Lourdes Figueiras i Abraham Arcavi. „Learning to See: The Viewpoint of the Blind”. W: *Selected Regular Lectures from the 12th International Congress on Mathematical Education*. Red. Sung Je Cho. Cham: Springer International Publishing, 2015, s. 175–186. ISBN: 978-3-319-17187-6. DOI: 10.1007/978-3-319-17187-6_10.
- [15] Clement Godard, Oisín Mac Aodha, Michael Firman i Gabriel Brostow. „Digging Into Self-Supervised Monocular Depth Estimation”. W: *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, s. 3827–3837. DOI: 10.1109/ICCV.2019.00393.
- [16] Maşide Güler i Mustafa Ürey. „Are we aware of the needs of students with visual impairment? A study on challenges and instructional needs in science lessons”. W: *Journal of Pedagogical Sociology and Psychology* 7 (2 2025), s. 53–70. DOI: 10.33902/jpsp.202529253.

-
- [17] Kaiming He, Georgia Gkioxari, Piotr Dollár i Ross Girshick. *Mask R-CNN*. 2018. arXiv: 1703.06870 [cs.CV]. URL: <https://arxiv.org/abs/1703.06870>.
- [18] Intel Corporation. *New Intel-Created System Offers Professor Stephen Hawking Ability to Better Communicate with the World*. 2 grud. 2014. URL: <https://www.intel.com/news-events/press-releases/detail/380/new-intel-created-system-offers-professor-stephen-hawking> (term. wiz. 10.05.2026).
- [19] Yuichiro Ito i Yuki Nishimura. „Storage Medium Having Stored Therein Stereoscopic Image Display Program, Stereoscopic Image Display Device, Stereoscopic Image Display System, and Stereoscopic Image Display Method”. European Patent Application EP2395763A3. Ltd. Nintendo Co. i Inc. HAL Laboratory. 29 lut. 2012. URL: <https://patents.google.com/patent/EP2395763A3/en>.
- [20] Glenn Jocher, Jing Qiu, Mengyu Liu, Shuai Lyu, Fatih Cagatay Akyon i Muhammet Kalfaoglu. *Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models*. Czer. 2026. DOI: 10.48550/arXiv.2606.03748.
- [21] Franklin Mingzhe Li, Lotus Zhang, Maryam Bandukda, Abigale Stangl, Kristen Shinohara, Leah Findlater i Patrick Carrington. „Understanding Visual Arts Experiences of Blind People”. W: *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. CHI '23. Hamburg, Germany: Association for Computing Machinery, 2023. ISBN: 9781450394215. DOI: 10.1145/3544548.3580941.
- [22] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He i Piotr Dollár. *Focal Loss for Dense Object Detection*. 2018. arXiv: 1708.02002 [cs.CV]. URL: <https://arxiv.org/abs/1708.02002>.
- [23] Tsung-Yi Lin, Michael Maire, Serge J. Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár i C. Lawrence Zitnick. „Microsoft COCO: Common Objects in Context”. W: *Computer Vision – ECCV 2014*. T. 8693. Lecture Notes in Computer Science. Springer, 2014, s. 740–755. DOI: 10.1007/978-3-319-10602-1_48.
- [24] John G. Linvill. „Reading Aid for the Blind”. Pat. US 3,229,387 (United States). 18 sty. 1966. URL: <https://patents.google.com/patent/US3229387A/en>.
- [25] Haibin Liu, Chao Wu i Huanjie Wang. „Real time object detection using LiDAR and camera fusion for autonomous driving”. W: *Scientific Reports* 13.1 (2023), s. 8056. ISSN: 2045-2322. DOI: 10.1038/s41598-023-35170-z. URL: <https://doi.org/10.1038/s41598-023-35170-z>.
- [26] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu i Alexander C. Berg. „SSD: Single Shot MultiBox Detector”. W: *Computer Vision – ECCV 2016*. Springer International Publishing, 2016, s. 21–37. ISBN: 9783319464480. DOI: 10.1007/978-3-319-46448-0_2. URL: http://dx.doi.org/10.1007/978-3-319-46448-0_2.

- [27] David G. Lowe. „Distinctive Image Features from Scale-Invariant Keypoints”. W: *International Journal of Computer Vision* 60.2 (2004), s. 91–110. ISSN: 1573-1405. DOI: 10.1023/B:VISI.0000029664.99615.94. URL: <https://doi.org/10.1023/B:VISI.0000029664.99615.94>.
- [28] Zhengyi Ma, Jiayin Zhu, Jiayi Lu, Zhenliang Li, Xihan Cao, Jingyi Yao, Runjiu Hu i Genwang Liu. „Yolo-Point:Lidar-Image Fusion based on yolov5 and Pointnet++ for 3D Objection Perception”. W: sty. 2024.
- [29] R. G. Maling i D. C. Clarkson. „Electronic controls for the tetraplegic (possum) (patient operated selector mechanisms—P.O. S.M.)” W: *Spinal Cord* 1.3 (1963), s. 161–174. ISSN: 1476-5624. DOI: 10.1038/sc.1963.15. URL: <https://doi.org/10.1038/sc.1963.15>.
- [30] Mara Mills. „Hearing Aids and the History of Electronics Miniaturization”. W: *IEEE Annals of the History of Computing* 33.2 (2011), s. 24–45. DOI: 10.1109/MAHC.2011.43.
- [31] Rafael Padilla, Wesley L. Passos, Thadeu L. B. Dias, Sergio L. Netto i Eduardo A. B. da Silva. „A Comparative Analysis of Object Detection Metrics with a Companion Open-Source Toolkit”. W: *Electronics* 10.3 (2021). ISSN: 2079-9292. DOI: 10.3390/electronics10030279.
- [32] Kedar Potdar, Chinmay Pai i Sukrut Akolkar. „A Convolutional Neural Network based Live Object Recognition System as Blind Aid”. W: (list. 2018). DOI: 10.48550/arXiv.1811.10399.
- [33] Abhinav Pratap, Sushant Kumar i Suchinton Chakravarty. *Adaptive Object Detection for Indoor Navigation Assistance: A Performance Evaluation of Real-Time Algorithms*. 2025. arXiv: 2501.18444 [cs.CV]. URL: <https://arxiv.org/abs/2501.18444>.
- [34] Lutz Prechelt. „Early Stopping - But When?” W: *Neural Networks: Tricks of the Trade*. Red. Genevieve B. Orr i Klaus-Robert Müller. Berlin, Heidelberg: Springer Berlin Heidelberg, 1998, s. 55–69. ISBN: 978-3-540-49430-0. DOI: 10.1007/3-540-49430-8_3. URL: https://doi.org/10.1007/3-540-49430-8_3.
- [35] Charles Qi, Or Litany, Kaiming He i Leonidas Guibas. „Deep Hough Voting for 3D Object Detection in Point Clouds”. W: paź. 2019, s. 9276–9285. DOI: 10.1109/ICCV.2019.00937.
- [36] Charles R. Qi, Xinlei Chen, Or Litany i Leonidas J. Guibas. „ImVoteNet: Boosting 3D Object Detection in Point Clouds With Image Votes”. W: *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, s. 4403–4412. DOI: 10.1109/CVPR42600.2020.00446.

- [37] Charles R. Qi, Li Yi, Hao Su i Leonidas J. Guibas. „PointNet++: deep hierarchical feature learning on point sets in a metric space”. W: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS’17. Long Beach, California, USA: Curran Associates Inc., 2017, s. 5105–5114. ISBN: 9781510860964.
- [38] Rene Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler i Vladlen Koltun. „Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer”. W: *IEEE Transactions on Pattern Analysis & Machine Intelligence* 44.03 (mar. 2022), s. 1623–1637. ISSN: 1939-3539. DOI: 10.1109/TPAMI.2020.3019967. URL: <https://doi.ieeecomputersociety.org/10.1109/TPAMI.2020.3019967>.
- [39] Joseph Redmon, Santosh Divvala, Ross Girshick i Ali Farhadi. *You Only Look Once: Unified, Real-Time Object Detection*. 2016. arXiv: 1506.02640 [cs.CV]. URL: <https://arxiv.org/abs/1506.02640>.
- [40] Joseph Redmon i Ali Farhadi. *YOLOv3: An Incremental Improvement*. 2018. arXiv: 1804.02767 [cs.CV]. URL: <https://arxiv.org/abs/1804.02767>.
- [41] Shaoqing Ren, Kaiming He, Ross Girshick i Jian Sun. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. 2016. arXiv: 1506.01497 [cs.CV]. URL: <https://arxiv.org/abs/1506.01497>.
- [42] Santiago Royo i Maria Ballesta-Garcia. „An Overview of LiDAR Imaging Systems for Autonomous Vehicles”. W: *Applied Sciences* 9.19 (2019), s. 4093. DOI: 10.3390/app9194093. URL: <https://doi.org/10.3390/app9194093>.
- [43] Rzeczpospolita Polska. *Ustawa z dnia 26 kwietnia 2024 r. o zapewnianiu spełniania wymagań dostępności niektórych produktów i usług przez podmioty gospodarcze*. Dz.U. z 2024 r. poz. 731. 2024. URL: <https://isap.sejm.gov.pl/isap.nsf/DocDetails.xsp?id=WDU20240000731> (term. wiz. 11.05.2026).
- [44] Rzeczpospolita Polska. *Ustawa z dnia 4 kwietnia 2019 r. o dostępności cyfrowej stron internetowych i aplikacji mobilnych podmiotów publicznych*. Dz.U. z 2019 r. poz. 848; tekst jednolity: Dz.U. z 2023 r. poz. 1440, z późn. zm. 2019. URL: <https://isap.sejm.gov.pl/isap.nsf/DocDetails.xsp?id=WDU20190000848> (term. wiz. 11.05.2026).
- [45] Pierre Sermanet, David Eigen, Xiang Zhang, Michael Mathieu, Rob Fergus i Yann LeCun. *OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks*. 2014. arXiv: 1312.6229 [cs.CV]. URL: <https://arxiv.org/abs/1312.6229>.

- [46] Sibi C. Sethuraman, Gaurav R. Tadkapally, Saraju P. Mohanty, Gautam Galada i Anitha Subramanian. *MagicEye: An Intelligent Wearable Towards Independent Living of Visually Impaired*. 2023. arXiv: 2303.13863 [cs.HC]. URL: <https://arxiv.org/abs/2303.13863>.
- [47] Dandan Shan, Jiaqi Geng, Michelle Shu i David F. Fouhey. „Understanding Human Hands in Contact at Internet Scale”. W: 2020. arXiv: 2006.06669 [cs.CV]. URL: <https://arxiv.org/abs/2006.06669>.
- [48] Irwin Sobel. „An Isotropic 3x3 Image Gradient Operator”. W: *Presentation at Stanford A.I. Project 1968* (lut. 2014).
- [49] Richard Szeliski. *Computer Vision: Algorithms and Applications*. 2nd. Texts in Computer Science. Springer Cham, 2022. DOI: 10.1007/978-3-030-34372-9. URL: <https://szeliski.org/Book/>.
- [50] Bugra Tekin, Federica Bogo i Marc Pollefeys. „H+O: Unified Egocentric Recognition of 3D Hand-Object Poses and Interactions”. W: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, s. 4506–4515. DOI: 10.1109/CVPR.2019.00464.
- [51] Ultralytics. *Performance Metrics Deep Dive*. YOLO Performance Metrics documentation. 12 list. 2023. URL: <https://docs.ultralytics.com/guides/yolo-performance-metrics/> (term. wiz. 17.12.2025).
- [52] Ultralytics. *Ultralytics YOLO26*. 25 wrz. 2025. URL: <https://docs.ultralytics.com/models/yolo26> (term. wiz. 04.11.2025).
- [53] Ultralytics. *YOLO Architecture Explained: From YOLOv3 to YOLO26*. 6 lip. 2026. URL: <https://docs.ultralytics.com/guides/yolo-architecture> (term. wiz. 10.07.2026).
- [54] United Nations. *Convention on the Rights of Persons with Disabilities*. United Nations General Assembly Resolution A/RES/61/106. 2006. URL: <https://www.ohchr.org/en/instruments-mechanisms/instruments/convention-rights-persons-disabilities> (term. wiz. 11.05.2026).
- [55] Gregg Vanderheiden. „A journey through early augmentative communication and computer access”. W: *Journal of rehabilitation research and development* 39 (sty. 2003), s. 39–54.
- [56] P. Viola i M. Jones. „Rapid object detection using a boosted cascade of simple features”. W: *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001*. T. 1. 2001, s. I–I. DOI: 10.1109/CVPR.2001.990517.

-
- [57] Saisai Wang, Jiashuai Cui, Fan Li i Liejun Wang. „Image Sampling Based on Dominant Color Component for Computer Vision”. W: *Electronics* 12.15 (2023), s. 3360. ISSN: 2079-9292. DOI: 10.3390/electronics12153360.
 - [58] Wei Wang, Bin Jing, Xiaoru Yu, Yan Sun, Liping Yang i Chunliang Wang. „YOLO-OD: Obstacle Detection for Visually Impaired Navigation Assistance”. W: *Sensors* 24.23 (2024). ISSN: 1424-8220. DOI: 10.3390/s24237621. URL: <https://www.mdpi.com/1424-8220/24/23/7621>.
 - [59] World Wide Web Consortium. *Web Content Accessibility Guidelines 1.0*. W3C Recommendation. Maj 1999. URL: <https://www.w3.org/TR/WAI-WEBCONTENT/> (term. wiz. 11.05.2026).
 - [60] World Wide Web Consortium. *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. 2024. URL: <https://www.w3.org/TR/WCAG22/> (term. wiz. 11.05.2026).
 - [61] Jingxi Xu, Han Lin, Shuran Song i Matei Ciocarlie. *TANDEM3D: Active Tactile Exploration for 3D Object Recognition*. 2023. DOI: 10.48550/arXiv.2209.08772. arXiv: 2209.08772 [cs.CV]. URL: <https://arxiv.org/abs/2209.08772>.
 - [62] W. Xu, X. Du, R. Li i L. Xing. „DTC-YOLO: Multimodal Object Detection via Depth-Texture Coupling and Dynamic Gating Optimization”. W: *Sensors* 25.18 (2025), s. 5731. DOI: 10.3390/s25185731.
 - [63] Chenyangguang Zhang, Guanlong Jiao, Yan Di, Gu Wang, Ziqin Huang, Ruida Zhang, Fabian Manhardt, Bowen Fu, Federico Tombari i Xiangyang Ji. „MOHO: Learning Single-view Hand-held Object Reconstruction with Multi-view Occlusion-Aware Supervision”. W: 2024. arXiv: 2310.11696 [cs.CV]. URL: <https://arxiv.org/abs/2310.11696>.
 - [64] Xuefeng ZHANG, Shaojie ZHANG, Xin CHEN i Jinhua ZHANG. „A tactile glove for object recognition based on palmar pressure and joint bending strain sensing”. W: *Journal of Measurement Science and Instrumentation* 16.2 (2025), s. 173–185. DOI: 10.62756/jmsi.1674-8042.2025017. URL: <https://www.sciopen.com/article/10.62756/jmsi.1674-8042.2025017>.
 - [65] Zhongyuan Zhang, Lichen Lin i Xiaoli Zhi. „R-PointNet: Robust 3D Object Recognition Network for Real-World Point Clouds Corruption”. W: *Applied Sciences* 14 (kw. 2024), s. 3649. DOI: 10.3390/app14093649.
 - [66] Robert Zieliński. „Cyfrowa edukacja dziś:między szansą a wyzwaniem – studium porównawcze”. W: *EBIS* 2.80 (2023), s. 234–267. ISSN: 1643-8779. DOI: 10.24131/3247.230212.

- [67] Zhengxia Zou, Keyan Chen, Zhenwei Shi, Yuhong Guo i Jieping Ye. „Object Detection in 20 Years: A Survey”. W: *Proceedings of the IEEE* 111.3 (2023), s. 257–276. DOI: 10.1109/JPROC.2023.3238524.

Dodatki

Spis skrótów i symboli

- 3D Three-Dimensional – trójwymiarowy sposób reprezentacji obiektów lub przestrzeni,
- AP Average Precision – średnia precyzja wyznaczana na podstawie krzywej precision-recall,
- C2f CSP Bottleneck with two convolutions and feature fusion – blok CSP wykorzystujący dwie operacje konwolucji i fuzję cech,
- C2PSA Cross-Stage Partial Spatial Attention – blok CSP wykorzystujący mechanizm uwagi przestrzennej,
- C3 CSP Bottleneck with 3 Convolutions – blok sieci wykorzystujący mechanizm CSP i trzy operacje konwolucji,
- C3k2 C3k2 block – wariant bloku CSP wykorzystujący moduły C3k lub bloki typu bottleneck,
- CNN Convolutional Neural Network – konwolucyjna sieć neuronowa,
- COCO Common Objects in Context – nazwa zbioru i benchmarku Microsoft COCO,
- CPU Central Processing Unit – procesor centralny,
- CSP Cross Stage Partial – mechanizm częściowego podziału i ponownego łączenia map cech,
- DETR Detection Transformer – model detekcji obiektów wykorzystujący architekturę transformera,
- DFL Distribution Focal Loss – funkcja straty wykorzystywana w modelach detekcji obiektów,
- DTC-YOLO Depth-Texture Coupling Mechanism YOLO – multimodalny model detekcji wykorzystujący połączenie informacji o głębi i teksturze,
- F1 $F1$ -score – średnia harmoniczna precyzji i czułości,
- FN False Negative – fałszywy negatyw, obiekt rzeczywisty niewykryty przez model,

FP False Positive – fałszywy pozytywny, błędna detekcja zwrócona przez model,

FPN Feature Pyramid Network – sieć piramidy cech umożliwiająca analizę obiektów w wielu skalach,

FPS Frames Per Second – liczba kadrów na sekundę,

GFLOPs Giga Floating-Point Operations – liczba miliardów operacji zmiennoprzecinkowych,

GPU Graphics Processing Unit – procesor graficzny,

IoU Intersection over Union – miara stopnia pokrycia ramki przewidywanej przez model z ramką referencyjną,

JPEG Joint Photographic Experts Group – stratny format kompresji i zapisu obrazów,

LiDAR Light Detection and Ranging – technologia pomiaru odległości za pomocą światła laserowego,

mAP mean Average Precision – średnia wartość AP obliczana dla wszystkich klas,

*mAP*₅₀ mean Average Precision at IoU 0.50 – średnia wartość AP wyznaczana przy progu IoU równym 0,5,

*mAP*₅₀₋₉₅ mean Average Precision at IoU 0.50–0.95 – średnia wartość AP obliczana dla progów IoU od 0,5 do 0,95,

MPS Metal Performance Shaders – framework do obliczeń równoległych na procesorach graficznych Apple,

NMS Non-Maximum Suppression – algorytm usuwania nakładających się ramek detekcji,

NSD Network Service Discovery – mechanizm wykrywania usług sieciowych w sieci lokalnej,

p.p. punkt procentowy,

PAN Path Aggregation Network – sieć agregacji ścieżek służąca do wieloskalowego łączenia cech,

PR Precision-Recall – zależność między precyzją a czułością,

Qt Qt – framework do tworzenia aplikacji wieloplatformowych,

R-CNN Region-based Convolutional Neural Network – konwolucyjna sieć neuronowa analizująca proponowane regiony obrazu,

- RGB Red-Green-Blue – przestrzeń kolorów reprezentowana przez trzy kanały: czerwony, zielony i niebieski,
- RGB-D Red-Green-Blue and Depth – reprezentacja danych zawierająca obraz kolorowy i informację o głębi,
- RT Real-Time – czas rzeczywisty,
- SPPF Spatial Pyramid Pooling – Fast – szybki moduł przestrzennego grupowania piramidowego,
- SSD Single Shot MultiBox Detector – model detekcji jednoetapowej obiektów w czasie rzeczywistym,
- TP True Positive – prawdziwy pozytywny, poprawnie wykryty obiekt,
- VI3DR Visually Impaired 3D Recognition – system wspomagający rozpoznawanie obiektów w 3D dla osób z niepełnosprawnością wzroku,
- VOC Visual Object Classes – zbiór nazw klas obiektów w benchmarku PASCAL VOC,
- WCAG Web Content Accessibility Guidelines – wytyczne dotyczące dostępności treści internetowych,
- YOLO You Only Look Once – rodzina modeli detekcji jednoetapowej obiektów w czasie rzeczywistym,

Lista dodatkowych plików, uzupełniających tekst pracy (jeżeli dotyczy)

W systemie do pracy dołączono dodatkowe pliki zawierające:

- źródła programu,
- zbiory danych użyte w eksperymentach,
- film pokazujący działanie opracowanego oprogramowania lub zaprojektowanego i wykonanego urządzenia,
- itp.

Spis rysunków

2.1	Uproszczony schemat architektury pierwszego modelu YOLO [39].	23
2.2	Uproszczony schemat architektury detektora YOLO26 [20].	25
3.1	Diagram blokowy architektury systemu	38
3.2	Bryły geometryczne wykorzystywane w projektowanym systemie	40
3.3	Diagram sekwencji przepływu danych w projektowanym systemie	42
4.1	Diagram komponentów aplikacji mobilnej z podziałem na kod współdzielony i implementacje platformowe	47
4.2	Interfejs aplikacji mobilnej	49
4.3	Interfejs graficzny serwera	55
5.1	Rozkład klas oraz położenia ramek ograniczających w zbiorze treningowym.	65
5.2	Warianty danych wejściowych.	68
6.1	Przebiegi treningu modeli YOLO26 na obrazach kolorowych o rozdzielczości 640 pikseli.	73
6.2	Wizualizacja różnicy rozmiaru obiektu na obrazie w zależności od rozdzielczości obrazu wejściowego.	75
6.3	Wybrane znormalizowane macierze pomyłek badanych modeli.	82
6.4	Krzywe precision–recall modeli E_3 i E_7 uzyskane na zbiorze testowym. . .	84
6.5	Krzywa $F1$ dla wariantu E_3	86

Spis tabel

2.1	Porównanie wybranych metod detekcji obiektów	18
2.2	Wybrane etapy rozwoju architektury modeli YOLO. Opracowanie własne na podstawie [20, 39, 40, 53].	24
2.3	Wybrane zadania obsługiwane przez modele YOLO26	26
2.4	Porównanie wariantów detekcyjnych YOLO26 na zbiorze COCO dla obrazów 640×640 . Opracowanie własne na podstawie [52].	27
4.1	Najważniejsze technologie wykorzystane w poszczególnych częściach systemu	46
4.2	Podstawowe komunikaty protokołu klient-serwer	58
5.1	Podział zbioru danych na zbiory treningowy, walidacyjny i testowy.	64
5.2	Podział nagrań wideo pomiędzy zbiory treningowy, walidacyjny i testowy.	66
6.1	Macierz modeli wytrenowanych w ramach badań.	71
6.2	Zestawienie wyników detekcji modeli na zbiorze testowym.	72
6.3	Porównanie jakości wariantów modeli YOLO26 n i s w eksperymencie A.	74
6.4	Porównanie jakości modeli dla rozmiarów obrazu wejściowego 640 i 1024 pikseli w eksperymencie B.	75
6.5	Porównanie jakości modeli pracujących na obrazach RGB i w odcieniach szarości w eksperymencie C.	77
6.6	Porównanie wybranych konfiguracji modeli pod względem jakości i kosztu obliczeniowego.	79